

台灣自編大學教科書

[@prof](#)

機器人學導論

機構、感知、控制與具身人工智慧

第 五 篇

智慧機器人與具身人工智慧

第 29 章 ROS 2 與機器人軟體框架

第 30 章 模擬與 Sim-to-Real

第 31 章 機器學習與強化學習

第 32 章 大型模型與任務規劃

第 33 章 具身人工智慧

編著：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

2026 年 9 月

第五篇 篇章導讀

前四篇讓機器人**做得到**，這一篇要讓它**知道該做什麼**。

經過二十八章，機器人已經具備了完整的物理能力：它能感知環境、精確地運動、與環境安全地互動、在未知空間中導航、能飛行也能行走。但這些能力全都建立在一個前提上——**人類明確地告訴它該做什麼**。軌跡是人規劃的、任務是人指定的、行為是人設計的。

第五篇的五章，就是要補上這最後一塊拼圖。

第 29 章 (ROS 2) 先解決一個看似平凡卻極為關鍵的問題：**軟體整合**。四組學生各自做出了不錯的成果，卻接不起來——因為語言不同、座標系定義不同、時間基準不同。ROS 2 提供的不是演算法，而是一套讓不同模組能夠對話的約定。它也是後續四章的共同基礎。

第 30 章 (模擬與 Sim-to-Real) 處理一個殘酷的現實：模擬中 98% 的成功率，到實機可能只剩兩成。但本章的答案出乎意料——**不是把模擬做得更真，而是刻意把模擬弄亂**。領域隨機化的思想是：與其縮小模擬與現實的距離，不如擴大模擬的涵蓋範圍，使現實成為其中一個樣本。

第 31 章 (機器學習與強化學習) 回答一個根本的問題：**當問題寫不出方程式時怎麼辦？** 逆運動學可以算到微米級，但摺一件 T 恤至今無人能及五歲小孩——因為衣服沒有方程式。這一章也回收了第 18 章最重要的伏筆：**強化學習與最佳控制共用同一條 Bellman 方程**，差別只在於模型是否已知。

第 32 章 (大型模型與任務規劃) 讓機器人第一次能「聽懂人話」。「幫我把桌上的杯子拿到廚房」這句話背後，藏著三十年來寫不完的常識——而大型語言模型從文字中學到了它們。但它也帶來新的危險：**它會非常有自信地說錯話**，而在機器人上，說錯就是一個真實的動作。

第 33 章 (具身人工智慧) 是全書理論部分的收束。它回到第 1 章的 Moravec 悖論，給出三層答案，並提出一個統一的觀點：**智慧不只在腦裡，也在身體裡，還在身體與環境的互動裡**。

這一篇要回答的六個問題

- 四組的成果都不錯，為什麼接不起來？
- 模擬裡 98% 的成功率，為什麼到實機只剩兩成？
- 算逆運動學能到微米級，為什麼摺不好一件 T 恤？

- 「幫我把桌上的杯子拿到廚房」——這句話難在哪裡？
- 為什麼電腦下棋能贏世界冠軍，卻摺不好一件衣服？
- 一隻蜜蜂的腦只有一百萬個神經元，為什麼能做到我們的機器人做不到的事？

三條貫穿全篇的線索

線索一：模型提供保證，學習提供泛化。這是本篇最核心的分工原則。第 31 章明確指出：能寫出方程式的就用模型（更準、更可靠、不需資料），寫不出來的才用學習。第 32 章的架構更是三者並用——語言模型提供常識、符號規劃提供正確性、底層控制提供安全。**不是誰取代誰，而是各自處理擅長的部分。**

線索二：智慧可以是機率性的，但安全必須是確定性的。學習與大型模型都沒有保證、都可能在分布外做出不可預測的行為。因應之道不是讓它們變得更可靠，而是**在其外加上一層模型式的、可驗證的安全機制**。一個力矩限制不該由神經網路判斷，而應該是一行明確的 if 敘述。

線索三：與其消除不確定性，不如設計對它不敏感的系統。這條主線從第 20 章的柔順控制、第 21 章的軟性夾爪一路延伸到第 30 章的領域隨機化，並在第 33 章以「形態計算」得到統一的解釋——**當身體的結構與任務相匹配時，大量的計算可以由物理本身完成。**

讀完這一篇之後

你將能夠：以 ROS 2 建立完整的機器人軟體系統、設計模擬環境並跨越 Sim-to-Real 落差、運用監督式學習與強化學習訓練機器人技能、以大型模型實現語言驅動的任務規劃、並為系統設計三層安全防護。

更重要的是，你將獲得三種判斷力：**知道何時該用哪個工具**（模型或學習、精確或柔順、規劃或反應）、**知道每個方法的邊界**（在什麼假設下成立、在什麼情況下失效）、以及**知道問題的本質**——很多時候，正確的答案不是更好的演算法，而是重新定義問題。

第 33 章的最後留下了一句話：**現實會回應你，這是機器人學最好的老師。**而第六篇「系統整合、產業應用與專題」將帶你真正走進現實——把這三十三章的知識，變成一個能運作的系統。

本篇章節配置

章次	章名	核心內容	叫獸開講
第 29	ROS 2 與機器	節點與話題、服務與動作、參數與啟動檔、	「四組的成果都不錯，為什麼接不起

章次	章名	核心內容	叫獸開講
章	人軟體框架	TF2、URDF、QoS、工具生態	來？」——整合的困難平方成長
第 30 章	模擬與 Sim-to-Real	物理引擎、保真度、落差的五個來源、領域隨機化、系統辨識、平行模擬	「模擬 98%，實機只剩兩成？」——模擬不必更真，策略必須更穩
第 31 章	機器學習與強化學習	三種典範、模仿學習與 DAgger、MDP、Bellman 方程、獎勵設計、PPO	「算逆運動學到微米級，為什麼摺不好 T 恤？」——衣服沒有方程式
第 32 章	大型模型與任務規劃	符號接地、PDDL、語言模型作為規劃器、SayCan、VLM 與 VLA、三層防護	「幫我把杯子拿到廚房」——那些寫不完的常識
第 33 章	具身人工智慧	Moravec 悖論的三層答案、具身認知、形態計算、主動感知、倫理與社會	「蜜蜂只有一百萬個神經元卻能飛行」——智慧不只在腦裡

第五篇 智慧機器人與具身人工智慧

第 29 章

ROS 2 與機器人軟體框架

編著：李世淵（ 機器人叫獸 ）

助教：李天宇、李宇晴

【章首情境圖建議】左半為兩種架構的對照：上圖是「點對點」——八個模組以雜亂交錯的線兩兩相連，共二十八條線，以紅色標註「介面平方成長」；下圖是「發布訂閱」——同樣八個模組各自連到中央的話題匯流排，線條清爽，標註「模組完全解耦」。右半為一台服務機器人的節點圖：以圓圈代表節點（驅動、SLAM、導航、辨識、手臂規劃、任務管理），以箭頭與話題名稱（`/scan`、`/map`、`/cmd_vel`）連接，並在旁側標出三種通訊模式的圖示。下緣以一棵 TF 樹（`map` → `odom` → `base_link` → 各感測器）收尾。標題文字：讓不同的程式能夠對話。

叫獸開講 「四組的成果都不錯，為什麼接不起來？」

學期末的專題發表會上，四組學生輪流上台。第一組做了視覺辨識、第二組做了機械手臂控制、第三組做了移動底盤導航、第四組做了語音介面。每一組單獨看都做得不錯。

叫獸聽完後問了一個問題：「現在，請你們把四組的成果組合起來，做成一台『聽到指令就去拿東西』的服務機器人。需要多久？」

教室安靜了。小翰苦笑：「視覺組用 Python 3.10、手臂組用 C++、底盤組寫在 Arduino 上、語音組是 Node.js.....而且大家的座標系定義都不一樣。」

「所以答案是？」「.....可能要重寫一半。」

「這就是機器人軟體最大的問題。」叫獸在白板上寫下一個詞：**整合**。「每一個子系統都不難，但把它們接起來極其困難。而且這個困難會隨著模組數量平方成長——三個模組要接三條線，十個模組要接四十五條。」

他停頓一下：「更糟的是，這個工作沒有任何學術價值。沒有人會因為『我把視覺和手臂接起來了』而發表論文。但如果不做，什麼系統都做不出來。」

小矧舉手：「所以有現成的解法嗎？」

「有，而且它改變了整個領域。」叫獸寫下三個字母：**ROS**。「機器人作業系統。但它的名字有點誤導——它不是作業系統，它是一套讓不同的程式能夠互相溝通的約定。」

他在白板上畫了幾個圓圈，用線連起來：「它的核心思想極為簡單：把每個功能寫成一個獨立的程式（節點），讓它們透過標準化的訊息互相溝通。視覺節點發布『我看到一個杯子在這個位置』，手臂節點訂閱這個訊息並執行抓取。兩者不需要知道對方是誰、用什麼語言寫的、甚至不需要在同一台電腦上。」

「這個設計解決的不只是技術問題，」叫獸最後說，「它解決的是協作問題。全世界的實驗室可以各自開發一個節點，然後互相使用。這就是為什麼今天你可以下載一個現成的 SLAM 套件、接上你的光達，十分鐘就跑起來——而這件事在二十年前，需要一個博士生做兩年。」

本章為什麼重要

前面二十八章建立了機器人的理論與能力：運動學、動力學、控制、感知、定位、規劃。但

這些知識要變成一台能運作的機器人，還缺少一樣東西——**把它們組織起來的軟體架構**。

這個問題常被低估。在教科書中，各章的內容是獨立的；但在真實的機器人上，它們必須**同時運行、互相通訊、共享時間基準、並在某個環節失效時仍能安全處置**。一個完整的機器人系統可能包含數十個並行的程式，處理著不同頻率的資料流（第 3 章的時間尺度金字塔）。

ROS 2 是目前這個問題的標準答案。它提供了三樣東西：**一套通訊機制**（讓模組能互相傳訊息）、**一套工具生態**（視覺化、記錄、除錯、模擬）、以及**一個社群**（數千個現成的套件可以直接使用）。學會它，就等於獲得了整個機器人社群數十年的累積。

本章也是全書的一個轉折點。前四篇教的是「**機器人學的知識**」，本章開始則是「**如何把知識變成系統**」——而這正是第五篇「**智慧機器人**」的必要基礎：第 30 章的模擬、第 31 章的學習、第 32 章的大型模型，全部都建立在這套軟體架構之上。

學習目標

- 能說明機器人軟體整合的困難，以及框架所解決的問題。
- 能說明 ROS 的核心設計哲學與其三個貢獻。
- 能比較 ROS 1 與 ROS 2 的架構差異及其動機。
- 能說明節點、話題、訊息的概念並實作發布與訂閱。
- 能區分話題、服務與動作三種通訊模式並選擇合適者。
- 能說明參數與啟動檔的作用並撰寫基本的啟動檔。
- 能運用 TF2 管理座標變換並理解其時間查詢機制。
- 能撰寫 URDF 描述機器人並在 RViz 中視覺化。
- 能說明 QoS 設定的意義並為不同的資料流選擇適當設定。
- 能運用 rosbag、rqt 與 RViz 進行記錄與除錯。
- 能說明即時性的需求與 ROS 2 的因應機制。
- 能設計一個完整機器人系統的節點架構。

先備知識

第 3 章的系統架構與時間尺度、第 8 章的座標系與齊次變換（TF2 的基礎）、第 4 章的連桿

與關節（URDF 的基礎）、第 7 章的嵌入式與即時性概念。

程式方面需要 Python 或 C++ 的基本能力（本章以 Python 為主）、以及 Linux 命令列的基本操作。**不需要**事先接觸過 ROS。若對 Linux 生疏，可先閱讀附錄 C。

核心名詞中英對照

表 29-1 本章核心名詞

中文	英文	說明
機器人作業系統	Robot Operating System, ROS	機器人軟體的中介層框架
節點	Node	執行特定功能的獨立程式單元
話題	Topic	多對多的非同步訊息通道
發布者 / 訂閱者	Publisher / Subscriber	發送與接收話題訊息的角色
訊息	Message	標準化的資料結構定義
服務	Service	一對一的同步請求回應模式
動作	Action	長時間執行且可回報進度的任務模式
參數	Parameter	節點的可設定值
啟動檔	Launch File	一次啟動多個節點的設定腳本
座標變換樹	TF2	管理各座標系關係的系統
統一機器人描述格式	URDF	描述機器人結構的 XML 格式
服務品質	Quality of Service, QoS	通訊可靠度與時效性的設定
資料分發服務	DDS	ROS 2 底層的通訊中介軟體
工作空間	Workspace	存放與編譯套件的目錄結構
套件	Package	ROS 程式的組織單位
生命週期節點	Lifecycle Node	具有明確狀態轉換的受管節點
執行器	Executor	決定回呼如何被執行的排程器
回呼群組	Callback Group	控制回呼並行方式的機制

29.1 為何需要框架

29.1.1 整合的困難

如開頭故事所述，機器人軟體最大的困難不在單一功能，而在**整合**。這個困難有四個具體的

來源。

來源一：介面的組合爆炸。若有 n 個模組要互相溝通，最壞情況需要 $n(n-1)/2$ 條介面。三個模組要 3 條，十個模組要 45 條——而且每一條都要各自定義資料格式、錯誤處理與同步機制。

來源二：異質性。不同的模組可能用不同的語言（C++ 求效能、Python 求開發速度）、跑在不同的硬體（PC、嵌入式板、微控制器）、甚至在不同的機器上（機器人本體與遠端工作站）。

來源三：時間與並行。第 3 章的時間尺度金字塔說明了機器人系統中並存著多種頻率——控制迴路 1 kHz、感知 30 Hz、規劃 1 Hz。這些必須並行執行且不互相阻塞。

來源四：除錯的困難。當系統由數十個並行程式構成時，「哪裡出錯了」極難判斷。若沒有工具能觀察系統內部的資料流，除錯幾乎不可能。

29.1.2 框架提供了什麼

一個機器人軟體框架，本質上是提供**四樣東西**：

表 29-2 機器人框架的四項核心功能

功能	解決的問題	ROS 2 中的對應
通訊機制	模組間如何傳資料（來源一、二）	話題、服務、動作
標準介面	資料格式如何統一	訊息類型（msg）、TF2、URDF
執行管理	多個程式如何啟動與並行（來源三）	啟動檔、執行器、生命週期
工具生態	如何觀察與除錯（來源四）	rqt、RViz、rosbag、ros2 CLI

◎ 為何不自己寫一套就好

許多團隊在初期會覺得「ROS 太複雜，我自己寫一個簡單的訊息機制就好」。這在專案初期確實較快，但通常在三個月後就會後悔。原因是：**你會逐步重新發明 ROS 的每一個功能**——先是需要記錄資料（於是寫一個 log 系統）、再需要視覺化（於是寫一個 GUI）、接著需要處理座標轉換（於是寫一個 TF）、然後需要多機通訊……而每一項你自己寫的版本，都不如 ROS 成熟，且**無法使用社群的任何現成套件**。這個決策的關鍵不在於「ROS 好不好」，而在於**你是否要放棄整個生態系**。

29.1.3 ROS 不是作業系統

必須釐清一個常見的誤解：**ROS 不是作業系統**。它跑在 Linux（或其他 OS）之上，是一個中介層（middleware）。

它之所以叫「作業系統」，是因為它提供了類似 OS 的服務：行程管理（啟動與監控節點）、行程間通訊（話題與服務）、硬體抽象（標準化的感測器與致動器介面）、以及套件管理。但底層的排程、記憶體管理、驅動程式，仍然由真正的作業系統負責。

29.2 ROS 1 到 ROS 2

29.2.1 ROS 1 的成功與限制

ROS 1 於 2007 年由 Willow Garage 推出，徹底改變了機器人研究。它的成功來自兩點：**開源**（任何人都能使用與貢獻）與**模組化**（節點可以獨立開發與重用）。

但隨著應用由研究走向產品，ROS 1 的四個限制逐漸浮現：

- **單點失效**：ROS 1 需要一個中央的 `roscore` (Master) 來協調節點的連線。若它掛掉，整個系統瓦解。
- **無即時性保證**：底層的通訊機制不支援即時排程，無法用於安全關鍵的控制迴路。
- **無安全機制**：訊息以明文傳輸，任何人接上網路都能讀取或注入指令。
- **多機器人支援薄弱**：跨網段、跨機器的通訊需要複雜的設定。

29.2.2 ROS 2 的三個關鍵改變

ROS 2 於 2017 年推出，針對上述問題做了根本性的重新設計。

改變一：底層改用 DDS。DDS (Data Distribution Service) 是一套工業標準的通訊中介軟體，原本用於航空、國防與工業自動化。它提供了三樣 ROS 1 沒有的東西：

- **去中心化的探索**：節點透過廣播自動找到彼此，**不需要中央的 Master**。這消除了單點失效。
- **QoS 設定**：可以為每個通道指定可靠度、時效性與歷史深度 (29.6 節)。
- **內建安全機制**：支援加密、認證與存取控制。

改變二：支援即時性。ROS 2 的設計避免了動態記憶體配置與不確定的鎖定，配合即時作業系統 (如 PREEMPT_RT) 可用於控制迴路。

改變三：生命週期管理。引入了「受管節點」 (managed node) 的概念——節點有明確的狀態 (未設定、非活動、活動、完成)，可以被外部協調地啟動、暫停與重設。這對產品級的

系統極為重要。

表 29-3 ROS 1 與 ROS 2 的比較

面向	ROS 1	ROS 2
中央協調	需要 roscore (單點失效)	去中心化 (DDS 探索)
通訊層	自訂的 TCPROS/UDPROS	DDS (工業標準)
即時性	不支援	可支援 (配合 RTOS)
安全性	無 (明文)	加密、認證、存取控制
QoS 設定	無	完整支援
多平台	主要 Linux	Linux、Windows、macOS、RTOS
生命週期	無	受管節點
Python 版本	Python 2 (舊版)	Python 3
現況	已停止支援 (2025 EOL)	現行標準

本書一律使用 ROS 2。ROS 1 的最後一個版本 Noetic 已於 2025 年停止支援，新的專案不應再使用。

29.3 核心概念：節點與話題

29.3.1 節點

節點 (node) 是 ROS 中最基本的執行單位——一個執行特定功能的獨立程式。典型的節點包括：

- **驅動節點**：讀取感測器或控制致動器 (如光達驅動、馬達控制器)。
- **處理節點**：對資料做運算 (如影像辨識、SLAM、路徑規劃)。
- **介面節點**：與人或外部系統互動 (如遙控器接收、網頁介面)。

設計節點的核心原則是「單一職責」——每個節點只做一件事。這使得節點容易測試、重用與替換。若一個節點同時做了影像辨識與路徑規劃，就應該考慮拆分。

29.3.2 話題：非同步的多對多通訊

話題 (topic) 是節點之間最主要的溝通方式。它的運作模式是**發布—訂閱** (publish-subscribe)：

發布者 → [話題名稱] → 訂閱者 (可以有多個)

發布者不知道有誰在訂閱、訂閱者也不知道是誰在發布。兩者完全解耦。

這個解耦是 ROS 最重要的設計。它帶來三個好處：

- **可替換性**：可以把真實的相機驅動換成模擬器，只要發布同樣的話題，下游節點完全不需修改——這是第 30 章 Sim-to-Real 的基礎。
- **可觀察性**：任何話題都可以被額外訂閱來監看或記錄，不影響原本的資料流。
- **可擴充性**：新增一個訂閱者不需要修改發布者。

29.3.3 訊息類型

話題上傳輸的資料有明確的**類型** (message type)，以`.msg`檔定義。ROS 2 提供了大量的標準訊息，這是「標準介面」的具體體現：

表 29-4 常用的標準訊息類型

訊息類型	內容	典型用途
geometry_msgs/Twist	線速度與角速度 (各三維)	移動機器人的速度指令
sensor_msgs/LaserScan	一圈的距離讀值陣列	二維光達
sensor_msgs/Image	影像資料	相機
sensor_msgs/PointCloud2	三維點雲	三維光達、深度相機
sensor_msgs/Imu	角速度、加速度、姿態	IMU
sensor_msgs/JointState	各關節的位置、速度、力矩	機械手臂狀態
nav_msgs/Odometry	位姿與速度 (含協方差)	里程計
nav_msgs/OccupancyGrid	佔據柵格地圖	SLAM 輸出 (第 25 章)
geometry_msgs/PoseStamped	帶時間戳的位姿	目標點、定位結果

標準訊息的價值在於**互通性**。因為所有的移動機器人都用`Twist`接收速度指令，所以任何導航套件都能驅動任何底盤——這正是社群能夠共用套件的關鍵。

29.3.4 最簡單的例子

以下是一個發布者與訂閱者的完整範例 (Python)：

```
# publisher_node.py — 發布速度指令
import rclpy
from rclpy.node import Node
```

```

from geometry_msgs.msg import Twist

class VelocityPublisher(Node):
    def __init__(self):
        super().__init__('velocity_publisher')    # 節點名稱
        # 建立發布者：話題名稱 'cmd_vel'、佇列深度 10
        self.pub = self.create_publisher(Twist, 'cmd_vel', 10)
        # 建立定時器：每 0.1 秒呼叫一次 (10 Hz)
        self.timer = self.create_timer(0.1, self.timer_callback)
        self.get_logger().info('速度發布節點已啟動')

    def timer_callback(self):
        msg = Twist()
        msg.linear.x = 0.2    # 前進 0.2 m/s
        msg.angular.z = 0.1   # 左轉 0.1 rad/s
        self.pub.publish(msg)

def main():
    rclpy.init()
    node = VelocityPublisher()
    rclpy.spin(node)        # 進入事件迴圈，等待定時器觸發
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

```

# subscriber_node.py — 訂閱並顯示速度指令
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class VelocitySubscriber(Node):
    def __init__(self):
        super().__init__('velocity_subscriber')
        # 建立訂閱者：收到訊息時呼叫 listener_callback
        self.sub = self.create_subscription(
            Twist, 'cmd_vel', self.listener_callback, 10)

    def listener_callback(self, msg):
        self.get_logger().info(
            f'收到指令: v={msg.linear.x:.2f} m/s, '
            f'w={msg.angular.z:.2f} rad/s')

def main():
    rclpy.init()
    node = VelocitySubscriber()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

```

```
if __name__ == '__main__':
    main()
```

注意兩個節點之間沒有任何直接的引用——它們只透過話題名稱 `cmd\_vel` 與訊息類型 `Twist` 這個「約定」連接。這正是解耦的具體體現。

29.4 服務與動作

29.4.1 三種通訊模式的分工

話題適合**持續的資料流**（感測器讀值、狀態回報），但不適合所有情況。ROS 2 提供三種通訊模式：

表 29-5 三種通訊模式的比較

模式	特性	是否等待回應	適用情境	範例
話題 Topic	多對多、非同步、持續	否	資料流、狀態廣播	感測器讀值、速度指令
服務 Service	一對一、同步、單次	是（阻塞）	快速的請求回應	查詢參數、觸發校正
動作 Action	一對一、非同步、可回報進度	是（非阻塞）	長時間任務	導航到目標、執行軌跡

29.4.2 服務

服務（service）是「問一個問題，等一個答案」的模式。它由**請求**（request）與**回應**（response）兩部分構成。

典型的用途包括：查詢機器人的狀態、切換模式、觸發一次性的動作（如拍照、校正）。

服務的**關鍵限制是它會阻塞**——呼叫者必須等待回應才能繼續。因此**絕不應該用服務執行長時間的任務**。若一個服務要花五秒才回應，呼叫的節點就會卡住五秒，可能導致整個系統的反應遲鈍。

29.4.3 動作

動作（action）正是為了解決這個問題而設計的。它適用於「**需要一段時間、且過程中想知道進度、還可能需要中途取消**」的任務。

動作由三部分構成：

- **目標 (Goal)**：要執行什麼。例如「導航到 (5, 3)」。
- **回饋 (Feedback)**：執行過程中持續回報。例如「已走 60%、剩餘距離 2.1 公尺」。
- **結果 (Result)**：完成後的最終回報。例如「成功抵達」或「失敗：路徑被阻擋」。

此外，動作還支援**取消 (cancel)**——呼叫者可以在中途要求中止。這對機器人極為重要：規劃好的導航可能因為突發狀況需要放棄。

最典型的動作範例是 Nav2 的導航介面（第 26.8.2 節）。上層下達「去廚房」的目標，導航系統持續回報進度，若途中被人擋住太久則回報失敗，而上層也可以隨時取消。

◎ 選擇通訊模式的判準

問三個問題：一、需要回應嗎？不需要 → 話題。二、會很快回應嗎（毫秒級）？是 → 服務。

三、需要進度回報或可能取消嗎？是 → 動作。一個常見的錯誤是濫用服務——把「移動手臂到某個姿態」寫成服務，結果呼叫者被卡住數秒。這類任務應該用動作。反之，也有人把單純的狀態查詢寫成動作，徒增複雜度。選對模式，程式碼會自然變簡單。

29.5 參數與啟動檔

29.5.1 參數：把設定與程式碼分離

一個節點通常有許多可調的設定：控制增益、感測器的裝置路徑、更新頻率、機器人的尺寸。這些不應該寫死在程式碼裡——否則每改一個數值就要重新編譯。

ROS 2 的參數 (parameter) 機制把設定外部化。節點在啟動時宣告自己需要哪些參數，這些參數可以由啟動檔、YAML 檔或命令列指定，甚至可以在執行中動態修改。

```
# 節點中宣告與讀取參數
class ControllerNode(Node):
    def __init__(self):
        super().__init__('controller')
        # 宣告參數並設定預設值
        self.declare_parameter('kp', 2.0)
        self.declare_parameter('kd', 0.5)
        self.declare_parameter('max_speed', 1.0)
        self.declare_parameter('frame_id', 'base_link')

        # 讀取參數值
        self.kp = self.get_parameter('kp').value
        self.kd = self.get_parameter('kd').value
        self.get_logger().info(f'控制增益: Kp={self.kp}, Kd={self.kd}')
```

```
# 註冊回呼，支援執行中動態調參
self.add_on_set_parameters_callback(self.param_callback)

def param_callback(self, params):
    from rcl_interfaces.msg import SetParametersResult
    for p in params:
        if p.name == 'kp':
            self.kp = p.value
            self.get_logger().info(f'Kp 已更新為 {self.kp}')
    return SetParametersResult(successful=True)
```

動態調參的價值極大。第 16 章的 PID 調校、第 26 章的導航參數、第 27 章的飛控增益——這些都需要反覆試驗。若每次都要重新編譯與重啟，調參會極為緩慢；有了動態參數，可以在機器人運行中即時調整並觀察效果。

29.5.2 YAML 參數檔

參數通常以 YAML 檔管理，這使得不同的機器人或不同的場景可以有各自的設定檔：

```
# config/controller_params.yaml
controller:
  ros__parameters:
    kp: 3.5
    kd: 0.8
    max_speed: 0.6
    frame_id: "base_link"

# 不同場景可以有不同的檔案
# config/controller_params_indoor.yaml → max_speed: 0.4
# config/controller_params_outdoor.yaml → max_speed: 1.2
```

29.5.3 啟動檔：一次啟動整個系統

一個完整的機器人系統可能需要同時啟動十幾個節點。若要一個一個手動啟動，既繁瑣又容易出錯。**啟動檔** (launch file) 解決了這個問題。

ROS 2 的啟動檔以 Python 撰寫 (也支援 XML 與 YAML)，這使它能夠包含邏輯——例如依參數決定是否啟動模擬器、或依機器人型號載入不同的設定。

```
# launch/robot_bringup.launch.py
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription
from launch.substitutions import LaunchConfiguration
import os
from ament_index_python.packages import get_package_share_directory
```



```
def generate_launch_description():
    pkg_dir = get_package_share_directory('my_robot')
    params = os.path.join(pkg_dir, 'config', 'controller_params.yaml')

    # 可由命令列覆寫的啟動參數
    use_sim = LaunchConfiguration('use_sim')

    return LaunchDescription([
        DeclareLaunchArgument('use_sim', default_value='false',
                               description='是否使用模擬器'),

        # 底盤驅動節點
        Node(package='my_robot', executable='base_driver',
             name='base_driver', parameters=[params],
             output='screen'),

        # 光達驅動節點
        Node(package='rplidar_ros', executable='rplidar_node',
             name='lidar', parameters=[{'serial_port': '/dev/ttyUSB0'}]),

        # 控制器節點 (重新命名話題)
        Node(package='my_robot', executable='controller',
             name='controller', parameters=[params],
             remappings=[('cmd_vel', 'cmd_vel_smoothed')]),

        # 機器人狀態發布 (讀取 URDF 並發布 TF)
        Node(package='robot_state_publisher',
             executable='robot_state_publisher',
             parameters=[{'robot_description': open(
                 os.path.join(pkg_dir, 'urdf', 'robot.urdf')).read()}]),
    ])

```

三個值得注意的機制：

- 參數檔的載入：`parameters=[params]` 使多個節點共用同一份設定。
- 話題重新映射 (remapping)：`remappings` 讓節點在不修改程式碼的情況下改變它發布或訂閱的話題名稱。這是重用第三方節點的關鍵技巧。
- 啟動參數：`LaunchConfiguration` 使同一個啟動檔能依情境調整 (如切換真實硬體與模擬器)。

29.6 TF2：座標變換的管理

29.6.1 為何需要專門的系統

第 8 章建立了座標變換的數學基礎。但在真實的機器人系統中，座標系的管理有三個超出數學的困難：

困難一：數量龐大。一台移動操作機器人可能有數十個座標系：世界、地圖、里程計、底盤、雷射、相機、手臂各連桿、夾爪、工具中心點.....

困難二：分散在不同節點。相機的外參由校正節點提供、手臂各連桿的變換由關節狀態計算、地圖到里程計的變換由定位節點提供。**沒有任何一個節點知道全部。**

困難三：時間問題。這是最關鍵也最容易被忽略的。相機在 t 時刻拍到一個物體，但這張影像的處理需要 50 毫秒。當結果出來時，機器人已經移動了——此時該用哪個時刻的座標變換？

TF2 正是為了解決這三個問題而生。它是一個分散式的座標變換系統，維護著一棵隨時間變化的變換樹。

29.6.2 變換樹的結構

TF2 把所有座標系組織成一棵樹 (tree)，這個結構有兩個嚴格的規則：

規則一：每個座標系只能有一個父座標系

規則二：不能有迴圈

樹的結構保證了任意兩個座標系之間有唯一的變換路徑。

一個典型的移動機器人 TF 樹：

```
map                ← 全域地圖座標系 ( 由 SLAM 或定位提供 )
├── odom            ← 里程計座標系 ( 連續但會漂移 )
│   ├── base_footprint ← 機器人在地面的投影
│   │   └── base_link  ← 機器人本體中心
│   │       ├── laser_link ← 光達
│   │       ├── camera_link ← 相機
│   │       ├── imu_link  ← IMU
│   │       └── arm_base_link ← 手臂基座
│   │           └── link1 → link2 → ... → gripper
```

◎ map → odom → base_link 的深意

這個三層結構是 ROS 導航的標準設計 (第 24.10.3 節)，其分工極為巧妙。**odom → base_link** 由里程計提供：**連續、高頻、但會漂移**。**map → odom** 由定位系統 (AMCL 或 SLAM) 提供：**低頻、會跳變、但不漂移**。兩者串接後得到的 **map → base_link** 就同時具備了連續性與不漂移。關

鍵在於：定位系統修正的不是機器人的位置，而是「地圖與里程計之間的偏差」。這使得下游的控制器可以放心使用 odom 座標系（保證連續，不會突然跳動），而不必擔心長期漂移。這是一個值得細細品味的架構設計。

29.6.3 時間查詢

TF2 最重要的功能是帶時間戳的查詢。它會儲存過去一段時間（預設 10 秒）的變換歷史，允許查詢「在某個特定時刻，A 相對於 B 的變換是什麼」。

```
import rclpy
from rclpy.node import Node
from tf2_ros import Buffer, TransformListener
from geometry_msgs.msg import PointStamped
import tf2_geometry_msgs # 註冊 PointStamped 的轉換支援

class TFExample(Node):
    def __init__(self):
        super().__init__('tf_example')
        self.buffer = Buffer()
        self.listener = TransformListener(self.buffer, self)

    def transform_point(self, point_in_camera):
        """把相機座標系中的一個點，轉換到 base_link 座標系"""
        try:
            # 關鍵：使用該點自身的時間戳查詢，而非「現在」
            result = self.buffer.transform(
                point_in_camera, 'base_link',
                timeout=rclpy.duration.Duration(seconds=0.1))
            return result
        except Exception as e:
            self.get_logger().warn(f'座標轉換失敗: {e}')
            return None

    def lookup_at_time(self, stamp):
        """查詢特定時刻的變換"""
        try:
            t = self.buffer.lookup_transform(
                'map', 'base_link', stamp,
                timeout=rclpy.duration.Duration(seconds=0.1))
            return t
        except Exception as e:
            self.get_logger().warn(f'查詢失敗: {e}')
            return None
```

注意 `self.buffer.transform(point\_in\_camera, ...)` 這一行——它使用了 `point\_in\_camera` 自身攜帶的時間戳。這意味著：即使處理花了 50 毫秒，TF2 也會回溯到影像拍攝的那一刻的座標變換來計算。

這正是第 24.9.3 節「時間戳不同步」問題的系統級解法。若沒有 TF2，每個節點都必須自己維護變換的歷史紀錄——這既繁瑣又容易出錯。

29.6.4 靜態與動態變換

TF2 區分兩種變換：

- **靜態變換 (Static)**：不隨時間改變的，如相機相對於底盤的安裝位置。只需發布一次，TF2 會永久記住。用 ``static_transform_publisher`` 或在 URDF 中定義。
- **動態變換 (Dynamic)**：隨時間改變的，如機器人在地圖中的位置、手臂各關節的角度。必須持續發布 (典型 10~100 Hz)。

一個常見的錯誤是把靜態變換當成動態的持續發布——這會浪費頻寬並增加系統負擔。相機的安裝位置一小時都不會變，卻每秒發布 50 次，是不必要的。

29.7 URDF 與機器人描述

29.7.1 用 XML 描述機器人

URDF (Unified Robot Description Format) 是描述機器人結構的標準格式。它以 XML 撰寫，直接對應第 4 章的連桿與關節概念。

一份 URDF 描述三類資訊：

- **運動學**：連桿之間的連接關係、關節類型與軸向、關節極限——這正是第 9 章 DH 參數的等價資訊。
- **動力學**：各連桿的質量、質心位置、慣性張量——第 13 章動力學計算所需的參數。
- **幾何**：視覺模型 (用於顯示) 與碰撞模型 (用於碰撞檢測，第 26 章)。

```
<?xml version="1.0"?>
<robot name="two_link_arm">

  <!-- 基座連桿 -->
  <link name="base_link">
    <visual>
      <geometry><cylinder radius="0.05" length="0.02"/></geometry>
      <material name="grey"><color rgba="0.5 0.5 0.5 1"/></material>
    </visual>
    <collision>
      <geometry><cylinder radius="0.05" length="0.02"/></geometry>
    </collision>
    <inertial>
      <mass value="1.0"/>
    </inertial>
  </link>

```

```

    <inertia ixx="0.001" iyy="0.001" izz="0.002"
            ixy="0" ixz="0" iyz="0"/>
  </inertial>
</link>

<!-- 第一個關節：旋轉關節，繞 Z 軸 -->
<joint name="joint1" type="revolute">
  <parent link="base_link"/>
  <child link="link1"/>
  <origin xyz="0 0 0.01" rpy="0 0 0"/>
  <axis xyz="0 0 1"/>
  <limit lower="-3.14" upper="3.14" effort="10" velocity="2.0"/>
</joint>

<!-- 第一節連桿：長 0.3 m -->
<link name="link1">
  <visual>
    <origin xyz="0.15 0 0" rpy="0 1.5708 0"/>
    <geometry><cylinder radius="0.02" length="0.3"/></geometry>
    <material name="blue"><color rgba="0.2 0.4 0.8 1"/></material>
  </visual>
  <collision>
    <origin xyz="0.15 0 0" rpy="0 1.5708 0"/>
    <geometry><cylinder radius="0.025" length="0.3"/></geometry>
  </collision>
  <inertial>
    <origin xyz="0.15 0 0"/>
    <mass value="0.5"/>
    <inertia ixx="0.0001" iyy="0.0038" izz="0.0038"
            ixy="0" ixz="0" iyz="0"/>
  </inertial>
</link>

<!-- 第二個關節 -->
<joint name="joint2" type="revolute">
  <parent link="link1"/>
  <child link="link2"/>
  <origin xyz="0.3 0 0" rpy="0 0 0"/>
  <axis xyz="0 0 1"/>
  <limit lower="-2.5" upper="2.5" effort="5" velocity="2.0"/>
</joint>

<link name="link2">
  <visual>
    <origin xyz="0.1 0 0" rpy="0 1.5708 0"/>
    <geometry><cylinder radius="0.015" length="0.2"/></geometry>
    <material name="green"><color rgba="0.2 0.7 0.3 1"/></material>
  </visual>
  <inertial>
    <origin xyz="0.1 0 0"/>
    <mass value="0.3"/>
    <inertia ixx="0.00005" iyy="0.001" izz="0.001"
            ixy="0" ixz="0" iyz="0"/>
  </inertial>
</link>

</robot>

```

這份檔案正是第 4 章實作的延續。當時我們用 `check\_urdf` 驗證了語法，現在要讓它真正動起來。

29.7.2 由 URDF 到 TF

URDF 與 TF2 之間有一個關鍵的橋樑：**robot_state_publisher** 節點。它的運作方式是：

1. 讀取 URDF，建立連桿與關節的結構。
2. 訂閱 `/joint\_states` 話題（訊息類型 `sensor\_msgs/JointState`），取得各關節的當前角度。
3. 用正向運動學計算各連桿的位姿（第 9 章的內容！）。
4. 把結果發布到 TF 樹上。

這是一個極為優雅的設計：只要有任何節點發布關節角度，整個機器人的座標變換就自動建立。無論這些角度來自真實的編碼器、模擬器、或是規劃器的輸出——下游完全不需要知道。

這也再次體現了 29.3.2 節的解耦原則：**真實機器人與模擬器可以完全互換**，只要它們都發布 `/joint\_states`。

29.7.3 Xacro：讓 URDF 可維護

純 URDF 有一個嚴重的缺點：**大量重複**。一台六軸手臂的六個連桿，其 XML 結構幾乎相同，只有尺寸與慣量不同。若手動撰寫，一份 URDF 可能有上千行，且極難維護。

Xacro (XML Macro) 解決了這個問題。它為 URDF 加上了巨集、變數與數學運算：

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="arm">

  <!-- 定義常數 -->
  <xacro:property name="link_radius" value="0.02"/>
  <xacro:property name="pi" value="3.14159265"/>

  <!-- 定義可重用的巨集 -->
  <xacro:macro name="arm_link" params="name length mass color">
    <link name="${name}">
      <visual>
        <origin xyz="${length/2} 0 0" rpy="0 ${pi/2} 0"/>
        <geometry>
          <cylinder radius="${link_radius}" length="${length}"/>
        </geometry>
      </visual>
    </link>
  </xacro:macro>
</robot>
```

```

    <material name="${color}"/>
  </visual>
  <inertial>
    <origin xyz="${length/2} 0 0"/>
    <mass value="${mass}"/>
    <!-- 圓柱體的慣量公式 (第 13.1.3 節) -->
    <inertia ixx="${mass*link_radius*link_radius/2}"
            iyy="${mass*length*length/12}"
            izz="${mass*length*length/12}"
            ixy="0" ixz="0" iyz="0"/>
  </inertial>
</link>
</xacro:macro>

<!-- 使用巨集，一行產生一個連桿 -->
<xacro:arm_link name="link1" length="0.3" mass="0.5" color="blue"/>
<xacro:arm_link name="link2" length="0.2" mass="0.3" color="green"/>
<xacro:arm_link name="link3" length="0.15" mass="0.2" color="red"/>

</robot>

```

注意慣量的計算直接寫在 XML 中 (``${mass*length*length/12}``) ——這正是第 13.1.3 節的圓柱體慣量公式。改變長度時，慣量會自動更新，不會出現「改了尺寸卻忘了改慣量」這種常見的錯誤。

29.8 QoS：通訊品質設定

29.8.1 為何需要設定

ROS 1 的通訊只有一種模式：可靠傳輸 (類似 TCP)。但機器人系統中不同的資料流有截然不同的需求：

- 影像串流：每秒 30 張，若掉了一張無所謂——寧可丟棄舊資料，也要拿到最新的。
- 地圖資料：只在更新時發布，但新加入的節點必須拿到——必須可靠，且要能取得最後一筆。
- 緊急停止指令：絕對不能遺失。
- 控制指令：高頻、且舊的指令毫無價值——寧可丟也不要延遲。

用同一種傳輸模式處理這些，必定有一方受害。QoS (Quality of Service) 讓每個話題可以各自設定。

29.8.2 主要的 QoS 選項

表 29-6 主要的 QoS 設定

設定	選項	意義	典型用途
可靠度 Reliability	RELIABLE	保證送達（會重傳）	指令、地圖、狀態
	BEST_EFFORT	盡力而為（不重傳）	感測器串流
持久性 Durability	VOLATILE	只送給當前的訂閱者	一般資料
	TRANSIENT_LOCAL	保留給後來的訂閱者	地圖、機器人描述
歷史 History	KEEP_LAST(n)	只保留最近 n 筆	多數情況
	KEEP_ALL	全部保留	不可遺失的資料
時效 Deadline	時間值	超過此間隔未收到即警告	監控感測器是否失效
生命期 Lifespan	時間值	超過此時間的訊息作廢	即時控制指令

29.8.3 實務上的設定

ROS 2 提供了預設的 QoS 組合 (profile)，涵蓋常見情境：

```

from rclpy.qos import QoSProfile, ReliabilityPolicy, DurabilityPolicy
from rclpy.qos import qos_profile_sensor_data

# 情境一：感測器資料——用內建的 sensor_data profile
# ( BEST_EFFORT + KEEP_LAST(5) · 適合高頻串流 )
self.scan_sub = self.create_subscription(
    LaserScan, 'scan', self.scan_callback, qos_profile_sensor_data)

# 情境二：地圖——必須可靠，且晚加入的節點也要拿得到
map_qos = QoSProfile(
    depth=1,
    reliability=ReliabilityPolicy.RELIABLE,
    durability=DurabilityPolicy.TRANSIENT_LOCAL) # 關鍵設定
self.map_pub = self.create_publisher(OccupancyGrid, 'map', map_qos)

# 情境三：控制指令——可靠但只要最新的
cmd_qos = QoSProfile(
    depth=1,
    reliability=ReliabilityPolicy.RELIABLE)
self.cmd_pub = self.create_publisher(Twist, 'cmd_vel', cmd_qos)

```

◎ QoS 不相容：一個經典的除錯陷阱

ROS 2 有一個規則：發布者與訂閱者的 QoS 必須相容，否則完全不會連線——而且不會報錯，只是「什麼都收不到」。最常見的情況是：發布者用 BEST_EFFORT，訂閱者用 RELIABLE (預設值)。訂閱者要求「保證送達」，但發布者不提供這個保證，於是兩者不建立連線。當你發現「明

明有節點在發布，但我就是收不到」時，第一件事就是用 `ros2 topic info -v <話題名稱>` 檢查兩端的 QoS 設定。這是 ROS 2 初學者最常遇到的坑，也是從 ROS 1 轉換過來的人最容易忽略的差異。

29.9 工具生態

29.9.1 命令列工具

ROS 2 的命令列工具（`ros2` 指令）是日常開發與除錯的第一道防線。掌握這幾個指令，能解決大部分的問題：

表 29-7 常用的 `ros2` 命令列工具

指令	用途	典型使用時機
<code>ros2 node list</code>	列出所有執行中的節點	確認節點是否啟動
<code>ros2 topic list</code>	列出所有話題	確認話題名稱是否正確
<code>ros2 topic echo <topic></code>	即時顯示話題內容	檢查資料是否正確
<code>ros2 topic hz <topic></code>	量測發布頻率	檢查是否掉頻
<code>ros2 topic info -v <topic></code>	顯示發布訂閱者與 QoS	除錯 QoS 不相容
<code>ros2 param list / get / set</code>	查看與設定參數	調參
<code>ros2 service call <srv></code>	手動呼叫服務	測試服務
<code>ros2 run <pkg> <exec></code>	執行單一節點	快速測試
<code>ros2 launch <pkg> <file></code>	執行啟動檔	啟動完整系統
<code>ros2 doctor</code>	檢查環境設定	環境有問題時

三個最有價值的除錯指令：`ros2 topic hz` 能立刻看出是否掉頻（第 3 章的時間尺度問題）、`ros2 topic echo` 能確認資料內容、`ros2 topic info -v` 能診斷 QoS 不相容（29.8.3 節的陷阱）。

29.9.2 RViz：三維視覺化

RViz 是 ROS 最重要的視覺化工具。它能同時顯示：

- 機器人模型：由 URDF 與 TF 即時呈現機器人的姿態。
- 感測資料：光達點雲、相機影像、深度點雲。
- 地圖與規劃：佔據柵格、路徑、代價地圖。

- **座標系**：TF 樹中所有座標系的位置與方向。
- **自訂標記**：以 `visualization\_msgs/Marker` 繪製任意的箭頭、線段、文字——用於顯示演算法的內部狀態。

RViz 的核心價值在於「讓不可見的變可見」。第 24 章的協方差橢圓、第 25 章的粒子分布、第 26 章的規劃路徑——這些抽象的內部狀態，都可以用 Marker 畫出來。當演算法出錯時，看見它在想什麼，往往比讀程式碼更快找到問題。

◎ RViz 不是模擬器

這是初學者最常見的誤解。**RViz 只是「顯示」——它不模擬任何物理**。它畫出的機器人會動，是因為有節點在發布 TF 與關節狀態；若那些資料來自真實機器人，RViz 就是即時的監控畫面；若來自模擬器（第 30 章的 Gazebo），RViz 顯示的就是模擬結果。**RViz 是眼睛，Gazebo 是世界——兩者角色完全不同，卻常被混淆。**

29.9.3 rosbag：記錄與重放

rosbag 能把所有話題的資料錄下來，之後完整重放。這個功能對機器人開發的價值極為重大：

- **離線除錯**：實機測試時錄下所有資料，回到辦公室慢慢分析。不需要重複跑實驗。
- **演算法比較**：用同一份資料測試不同的演算法版本，確保比較是公平的。
- **回歸測試**：修改程式後，用歷史資料驗證行為沒有退化。
- **資料集建立**：第 31 章的機器學習需要大量資料，rosbag 是最自然的收集方式。

```
# 錄製所有話題
ros2 bag record -a -o my_experiment

# 只錄製特定話題 (節省空間)
ros2 bag record /scan /odom /tf /camera/image_raw -o experiment_01

# 查看錄製內容的資訊
ros2 bag info experiment_01

# 重放 (可調速度)
ros2 bag play experiment_01
ros2 bag play experiment_01 --rate 0.5    # 半速，便於觀察
ros2 bag play experiment_01 --loop        # 循環播放
```

一個實務上的重要習慣：每次實機測試都錄 bag。硬碟很便宜，但「回到現場重做一次實驗」極為昂貴——尤其是那些偶發的、難以重現的錯誤。

29.9.4 rqt：圖形化工具集

rqt 是一組圖形化的除錯工具，常用的包括：

- **rqt_graph**：顯示節點與話題的连接圖。這是理解一個陌生系統最快的方式——一眼看出誰在跟誰講話。
- **rqt_plot**：即時繪製數值話題的曲線。用於觀察控制器的響應（第 16 章的階躍響應就可以這樣看）。
- **rqt_reconfigure**：圖形化的動態調參介面。配合 29.5.1 節的參數回呼，可以拖動滑桿即時調整增益。
- **rqt_console**：集中顯示所有節點的 log 訊息，可依等級與節點過濾。

29.10 即時性與系統設計

29.10.1 ROS 2 能做即時控制嗎

這是一個需要仔細回答的問題。簡短的答案是：可以，但需要條件。

ROS 2 在設計上支援即時性，但要真正達成，需要三個層次的配合：

1. **作業系統層**：必須使用即時核心（如 Linux 的 PREEMPT_RT 修補）或即時作業系統。標準的 Linux 無法保證延遲上限。
2. **中介軟體層**：DDS 的實作必須支援即時（如 eProsima 的 Fast DDS 有即時模式）。
3. **應用層**：節點的程式碼必須避免動態記憶體配置、避免不確定的鎖定、並使用即時安全的 API。

實務上的常見架構是分層混合：

表 29-8 典型的即時性分層

層級	頻率	執行環境	為何如此
伺服控制	1~10 kHz	微控制器或即時核心	硬即時要求，不容許延遲
狀態估測	100~1000 Hz	即時核心或高優先權	軟即時，偶爾延遲可容忍
感知與規劃	1~30 Hz	一般的 ROS 2 節點	無即時要求

層級	頻率	執行環境	為何如此
任務決策	< 1 Hz	一般節點或雲端	完全無即時要求

這正是第 3 章時間尺度金字塔的軟體實現。最底層的硬即時通常交給專用的硬體（第 5 章的伺服驅動器、第 7 章的微控制器），ROS 2 則負責其上的所有層級。

29.10.2 執行器與回呼群組

一個 ROS 2 節點可能有多個回呼（訂閱、定時器、服務）。它們如何被排程？這由執行器（executor）決定。

預設的 `SingleThreadedExecutor` 是單執行緒的——所有回呼依序執行，一次只能跑一個。這帶來一個重要的後果：

◎ 一個常見的阻塞陷阱

假設一個節點有兩個回呼：一個是 10 Hz 的感測器訂閱（處理很快）、另一個是服務（需要執行 2 秒的規劃）。在單執行緒執行器下，當服務在執行的那 2 秒內，感測器回呼完全不會被呼叫——資料全部堆積或遺失。這是 ROS 2 初學者最常遇到的效能問題。解法有三：一、改用 `MultiThreadedExecutor` 並搭配回呼群組（callback group）；二、把耗時的工作改成動作（29.4.3 節）而非服務；三、把耗時的工作拆到另一個節點。

```
import rclpy
from rclpy.node import Node
from rclpy.executors import MultiThreadedExecutor
from rclpy.callback_groups import (MutuallyExclusiveCallbackGroup,
                                   ReentrantCallbackGroup)

class MixedNode(Node):
    def __init__(self):
        super().__init__('mixed_node')
        # 群組一：感測器回呼，彼此互斥但可與其他群組並行
        sensor_group = MutuallyExclusiveCallbackGroup()
        # 群組二：耗時的服務，可重入
        service_group = ReentrantCallbackGroup()

        self.sub = self.create_subscription(
            LaserScan, 'scan', self.scan_cb, 10,
            callback_group=sensor_group)
        self.srv = self.create_service(
            Trigger, 'plan', self.plan_cb,
            callback_group=service_group)

def main():
    rclpy.init()
```

```
node = MixedNode()
executor = MultiThreadedExecutor(num_threads=4) # 多執行緒
executor.add_node(node)
executor.spin()
```

29.10.3 系統設計的原則

整合前面各節，設計一個 ROS 2 系統時應遵循五個原則：

- **單一職責**：每個節點只做一件事。若一個節點超過五百行，通常該拆了。
- **依頻率分層**：把不同時間尺度的工作放在不同的節點，避免慢的拖累快的。
- **使用標準訊息**：能用標準訊息就不要自訂——這決定了你能否使用社群的套件。
- **參數外部化**：所有可能需要調整的數值都應該是參數，而非寫死的常數。
- **設計失效行為**：每個節點都應該考慮「若上游沒有資料怎麼辦」。第 26.8.4 節的復原行為正是這個原則的體現。

29.11 由框架邁向智慧

29.11.1 本章的成果

本章建立了機器人系統的軟體基礎。你現在能夠：說明機器人軟體整合的困難與框架的價值、理解 ROS 2 的節點與話題架構、選擇適當的通訊模式、運用參數與啟動檔管理系統、以 TF2 處理座標變換與時間查詢、撰寫 URDF 描述機器人、設定 QoS、運用工具生態除錯、並設計符合即時性需求的系統架構。

我們也回收了兩個伏筆：**第 2 章**提到的軟體環境建置、以及**第 4 章**那份 `two\_link.urdf`——現在它能在 RViz 中真正動起來了。

更重要的是，本章展示了一個貫穿全書的主題的**軟體版本：解耦**。第 17 章用模型與回授解耦已知與未知、第 27 章用時間尺度解耦快慢動態、而本章用話題解耦了模組之間的依賴。**好的架構，就是讓每一部分都能獨立地改變。**

29.11.2 接下來：讓機器人變聰明

有了軟體框架，第五篇的其餘四章將處理「智慧」的各個面向：

第 30 章 (模擬與 Sim-to-Real)：在真實機器人上測試既昂貴又危險，而模擬能提供無限

的試錯機會。但模擬與現實之間有落差（第 14.7.3 節已預告過）——這一章要處理如何縮小它。屆時 29.3.2 節「模擬器與真實硬體可以互換」的設計將發揮關鍵作用。

第 31 章（機器學習與強化學習）：第 21 章的抓取、第 28 章的運動控制都已提過學習方法的威力。這一章將系統性地建立其原理，並回收第 18 章 Bellman 方程的伏筆——強化學習與最佳控制共用同一套數學。

第 32 章（大型模型與任務規劃）：近年最重大的變化。大型語言模型使機器人能理解自然語言的指令、並自行分解任務。但它們也帶來新的問題：如何確保生成的計畫是**可執行且安全的**？

第 33 章（具身人工智慧）：整個第五篇的收束。它要探討一個根本的問題——**智慧是否需要身體**？而答案將把我們帶回第 1 章的 Moravec 悖論。

換句話說：第 29 章讓各個模組**能夠對話**，第 30~33 章則讓它們**開始思考**。

完整計算例題

例題 29-1 系統架構設計

題目：設計一台室內服務機器人的 ROS 2 節點架構。需求為：以光達 SLAM 建圖與定位、用 RGB-D 相機辨識物體、能自主導航到指定地點、有一支六軸手臂能抓取物體、並提供語音指令介面。（a）列出所需的節點與其職責。（b）標明各節點之間的話題連接。（c）指出應使用服務與動作的地方。（d）分析各節點的頻率需求並分層。

解（a）：依「單一職責」原則，規劃十個節點：

表 29-9 服務機器人的節點設計

節點	職責	訂閱	發布
lidar_driver	讀取光達硬體	—	/scan
camera_driver	讀取 RGB-D 相機	—	/image_raw, /depth, /points
base_driver	底盤馬達控制與里程計	/cmd_vel	/odom, /joint_states
arm_driver	手臂關節控制	/arm_cmd	/joint_states
slam	建圖與定位	/scan, /odom	/map, /tf(map→odom)
object_detector	物體辨識與位姿估	/image_raw, /points	/detected_objects

節點	職責	訂閱	發布
	計		
navigation	路徑規劃與追隨	/map, /goal_pose, /scan	/cmd_vel
arm_planner	手臂運動規劃	/detected_objects	/arm_cmd
speech_interface	語音辨識與合成	—	/user_command
task_manager	任務協調與決策	/user_command, /detected_objects	/goal_pose (透過動作)

解 (b) : 資料流可分為三條主鏈 :

感知鏈 : lidar_driver → slam → /map → navigation

視覺鏈 : camera_driver → object_detector → arm_planner

控制鏈 : navigation → /cmd_vel → base_driver

三條並行的資料流。task_manager 位於最上層協調三者。

解 (c) : 三種通訊模式的分配 :

- **話題** : 所有的感測資料流 (/scan 、 /image_raw 、 /odom) 、 狀態回報 (/joint_states) 、 以及高頻的控制指令 (/cmd_vel) 。
- **服務** : 一次性且快速的操作——如「儲存目前的地圖」、「重設里程計」、「切換手臂的控制模式」。
- **動作** : 兩個長時間任務——導航到目標點 (Nav2 的 NavigateToPose) 與手臂執行抓取。兩者都需要進度回報與取消能力。

特別注意 task_manager 與 navigation 之間必須用動作而非服務。導航可能需要數十秒，若用服務，task_manager 會被完全阻塞，無法回應使用者的中止指令。

解 (d) : 依第 3 章的時間尺度金字塔分層 :

表 29-10 節點的頻率分層

層級	節點	頻率	即時性需求
硬體控制	base_driver, arm_driver (內部迴路)	1 kHz	硬即時 (在微控制器上)
感測擷取	lidar_driver, camera_driver	10~40 Hz	軟即時
狀態估測	slam	10~20 Hz	軟即時
感知處理	object_detector	5~15 Hz	無

層級	節點	頻率	即時性需求
規劃控制	navigation (局部)	10~20 Hz	軟即時
規劃控制	navigation (全域) , arm_planner	0.5~2 Hz	無
決策	task_manager, speech_interface	< 1 Hz	無

設計判讀：只有最底層的伺服控制需要硬即時，因此應在專用硬體上執行（第 7 章）；其餘各層都能在標準的 ROS 2 環境中運行。這個分層也決定了 QoS 的選擇——感測資料用 BEST_EFFORT、地圖用 TRANSIENT_LOCAL、控制指令用 RELIABLE。

例題 29-2 TF 樹與時間查詢

題目：某移動機器人在 $t = 10.000$ 秒時，相機拍到一個物體，其在相機座標系中的位置為 $(0.5, 0.1, 0.8)$ m。影像處理花了 80 毫秒，於 $t = 10.080$ 秒完成。機器人以 0.5 m/s 前進、0.2 rad/s 左轉。（a）說明應查詢哪個時刻的 TF。（b）估算若誤用「現在」時刻會產生多大的位置誤差。（c）說明 TF2 如何解決這個問題。（d）若 TF 的緩衝時間設為 0.05 秒，會發生什麼事？

解（a）：應查詢 $t = 10.000$ 秒——也就是影像被拍攝的時刻，而非處理完成的時刻。

理由：那個位置 $(0.5, 0.1, 0.8)$ 是相機在 $t = 10.000$ 時「看到」的。若要把它轉換到世界座標系，必須使用相機在當時的位姿。

解（b）：80 毫秒內機器人的移動量：

$$\text{平移} = 0.5 \text{ m/s} \times 0.08 \text{ s} = 0.04 \text{ m} = 4 \text{ 公分}$$

$$\text{旋轉} = 0.2 \text{ rad/s} \times 0.08 \text{ s} = 0.016 \text{ rad} = 0.92^\circ$$

機器人在處理期間的移動。

旋轉造成的額外誤差（物體距離約 0.95 m）：

$$\text{旋轉誤差} \approx 0.95 \times 0.016 = 0.0152 \text{ m} = 1.52 \text{ 公分}$$

角度誤差在距離上的放大。

$$\text{總誤差} \approx \sqrt{4^2 + 1.52^2} \approx 4.3 \text{ 公分}$$

若誤用「現在」的 TF，物體位置會偏差超過 4 公分。

工程意義：對抓取任務而言，4 公分的誤差是災難性的——夾爪會完全抓空（第 21 章的容差通常只有數公釐到 1 公分）。這說明了時間戳處理絕非細節，而是**成敗關鍵**。

解 (c)：TF2 儲存變換的**歷史紀錄** (29.6.3 節)。當呼叫 `buffer.transform(point, target_frame)` 時，它會：

1. 讀取 `point` 訊息中攜帶的時間戳 ($t = 10.000$)。
2. 在緩衝區中找出該時刻前後的兩筆變換紀錄。
3. 在兩者之間做內插，得到 $t = 10.000$ 時的精確變換。
4. 用這個變換計算結果。

這個內插機制是 TF2 最有價值的功能之一。它讓每個節點都不必自己維護歷史紀錄，也不必擔心資料到達的順序。

解 (d)：若緩衝時間只有 0.05 秒 (50 毫秒)，而查詢的時刻已經是 80 毫秒之前——**該筆紀錄已經被丟棄了**。

結果是拋出 `ExtrapolationException` 或 `LookupException`，轉換失敗。程式必須妥善處理這個例外 (如放棄這次偵測結果並記錄警告)，否則節點可能崩潰。

實務建議：TF 緩衝時間應設為**最長處理延遲的三到五倍**。預設的 10 秒對多數應用綽綽有餘；但若記憶體吃緊 (如嵌入式平台)，可以縮短——但絕不應短於系統中最慢的處理鏈。

例題 29-3 QoS 設定與頻寬估算

題目：某機器人系統包含：640×480 的 RGB 影像 (30 Hz)、640×480 的深度圖 (30 Hz)、二維光達 (每圈 720 點，10 Hz)、以及 200 Hz 的關節狀態 (12 個關節)。(a) 估算各話題的頻寬需求。(b) 為各話題選擇適當的 QoS。(c) 若透過 100 Mbps 的網路傳輸，判斷是否可行。(d) 提出三個降低頻寬的策略。

解 (a)：逐一計算：

RGB 影像： $640 \times 480 \times 3 \text{ bytes} \times 30 \text{ Hz} = 27.6 \text{ MB/s} = 221 \text{ Mbps}$

未壓縮的影像頻寬極大。

深度圖： $640 \times 480 \times 2 \text{ bytes} \times 30 \text{ Hz} = 18.4 \text{ MB/s} = 147 \text{ Mbps}$

深度圖每點 16 位元。

光達： $720 \text{ 點} \times 4 \text{ bytes} \times 10 \text{ Hz} = 28.8 \text{ KB/s} = 0.23 \text{ Mbps}$

光達的頻寬極小。

關節狀態：12 關節 × 3 值 × 8 bytes × 200 Hz = 57.6 KB/s = 0.46 Mbps

同樣極小。

總計 ≈ 369 Mbps

主要由影像主導，佔 99.8%。

解 (b)：QoS 的選擇：

表 29-11 各話題的 QoS 建議

話題	可靠度	歷史深度	持久性	理由
/image_raw	BEST_EFFORT	KEEP_LAST(1)	VOLATILE	高頻串流，掉幀無妨，只要最新的
/depth	BEST_EFFORT	KEEP_LAST(1)	VOLATILE	同上
/scan	BEST_EFFORT	KEEP_LAST(5)	VOLATILE	感測資料，保留少量歷史
/joint_states	RELIABLE	KEEP_LAST(10)	VOLATILE	狀態資料，不應遺失
/map	RELIABLE	KEEP_LAST(1)	TRANSIENT_LOCAL	晚加入的節點也要取得
/cmd_vel	RELIABLE	KEEP_LAST(1)	VOLATILE	指令必須送達，但只要最新的

解 (c)：369 Mbps 遠超過 100 Mbps 的網路容量——不可行。實際上連 1 Gbps 網路也會相當吃緊（佔用 37%），且會與其他流量競爭。

解 (d)：三個降低頻寬的策略：

- **策略一：壓縮**。使用 `image\_transport` 的壓縮傳輸（JPEG 或 PNG）。JPEG 通常能達到 10~20 倍的壓縮率，把 221 Mbps 降到約 15 Mbps。這是最有效的一步。
- **策略二：就地處理**。把影像處理節點放在機器人本體上，只傳輸處理後的結果（如偵測到的物體位姿，僅數百 bytes）。這是 29.10.3 節「依頻率分層」的延伸——頻寬也應該分層。
- **策略三：降低解析度或頻率**。若辨識任務不需要 30 Hz，降到 10 Hz 就能省下三分之二。同樣地，320×240 的解析度對多數辨識任務已經足夠。

實務上三者並用：本體上做高頻的處理、只把低頻的結果傳出去、且必要的影像串流一律壓縮。這也解釋了為何機器人上通常會裝一台效能不錯的電腦（第 3 章的運算平台選擇）——把資料留在本地處理，遠比傳出去便宜。

程式範例：一個完整的 ROS 2 套件

以下是一個完整可運行的 ROS 2 套件，包含一個簡單的速度控制器節點——它訂閱目標位置、發布速度指令，並示範參數、TF 查詢與生命週期管理。

```
# =====
# 檔案結構
# my_robot/
#   ├── package.xml
#   ├── setup.py
#   └── my_robot/
#       ├── __init__.py
#       └── goal_controller.py
#   ├── launch/
#       └── controller.launch.py
#   └── config/
#       └── params.yaml
# =====

# ----- my_robot/goal_controller.py -----
import math
import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy
from geometry_msgs.msg import Twist, PoseStamped
from nav_msgs.msg import Odometry
from tf2_ros import Buffer, TransformListener
from rcl_interfaces.msg import SetParametersResult

class GoalController(Node):
    """簡單的目標點控制器：以 Pure Pursuit 概念導向目標 (第 23.9.3 節)"""

    def __init__(self):
        super().__init__('goal_controller')

        # ----- 參數宣告 (29.5.1 節) -----
        self.declare_parameter('k_linear', 0.5)
        self.declare_parameter('k_angular', 1.5)
        self.declare_parameter('max_linear', 0.4)
        self.declare_parameter('max_angular', 1.0)
        self.declare_parameter('goal_tolerance', 0.10)
        self.declare_parameter('control_rate', 20.0)
        self._load_params()
        self.add_on_set_parameters_callback(self.on_param_change)

        # ----- TF (29.6 節) -----
        self.tf_buffer = Buffer()
        self.tf_listener = TransformListener(self.tf_buffer, self)

        # ----- 通訊 (29.3、29.8 節) -----
        cmd_qos = QoSProfile(depth=1,
```

```

        reliability=ReliabilityPolicy.RELIABLE)
self.cmd_pub = self.create_publisher(Twist, 'cmd_vel', cmd_qos)

self.goal_sub = self.create_subscription(
    PoseStamped, 'goal_pose', self.goal_callback, 10)
self.odom_sub = self.create_subscription(
    Odometry, 'odom', self.odom_callback, 10)

# ----- 狀態 -----
self.goal = None
self.pose = None

# ----- 控制迴路 -----
period = 1.0 / self.get_parameter('control_rate').value
self.timer = self.create_timer(period, self.control_loop)

self.get_logger().info(
    f'目標控制器已啟動 ( {1/period:.0f} Hz ) ')

# ===== 參數處理 =====
def _load_params(self):
    g = lambda n: self.get_parameter(n).value
    self.k_lin = g('k_linear')
    self.k_ang = g('k_angular')
    self.v_max = g('max_linear')
    self.w_max = g('max_angular')
    self.tol = g('goal_tolerance')

def on_param_change(self, params):
    """支援執行中動態調參"""
    for p in params:
        self.get_logger().info(f'參數 {p.name} → {p.value}')
    # 先讓框架寫入新值，再重新載入
    result = SetParametersResult(successful=True)
    self.create_timer(0.0, self._reload_once)
    return result

def _reload_once(self):
    self._load_params()
    self.destroy_timer(self.timer) # 避免重複觸發

# ===== 回呼 =====
def goal_callback(self, msg):
    self.goal = msg
    self.get_logger().info(
        f'收到新目標: ({msg.pose.position.x:.2f}, '
        f'{msg.pose.position.y:.2f}) in {msg.header.frame_id}')

def odom_callback(self, msg):
    self.pose = msg.pose.pose

```

```

# ===== 控制迴路 =====
def control_loop(self):
    if self.goal is None or self.pose is None:
        return

    # 把目標轉換到機器人座標系 ( 29.6.3 節：用目標自身的時間戳 )
    try:
        goal_in_base = self.tf_buffer.transform(
            self.goal, 'base_link',
            timeout=rclpy.duration.Duration(seconds=0.1))
    except Exception as e:
        self.get_logger().warn(f'TF 轉換失敗: {e}', throttle_duration_sec=2.0)
        return

    dx = goal_in_base.pose.position.x
    dy = goal_in_base.pose.position.y
    dist = math.hypot(dx, dy)

    cmd = Twist()
    if dist < self.tol:
        # 到達目標 · 停止
        self.cmd_pub.publish(cmd)      # 全零
        self.get_logger().info('已抵達目標', once=True)
        self.goal = None
        return

    # 簡化的 Pure Pursuit：朝目標方向轉、同時前進
    alpha = math.atan2(dy, dx)
    cmd.angular.z = max(-self.w_max,
                        min(self.w_max, self.k_ang * alpha))

    # 角度誤差大時減速 ( 避免走出大弧線 )
    speed_scale = max(0.0, math.cos(alpha))
    cmd.linear.x = max(0.0, min(self.v_max,
                                self.k_lin * dist * speed_scale))
    self.cmd_pub.publish(cmd)

def main(args=None):
    rclpy.init(args=args)
    node = GoalController()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        # 安全停止：離開前送出零速度 ( 重要的失效處理！ )
        node.cmd_pub.publish(Twist())
        node.destroy_node()
        rclpy.shutdown()

```

```
if __name__ == '__main__':
    main()
```

```
# ----- config/params.yaml -----
goal_controller:
  ros__parameters:
    k_linear: 0.6
    k_angular: 1.8
    max_linear: 0.35
    max_angular: 1.2
    goal_tolerance: 0.08
    control_rate: 20.0

# ----- launch/controller.launch.py -----
from launch import LaunchDescription
from launch_ros.actions import Node
import os
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    params = os.path.join(
        get_package_share_directory('my_robot'), 'config', 'params.yaml')
    return LaunchDescription([
        Node(package='my_robot', executable='goal_controller',
            name='goal_controller', parameters=[params],
            output='screen'),
    ])

# ----- setup.py 的關鍵部分 -----
# entry_points={
#     'console_scripts': [
#         'goal_controller = my_robot.goal_controller:main',
#     ],
# },
# data_files=[
#     ('share/my_robot/launch', ['launch/controller.launch.py']),
#     ('share/my_robot/config', ['config/params.yaml']),
# ],
```

四個值得注意的設計：

- **參數全部外部化**：所有增益與限制都是參數，可用 `rqt_reconfigure` 即時調整（29.5.1 節）。
- **TF 使用目標自身的時間戳**：而非「現在」，這正是例題 29-2 的正確作法。
- **離開前送出零速度**：`finally` 區塊中的 `publish(Twist())` 確保程式中斷時機器人會停止——這是失效導向安全（第 7.9 節）的軟體實現。
- **警告訊息使用 throttle**：`throttle\_duration\_sec=2.0` 避免 TF 失敗時每秒印出二十筆相同的警告，淹沒 log。

請試著修改：（一）把控制律改為第 23.9.3 節完整的 Pure Pursuit（含前視距離）。（二）加入一個服務，讓外部能查詢「距離目標還有多遠」。（三）把導航改為動作介面，支援進度回報與取消。

實作：建立你的第一個 ROS 2 系統

本章實作將前面各節整合為一個可運行的系統。建議在 Ubuntu 22.04 或 24.04 上安裝 ROS 2（Humble 或 Jazzy），Windows 使用者可用 WSL2。

1. **環境建置**：安裝 ROS 2、設定環境變數、執行 `ros2 doctor` 確認無誤。這回收了第 2.13.4 節的伏筆。
2. **Hello ROS**：執行內建的 talker/listener 範例，用 `ros2 topic list` 與 `ros2 topic echo` 觀察話題。用 `rqt_graph` 看兩者的連接。
3. **建立套件**：用 `ros2 pkg create` 建立自己的套件，撰寫 29.3.4 節的發布者與訂閱者並執行。
4. **參數與啟動檔**：把節點的參數外部化，撰寫 YAML 與啟動檔，用一個指令啟動整個系統。
5. **動態調參**：加入參數回呼，用 `ros2 param set` 或 `rqt_reconfigure` 在執行中改變數值，觀察行為變化。
6. **URDF 與 RViz**：把第 4 章的 `two_link.urdf` 擴充（加入慣量與碰撞模型），用 `robot_state_publisher` 與 `joint_state_publisher_gui` 在 RViz 中拖動關節——這回收了第 4 章的伏筆。
7. **TF 實驗**：用 `ros2 run tf2_tools view_frames` 產生 TF 樹的圖形。加入一個靜態變換發布者（模擬相機的安裝位置），驗證變換是否正確。
8. **QoS 實驗**：故意讓發布者用 BEST_EFFORT、訂閱者用 RELIABLE，觀察「什麼都收不到」的現象。用 `ros2 topic info -v` 診斷，再修正。這個經驗極為寶貴。
9. **rosvbag 錄製與重放**：錄下一段資料，用 `ros2 bag info` 查看，再以半速重放並在 RViz 中觀察。
10. **阻塞實驗**：撰寫一個節點，同時有 10 Hz 的訂閱與一個耗時 2 秒的服務。用單執行緒執行器觀察訂閱被阻塞的現象，再改用多執行緒執行器與回呼群組解決。
11. **（進階）整合模擬器**：安裝 Gazebo（或 TurtleBot 模擬套件），讓你的控制器節點驅動

模擬中的機器人。驗證 29.3.2 節「模擬與真實可互換」的設計——同一份控制器程式碼不需修改。

表 29-12 ROS 2 實作記錄表 (前兩列為範例)

實作項目	遇到的問題	診斷方式	解法
發布訂閱	訂閱者收不到訊息	ros2 topic info -v	QoS 不相容，改為一致
TF 查詢	ExtrapolationException	檢查時間戳	改用訊息自身的時間戳
(自行填寫其餘項目)			

國中小也看得懂：園遊會的公布欄

想像你們班要辦一場園遊會，有四個小組：一組負責做食物、一組負責收錢、一組負責招呼客人、一組負責整理場地。

問題來了：這四組要怎麼溝通？

最笨的方法是「每一組都要認識其他三組」——做食物的要知道收錢的是誰、收錢的要知道整理場地的是誰……四組要建立六條聯絡管道。如果有十組呢？就要四十五條！光是記住誰要跟誰講話，就會累死。

聰明的方法是：設一個公布欄。做食物的把「食物好了！」貼在公布欄上，需要知道的人自己去看。收錢的把「今天賣了多少錢」貼上去，老師想看就去看。

這樣一來，每一組都不需要認識其他組，只要知道「該把消息貼在哪個公布欄」就好。而且新加入的小組也很簡單——直接看公布欄就行了。

機器人的軟體也是這樣運作的！這個「公布欄」在機器人的世界裡叫做「話題」(topic)。

舉個例子：機器人的相機看到一個杯子，它就把「我看到杯子在這個位置」貼到公布欄上。手臂的程式看到這個消息，就伸手去抓。相機完全不需要知道手臂是誰寫的、用什麼程式語言、甚至不需要在同一台電腦上。

這個方法有多厲害呢？因為全世界的科學家都用同一套公布欄的規則，所以你可以直接下載別人寫好的程式來用。想讓機器人畫地圖？下載一個現成的就好，十分鐘就跑起來了——而這件事在二十年前，一個研究生要花兩年才做得出來！

叫獸提醒與常見錯誤

- 誤區一：以為 ROS 是作業系統。它是跑在 Linux 之上的中介層，負責通訊、工具與標準介面。
- 誤區二：初期覺得 ROS 太複雜而自己寫一套。你會逐步重新發明它的每一個功能，且無法使用社群的任何套件。
- 誤區三：一個節點做太多事。應遵循單一職責原則，超過五百行通常就該拆分。
- 誤區四：用服務執行長時間任務。服務會阻塞呼叫者，長任務應改用動作（可回報進度與取消）。
- 誤區五：忽略 QoS 不相容。發布者與訂閱者的 QoS 不相容時「完全不會連線且不報錯」，應用 `ros2 topic info -v` 診斷。
- 誤區六：TF 查詢使用「現在」時刻。應使用資料自身的時間戳，否則處理延遲會造成顯著的位置誤差（例題 29-2）。
- 誤區七：把靜態變換當成動態的持續發布。相機的安裝位置不會變，不需要每秒發布五十次。
- 誤區八：參數寫死在程式碼中。所有可能調整的數值都應外部化，否則調參要重新編譯。
- 誤區九：在單執行緒執行器中執行耗時的回呼。這會阻塞其他所有回呼，應改用多執行緒與回呼群組。
- 誤區十：未壓縮就在網路上傳輸影像。單一路 $640 \times 480 @ 30\text{fps}$ 的未壓縮影像就需要 221 Mbps。
- 誤區十一：把 RViz 當成模擬器。RViz 只負責顯示，不模擬任何物理——世界是由 Gazebo 提供的。
- 誤區十二：程式結束時未送出停止指令。應在 `finally` 區塊中發布零速度，這是失效導向安全的軟體實現。

課堂討論

1. 開頭故事中四組學生的成果無法整合。請討論：除了通訊機制之外，還需要哪些「約定」才能讓不同團隊的成果互通？（提示：座標系、單位、時間基準、命名規範）
2. 話題的發布—訂閱模式使模組完全解耦。請討論這個設計的代價——在什麼情況下，這種

解耦反而造成困難？

3. $\text{map} \rightarrow \text{odom} \rightarrow \text{base_link}$ 的三層設計，把「連續但漂移」與「不漂移但跳變」分離。請討論這個思維在本書其他章節的對應（提示：第 24 章的互補濾波、第 17 章的前饋加回授）。
4. 例題 29-3 顯示影像佔了 99.8% 的頻寬。請討論「就地處理」這個策略對系統架構的影響，以及它與雲端運算的取捨。
5. ROS 2 用 DDS 取代了 ROS 1 的自訂通訊。請討論「採用工業標準」相對於「自行開發」的優缺點，並舉出本書中其他類似的取捨。

章末摘要

- 機器人軟體的核心困難是整合：介面組合爆炸、異質性、並行與時間、以及除錯困難。
- 框架提供四項功能：通訊機制、標準介面、執行管理與工具生態。
- ROS 不是作業系統，而是跑在 OS 之上的中介層。
- ROS 2 相對 ROS 1 的三個關鍵改變：改用 DDS（去中心化、QoS、安全）、支援即時性、生命週期管理。
- 節點是基本執行單位，應遵循單一職責原則。
- 話題採發布—訂閱模式，發布者與訂閱者完全解耦，這使模擬器與真實硬體可以互換。
- 標準訊息類型是社群能共用套件的關鍵——所有底盤都用 Twist、所有光達都用 LaserScan。
- 三種通訊模式：話題（非同步資料流）、服務（同步請求回應）、動作（長時間任務含進度與取消）。
- 選擇模式的三個問題：需要回應嗎、會很快回應嗎、需要進度或取消嗎。
- 參數使設定與程式碼分離，並支援執行中動態調整——這對調參極為重要。
- 啟動檔一次啟動多個節點，並支援話題重新映射與條件邏輯。
- TF2 管理座標變換樹，其最重要的功能是帶時間戳的查詢與內插。
- $\text{map} \rightarrow \text{odom} \rightarrow \text{base_link}$ 的三層設計，分離了「連續但漂移」與「不漂移但跳變」兩種特性。
- URDF 描述機器人的運動學、動力學與幾何；robot_state_publisher 用正向運動學把它轉為

TF。

- Xacro 為 URDF 加上巨集與運算，避免重複並使慣量能隨尺寸自動更新。
- QoS 讓每個話題各自設定可靠度、持久性與歷史深度；不相容時會完全不連線且不報錯。
- 工具生態是 ROS 的核心價值：ros2 CLI 診斷、RViz 視覺化、rosviz 記錄重放、rqt 圖形化除錯。
- RViz 只負責顯示不模擬物理；Gazebo 才是模擬器。
- 即時性需要作業系統、中介軟體與應用層三個層次的配合；實務上採分層混合架構。
- 單執行緒執行器中的耗時回呼會阻塞其他回呼，應改用多執行緒與回呼群組。

觀念題與計算題

觀念題

1. 說明機器人軟體整合困難的四個來源，並解釋為何介面數量會平方成長。
2. 列舉機器人框架提供的四項核心功能及其在 ROS 2 中的對應。
3. 為何說「ROS 不是作業系統」？它與真正的作業系統如何分工？
4. 列舉 ROS 1 的四個限制，以及 ROS 2 針對它們所做的三個關鍵改變。
5. 說明採用 DDS 帶來的三個好處。
6. 說明發布—訂閱模式的解耦特性，以及它帶來的三個好處。
7. 為何標準訊息類型對社群生態如此重要？
8. 比較話題、服務與動作三種通訊模式，並說明選擇的三個判準。
9. 為何不應該用服務執行長時間任務？應改用什麼？
10. 說明參數機制的價值，以及動態調參對哪些章節的工作特別有幫助。
11. 說明啟動檔的三個機制：參數載入、話題重新映射、啟動參數。
12. 說明 TF2 解決的三個困難，以及變換樹的兩個結構規則。
13. 解釋 map → odom → base_link 三層設計的分工與其巧妙之處。
14. 說明 TF2 的時間查詢與內插機制，以及它為何是時間戳問題的系統級解法。
15. 說明 URDF 描述的三類資訊，以及 robot_state_publisher 的運作方式。

16. 說明 Xacro 相對於純 URDF 的優勢，並舉例說明它如何避免錯誤。
17. 列舉四個主要的 QoS 設定選項，並各舉一個適用情境。
18. 說明 QoS 不相容的現象與診斷方式，以及為何它是常見的除錯陷阱。
19. 說明 RViz 與 Gazebo 的角色差異。
20. 說明 ROS 2 達成即時性所需的三個層次，以及實務上的分層混合架構。

計算題

1. 某系統有 8 個模組需要互相通訊。(a) 若採用點對點的直接介面，最壞情況需要幾條介面？(b) 若採用發布訂閱模式，每個模組只需知道話題名稱，共需幾個「約定」？(c) 若模組數增至 20，重新計算兩者。(d) 由此說明框架的價值。
2. 某機器人以 0.8 m/s 前進、0.3 rad/s 轉向。相機在 $t = 5.000$ s 拍到距離 1.2 m 的物體，處理耗時 120 ms。(a) 求處理期間的平移量與旋轉量。(b) 求旋轉造成的橫向誤差。(c) 求總位置誤差。(d) 判斷此誤差對抓取任務（容差 8 mm）是否可接受。
3. 某系統的話題包含：1280×720 RGB 影像（15 Hz）、三維點雲（每幀 30 萬點、每點 16 bytes、10 Hz）、IMU（100 Hz、9 個 float64）。(a) 分別計算三者的頻寬（Mbps）。(b) 求總頻寬。(c) 若使用 1 Gbps 網路，計算佔用率。(d) 若影像以 JPEG 壓縮 15 倍、點雲改為就地處理只傳結果，重新計算總頻寬。
4. 某節點有三個回呼：感測器訂閱（20 Hz、每次處理 5 ms）、控制定時器（50 Hz、每次 3 ms）、規劃服務（每分鐘一次、每次 800 ms）。(a) 計算前兩者的 CPU 佔用率。(b) 在單執行緒執行器下，規劃服務執行時會遺失多少次感測器回呼與控制回呼？(c) 說明這對系統的影響。(d) 提出兩種解法並比較。

動手做與專題延伸

本章實作：機器人軟體框架

把前面各章的演算法，整合到 ROS 2 的架構中。這是全書程式的整合起點。

- 階段一：建立工作空間與套件，實作基本的發布訂閱節點。
- 階段二：把第 23 章的里程計與第 23.9.3 節的 Pure Pursuit 包裝成 ROS 2 節點。
- 階段三：把第 24 章的 Kalman 濾波器包裝成節點，訂閱多個感測話題並發布融合後的位姿。

- **階段四**：撰寫完整的 URDF (含慣量與碰撞)，在 RViz 中視覺化並驗證 TF 樹。
- **階段五**：撰寫啟動檔，一個指令啟動整個系統；並用 rosbag 錄製資料供離線分析。

專題延伸

- **系統架構圖的繪製與分析**：為一台你設計的機器人 (可用例題 29-1 的服務機器人) 繪製完整的節點與話題連接圖。標註每個話題的訊息類型、頻率、QoS 設定與估計頻寬。接著用 rqt_graph 產生實際的圖，與你的設計圖比對——你會發現真實系統往往比設計時複雜得多。討論哪些連接是必要的、哪些可以簡化。
- **時間戳誤差的實驗驗證**：撰寫兩個版本的座標轉換節點——一個使用資料自身的時間戳、一個使用「現在」。讓機器人 (或模擬器) 以不同速度移動，並人為加入處理延遲 (如 sleep 100 ms)。量測兩者的位置誤差差異，繪出「速度 vs 誤差」曲線，驗證例題 29-2 的計算。這個實驗能讓「時間戳很重要」從一句話變成一個具體的數字。
- **進階：從零建立一個最小的通訊框架**。用 Python 的 socket 或 ZeroMQ，自行實作一個極簡的發布訂閱機制 (約兩百行)。接著逐步加入需求：多對多連線、訊息序列化、話題的自動探索、時間戳、以及跨機器通訊。**記錄每加入一項功能所需的程式碼量與遇到的問題**。這個練習的目的不是要取代 ROS，而是親身體會 ROS 為你解決了多少問題——完成後你會對 29.1.2 節的知識框有完全不同的體會。

參考文獻與延伸閱讀

- ROS 2 官方文件 (docs.ros.org)：最權威且持續更新的資料來源，含完整的教學與 API 參考。
- Macenski, S. et al., "Robot Operating System 2: Design, Architecture, and Uses In The Wild," Science Robotics, 2022 (ROS 2 的設計理念與實際應用回顧)。
- Quigley, M. et al., "ROS: An Open-Source Robot Operating System," ICRA Workshop on Open Source Software, 2009 (ROS 的原始論文)。
- Quigley, M., Gerkey, B. and Smart, W. D., Programming Robots with ROS, O'Reilly (ROS 程式設計的實務教材)。
- Newman, W. S., A Systematic Approach to Learning Robot Programming with ROS, CRC Press (系統性的 ROS 教材，含大量範例)。

- Foote, T., "tf: The Transform Library," TePRA 2013 (TF 系統的設計論文) 。
- OMG DDS 標準文件：ROS 2 底層通訊中介軟體的規格。
- Macenski, S. et al., "The Marathon 2: A Navigation System," IROS 2020 (Nav2 的設計，本章分層架構的實例) 。
- ROS 2 Design 網站 (design.ros2.org)：記錄了 ROS 2 各項設計決策的理由，對理解「為何這樣設計」極有幫助。
- Awesome ROS 2 ([GitHub](#))：社群整理的 ROS 2 資源清單，含套件、教學與工具。

第 30 章

模擬與 Sim-to-Real

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為兩格對照：上格是模擬畫面中機械手臂流暢抓起物體，標註「模擬成功率 98%」；下格是實驗室的真實照片——手臂抓空、物件倒了一桌，標註「實機成功率 20%」。右半為領域隨機化的核心示意：左側「傳統思路」是兩個相距的小圓點（SIM 與 REAL）以雙向箭頭標註「努力縮小落差」；右側「領域隨機化」是一個涵蓋範圍極大的橢圓（內部散布數百個代表不同參數組合的小點），REAL 落在其中，標註「真實世界只是其中一個樣本」。下緣文字：模擬不必更真，策略必須更穩。

叫獸開講 「模擬裡 98% 的成功率，為什麼到實機只剩兩成？」

小妤花了兩週，在模擬器裡訓練出一個抓取策略。成功率 98%——她把畫面投影出來，機械手臂流暢地抓起各種形狀的物體，連歪斜的圓柱都能穩穩夾住。

「很漂亮。」叫獸點頭，「現在搬到實機上跑。」

三十分鐘後，實驗室裡一片狼藉。手臂抓空了七次、把三個物件推倒、有兩次夾爪碰到桌面觸發了過載保護。實測成功率：約兩成。

「98% 變成 20%。」小妤盯著螢幕，「模擬器是不是寫錯了？」

「模擬器沒錯。」叫獸拿起一個剛才被推倒的塑膠瓶，「錯的是你以為模擬器該是什麼。」

他把瓶子放在桌上輕輕一推，瓶子滑了一小段停下來。「你的模擬裡，這個瓶子的摩擦係數是多少？」「0.6，我查表填的。」

「那實際上呢？」叫獸把瓶子倒過來，「這一面沾了點灰，摩擦係數可能只有 0.3。而且桌面有一層薄薄的油膜——你上禮拜擦桌子留下的。」

小翰插話：「所以只要把摩擦係數量準一點就好了？」

「那你要量幾個？」叫獸反問，「每一個物體、每一面、每一種濕度、每一次擦桌子之後。而且還有質量分布、表面彈性、夾爪墊的老化程度、馬達的溫度.....你永遠追不完。」

他在白板上寫下一行字：模擬永遠不等於現實，這不是缺陷，而是定義。

「所以正確的問題不是『怎麼讓模擬更真』，而是『怎麼讓學到的東西不依賴那些不準的細節』。」叫獸停頓一下，「如果你的策略只在摩擦係數恰好是 0.6 時才有效，那它本來就是脆弱的——就算模擬器完美無缺，換一個沾了灰的瓶子它還是會失敗。」

最後他寫下今天的核心：「與其追求完美的模擬，不如刻意把模擬弄亂——每次訓練都隨機改變摩擦、質量、光線、延遲。逼你的策略學會在**所有可能的世界**裡都能運作。這樣它到了真實世界，只不過是又多了一個它沒見過、但也不特別的世界罷了。」

本章為什麼重要

模擬是現代機器人開發不可或缺的環節。它的價值來自三個現實的限制：**真實機器人昂貴**

(一台協作手臂數十萬元、一台四足機器人上百萬元)、**測試危險** (第 27 章的無人機會墜落、第 28 章的腿式機器人會摔壞)、**時間緩慢** (一次抓取嘗試需數秒，而第 31 章的強化學習需要數百萬次)。

模擬把這三個限制通通解除：可以同時跑一千個環境、可以讓機器人摔一萬次、可以用比實時快百倍的速度執行。**這正是近年強化學習能在機器人上取得突破的物質基礎** (第 28.9 節的 ANYmal、第 21.9.3 節的抓取資料集)。

但模擬有一個根本的限制——**它永遠不等於現實**。這個落差稱為 **Sim-to-Real Gap**，是本章的核心議題。第 14.7.3 節已經預告過接觸模擬的困難，第 21.2.3 節提過摩擦係數的不確定性，本章要系統性地處理：**落差從哪裡來、如何量化、以及如何跨越**。

本章的核心洞見與全書多處呼應：**不要追求消除不確定性，而要設計對不確定性不敏感的系統**。這與第 20 章的柔順控制、第 21 章的軟性夾爪、第 24 章的機率估測，是同一種工程哲學的不同體現。

學習目標

- 能說明模擬在機器人開發中的三個價值與其限制。
- 能描述物理引擎的四個核心模組及其計算瓶頸。
- 能區分不同層次的模擬保真度並依需求選擇。
- 能分析 Sim-to-Real Gap 的五個主要來源。
- 能說明領域隨機化的原理、實作與參數選擇。
- 能區分領域隨機化與領域自適應兩種策略。
- 能說明系統辨識如何縮小落差。
- 能比較常見模擬器的特性並為特定任務選擇。
- 能說明大規模平行模擬的架構與其對學習的意義。
- 能設計模擬到實機的漸進式驗證流程。
- 能說明數位孿生的概念及其與模擬的差異。
- 能評估一個模擬環境是否足以支持特定的開發目標。

先備知識

第 14 章的正向動力學與數值積分（模擬器的核心）、第 14.7 節的接觸模型、第 21.2 節的摩擦錐、第 29 章的 ROS 2 架構（模擬器與真實硬體的可互換性）、第 22 章的相機模型（視覺模擬）。

本章的數學不深，但**觀念的轉換很關鍵**——必須從「模擬要多真才夠」轉向「哪些不真無所謂」。建議讀完 30.4 節後停下來思考自己過去的專案，是否曾誤以為「模擬對了就等於做對了」。

核心名詞中英對照

表 30-1 本章核心名詞

中文	英文	說明
模擬器	Simulator	以計算重現物理與感測的軟體
物理引擎	Physics Engine	計算剛體動力學與接觸的核心模組
模擬到現實落差	Sim-to-Real Gap	模擬行為與真實行為的差異
保真度	Fidelity	模擬與現實相符的程度
領域隨機化	Domain Randomization	訓練時隨機改變模擬參數的策略
領域自適應	Domain Adaptation	使模型適應目標領域的技術
系統辨識	System Identification	由實測資料估計模型參數
即時因子	Real-Time Factor, RTF	模擬速度與真實時間的比值
渲染	Rendering	產生影像的圖形計算
光線追蹤	Ray Tracing	以模擬光線路徑產生逼真影像
碰撞偵測	Collision Detection	判斷物體是否接觸
約束求解器	Constraint Solver	計算接觸與關節約束力的模組
軟接觸	Soft Contact	允許微小穿透的接觸模型
數位孿生	Digital Twin	與實體同步的虛擬複本
硬體在環	Hardware-in-the-Loop, HIL	真實硬體接入模擬迴路
場景隨機化	Scene Randomization	隨機改變環境外觀與配置
合成資料	Synthetic Data	由模擬產生的訓練資料
漸進式驗證	Progressive Validation	由模擬逐步過渡到實機的流程

30.1 為何需要模擬

30.1.1 三個現實的限制

模擬的價值，來自真實機器人開發的三個限制。

限制一：成本。一台協作機械手臂數十萬元、一台四足機器人上百萬元、一台自駕測試車更是千萬等級。而研究通常需要**多台**——不同的構型、不同的配置、以及備品。模擬則可以無限複製。

限制二：風險。第 27 章的無人機失控會墜毀、第 28 章的腿式機器人跌倒會損壞關節、第 20 章的力控制參數錯誤會壓壞工件。而學習演算法的本質就是「試錯」——第 31 章的強化學習在初期幾乎全是失敗的動作。在真機上訓練，代價無法承受。

限制三：時間。這是最根本的限制。一次抓取嘗試約需 5 秒，而訓練一個抓取策略可能需要數十萬次嘗試：

100,000 次 × 5 秒 ≈ 139 小時 (不眠不休近 6 天)

而且還要人工重置場景。在真機上，這幾乎不可行。

模擬把這三個限制通通解除：

表 30-2 模擬解除的三個限制

限制	真實機器人	模擬
成本	數十萬~千萬元/台	近乎零 (只需算力)
風險	損壞、傷人	無 (重來即可)
速度	實時 (1×)	可達 100~1000×
平行度	1 台	可達數千個環境
重置	需人工	自動、瞬間
可觀測性	受感測器限制	全知 (所有狀態皆可讀)

最後一列「可觀測性」常被低估。在模擬中我們知道每一個物體的精確位置、每一個接觸點的力、每一個關節的真實速度——這些在真機上要嘛量不到、要嘛有雜訊。這使得**除錯與分析變得容易得多**，也使第 28.9.3 節的「教師—學生」訓練成為可能。

30.1.2 模擬的四種用途

模擬在機器人開發中扮演四種不同的角色，需求各不相同：

- **演算法開發與除錯**：驗證程式邏輯是否正確。**不需要高保真度**——只要能跑起來、能看見行為即可。
- **控制器驗證**：測試控制器在各種情況下的表現。**需要動力學準確**，但視覺可以很粗糙。
- **學習訓練**：產生大量的訓練資料。**需要速度與多樣性**，而非單一環境的精確度。
- **視覺感知訓練**：產生標註好的影像資料。**需要視覺逼真**，但物理可以簡化。

這四種用途對「保真度」的需求截然不同（30.3 節）。一個常見的錯誤是不分用途地追求「最真實的模擬」——這既浪費資源，也往往找錯了方向。

30.2 物理引擎的架構

30.2.1 四個核心模組

第 14.8.1 節已介紹過物理引擎的組成，此處深入其計算特性。一個模擬步驟包含四個階段：

表 30-3 物理引擎的四個模組

模組	任務	計算複雜度	通常的瓶頸
碰撞偵測	找出哪些物體接觸、接觸點在哪	$O(n \log n) \sim O(n^2)$	常是最大瓶頸
約束求解	計算接觸力與關節約束力	迭代，視接觸數而定	接觸多時很慢
正向動力學	由力求加速度（第 14.5 節）	$O(n) \sim O(n^3)$	自由度高時
數值積分	推進到下一個時刻（第 14.6 節）	$O(n)$	很少是瓶頸

碰撞偵測通常是最大的瓶頸，這與直覺不符——多數人以為動力學計算最耗時。原因是碰撞偵測需要處理大量的幾何配對，且複雜的網格模型（mesh）的檢測成本極高。

因應之道是使用簡化的碰撞幾何：用球、膠囊、盒子等基本形狀近似複雜的模型。這正是第 29.7.1 節 URDF 中「視覺模型」與「碰撞模型」分開的原因——視覺可以精細（只影響顯示），碰撞則應盡量簡單。

◎ 一個常見的效能陷阱

許多初學者直接把 CAD 匯出的高精度網格（數萬個三角面）當成碰撞模型。結果是模擬慢得無法使用——即時因子可能只有 0.05（模擬一秒要花二十秒）。正確的作法是為碰撞另外建立簡化模型：一根連桿用一個膠囊、一個底盤用一個盒子、一個夾爪用兩個小盒子。這通常能把速度提升一

到兩個數量級，而對物理行為的影響微乎其微。

30.2.2 時間步長的取捨

第 14.6.3 節討論過步長的選擇，在模擬器中這個取捨更為關鍵：

步長太大 → 穿透、彈跳、發散

步長太小 → 模擬太慢

而含接觸的模擬需要極小的步長 ($0.1 \sim 1 \text{ ms}$)，這是模擬速度的主要限制。

兩個常用的技巧：

- 子步進 (substepping)：物理以小步長計算 (如 1 ms)，但只在較大的間隔 (如 10 ms) 回報結果給控制器。這使物理準確而通訊負擔小。
- 分離的時間尺度：物理 1 kHz 、控制 100 Hz 、渲染 30 Hz 、感測器發布依各自頻率。這正是第 3 章時間尺度金字塔在模擬中的實現。

30.2.3 即時因子

即時因子 (Real-Time Factor, RTF) 是模擬速度的標準指標：

$\text{RTF} = \text{模擬時間} / \text{實際耗費的時間}$

$\text{RTF} = 1$ 代表與現實同速； $\text{RTF} = 100$ 代表快一百倍； $\text{RTF} = 0.1$ 代表慢十倍。

不同用途對 RTF 的需求不同：

表 30-4 各用途的 RTF 需求

用途	RTF 需求	理由
硬體在環 (HIL)	必須 = 1	真實硬體以真實時間運行
人在迴路測試 (遙控)	接近 1	操作員的反應需要真實時間感
控制器驗證	≥ 0.5 即可	可接受慢一些，但要能觀察
強化學習訓練	越高越好 ($100+$)	決定訓練所需的時間
視覺資料生成	不重要	離線批次產生即可

30.3 保真度的層次

30.3.1 「夠真」是相對的

「模擬要多真才夠？」這個問題沒有絕對的答案——它取決於你要用模擬回答什麼問題。

以一台移動機器人的開發為例，不同階段所需的保真度截然不同：

表 30-5 依開發階段的保真度需求

階段	要回答的問題	所需保真度	可以簡化的部分
架構設計	節點該怎麼切分？	極低（可用假資料）	幾乎全部
演算法邏輯	程式會不會當？	低（運動學即可）	動力學、接觸、視覺
導航測試	路徑規劃對不對？	中（需碰撞與里程計）	精確動力學、視覺細節
控制器調校	增益設得對嗎？	高（需準確動力學）	視覺、外觀
抓取學習	策略能否泛化？	中（但需多樣性）	單一環境的精確度
視覺辨識訓練	能否辨識物體？	視覺高、物理低	動力學可完全省略

最後兩列特別值得注意：抓取學習需要的不是「更真」，而是「更多樣」；視覺訓練需要的是渲染逼真，物理則幾乎可以不要。

30.3.2 三個保真度的維度

保真度不是一個單一的量，而是至少三個獨立的維度：

維度一：動力學保真度。質量、慣量、摩擦、關節特性是否準確。影響控制器的行為。

維度二：感測保真度。感測器的雜訊、延遲、失效模式是否模擬。**這一項最常被忽略——多數模擬器預設輸出「完美的」感測資料，這與現實差距極大。**

維度三：視覺保真度。光照、材質、陰影、反射是否逼真。只影響視覺相關的任務。

這三個維度可以獨立調整，而且應該依需求分別決定。追求全部都高，只會拖慢模擬速度卻沒有對應的收益。

◎ 最容易被忽略的保真度：感測器缺陷

多數 Sim-to-Real 失敗的案例，問題不在物理而在**感測**。模擬中的相機沒有動態模糊、沒有曝光變化、沒有鏡頭畸變；模擬中的光達沒有多重反射、不會被玻璃穿透、不會在陽光下失效；模擬中的編碼器沒有量化誤差、IMU 沒有零漂。一個在完美感測下訓練出的策略，遇到真實的雜訊就會**崩潰**。因此在模擬中「刻意加入缺陷」——雜訊、延遲、偶爾的資料遺失——往往比提升物理精度更有價值。這正是 30.5 節領域隨機化的一個重要面向。

30.4 Sim-to-Real Gap 的來源

30.4.1 五個主要來源

模擬與現實的落差，可以歸納為五個來源。理解它們的相對重要性，才能把資源投在對的地方。

來源一：接觸與摩擦（最嚴重）。這是第 14.7 節與第 21.2.3 節反覆提及的困難：

- 摩擦係數不確定且會變（灰塵、濕度、磨損），實測範圍可達數倍。
- 接觸模型本身是近似的——懲罰法允許穿透、約束法需要正則化。
- 真實的接觸涉及材料變形、微觀嵌合，這些幾乎無法模擬。

來源二：致動器動態。模擬中的馬達常被理想化為「指令什麼就輸出什麼」。真實的馬達有：延遲、力矩上限、摩擦、背隙、發熱降額、電池電壓下降（第 27.8.2 節）。

來源三：感測器缺陷（30.3.2 節的知識框）。雜訊、延遲、量化、偶發失效。

來源四：未建模的物理。空氣阻力、線材的拉扯力、結構的柔性、溫度影響、以及所有「你沒想到」的效應。

來源五：環境的差異。模擬中的世界是你建的，而真實世界永遠更複雜——地面不平、光線變化、有人走過、物體被移動。

表 30-6 Sim-to-Real Gap 的來源與嚴重程度

來源	嚴重程度	可否測量	因應策略
接觸與摩擦	最高	困難（且會變）	領域隨機化、柔順設計
致動器動態	高	可（實驗辨識）	系統辨識、加入延遲模型
感測器缺陷	高	可（統計量測）	在模擬中加入雜訊
未建模的物理	中	難	領域隨機化、保守設計
環境差異	中~高	部分可	場景隨機化、實地測試

30.4.2 哪些落差重要

一個關鍵的判斷是：不是所有落差都同等重要。判準是「這個落差會不會改變決策」。

舉例說明：

- 若模擬中的桌面顏色與實際不同——對力控制的抓取毫無影響，但對視覺辨識可能是致命

的。

- 若模擬中的馬達延遲少了 20 毫秒——對慢速的取放無影響，但對第 27 章的無人機姿態控制可能導致失穩。
- 若模擬中的摩擦係數高了一倍——對導航無影響，但對抓取是決定性的。

因此正確的作法是：先確認任務對哪些物理量敏感，再針對那些量下功夫。這需要做敏感度分析——系統性地改變各個參數，看性能如何變化（30.10 節的專題）。

30.4.3 一個常被誤解的觀念

最後要澄清一個核心的觀念，這正是開頭故事的重點：

◎ 模擬不必更真，策略必須更穩

許多人面對 Sim-to-Real 失敗時的第一反應是「把模擬做得更真」。這個方向有時有效，但往往是投資報酬率最低的路徑——因為真實世界的參數本身就在變動。若一個策略只在摩擦係數恰好為 0.6 時有效，那麼即使模擬完美無缺，它在真實世界遇到沾灰的物體時仍然會失敗。真正的問題不是模擬不夠真，而是策略太脆弱。因此更有效的方向是：讓策略在參數的整個合理範圍內都能運作。這就是下一節領域隨機化的核心思想，也是本書反覆出現的哲學——與其消除不確定性，不如設計對不確定性不敏感的系統（第 20 章的柔順控制、第 21 章的軟性夾爪、第 24 章的機率估測）。

30.5 領域隨機化

30.5.1 核心思想

領域隨機化（Domain Randomization）是目前跨越 Sim-to-Real Gap 最有效的方法。它的思想極為簡單卻極為有力：

◎ 領域隨機化的核心思想

不要試圖精確模擬真實世界，而是模擬「所有可能的世界」。每次訓練時隨機改變物理參數、外觀、感測雜訊。若一個策略能在這數百萬個各不相同的世界裡都成功，那麼真實世界對它而言，只不過是又一個它沒見過、但也不特別的世界。

這個轉換極為關鍵。傳統的思路是「縮小 sim 與 real 的距離」；領域隨機化則是「擴大 sim 的涵蓋範圍，使 real 落在其中」。

圖 30-1 領域隨機化的概念

建議配圖：兩格對照的示意圖。左格「傳統思路」——一個標示 *SIM* 的小圓點與一個標示 *REAL* 的小圓點相距一段距離，中間以雙向箭頭標註「努力縮小落差」，並註明「但 *REAL* 本身也在移動」（以虛線圓表示真實參數的變動範圍）。右格「領域隨機化」——一個涵蓋範圍極大的橢圓標示 *SIM*（內部散布著數百個小點，代表各種隨機參數組合），*REAL* 的圓點落在這個橢圓之內，標註「真實世界只是其中一個樣本」。下緣文字：不要縮小距離，而要擴大涵蓋。

30.5.2 該隨機化什麼

領域隨機化的對象可分為三類，各對應第 30.3.2 節的三個保真度維度：

表 30-7 領域隨機化的三類參數

類別	隨機化的對象	典型範圍	主要影響
動力學	質量、慣量、摩擦係數、關節阻尼、彈性	$\pm 20\sim 50\%$	控制與接觸行為
	致動器延遲、力矩上限、背隙	$\pm 10\sim 30\%$	控制的穩定性
感測	雜訊大小、延遲、量化、偶發失效	依實測分布	估測與感知的強健性
視覺	光照、材質、顏色、紋理、相機位姿	大範圍	視覺辨識的泛化
場景	物體位置、數量、種類、地形起伏	任務相關	策略的適應性

一個實務上的優先順序建議：先隨機化感測雜訊與致動器延遲（成本低、效果顯著）、再隨機化摩擦與質量（對接觸任務關鍵）、最後才是視覺（只在需要視覺的任務中重要）。

30.5.3 範圍該設多寬

這是領域隨機化最需要工程判斷的部分。範圍的設定有一個明確的取舍：

範圍太窄 → 涵蓋不到真實世界，策略仍會失敗

範圍太寬 → 任務變得過難，策略學不會或變得過度保守

這個取舍與第 24 章 *Q* 矩陣的設定、第 18 章 *LQR* 權重的選擇是同一類問題。

三個實用的原則：

- 以實測為中心：先量測真實系統的參數（30.6 節的系統辨識），再以該值為中心設定範圍。盲目地隨機化是無效的——若真實的摩擦是 0.4 而你隨機化 0.8~1.2，涵蓋不到就沒有意義。
- 寧寬勿窄：若不確定，寧可設寬一些。代價是訓練較慢、策略較保守，但至少能運作；設太窄則可能完全失敗。
- 漸進式擴大（Curriculum）：訓練初期用窄範圍（讓策略先學會基本行為），再逐步擴大。

這能顯著加速收斂——這與人類的學習方式一致：先在簡單環境練熟，再挑戰困難的。

30.5.4 實作範例

以下是一個典型的領域隨機化實作：

```
import numpy as np

class DomainRandomizer:
    """領域隨機化的參數產生器"""

    def __init__(self, nominal, curriculum_level=1.0):
        """
        nominal: 由系統辨識得到的標稱參數 (30.6 節)
        curriculum_level: 0.0~1.0，控制隨機化的強度
        """
        self.nom = nominal
        self.level = curriculum_level

    def _rand_scale(self, low, high):
        """依 curriculum 調整範圍: level 小時範圍窄"""
        mid = 1.0
        lo = mid + (low - mid) * self.level
        hi = mid + (high - mid) * self.level
        return np.random.uniform(lo, hi)

    def sample(self):
        """為一次訓練回合產生一組隨機參數"""
        p = {}

        # ---- 動力學參數 (以標稱值為中心) ----
        p['mass'] = self.nom['mass'] * self._rand_scale(0.7, 1.3)
        p['friction'] = self.nom['friction'] * self._rand_scale(0.5, 1.8)
        p['damping'] = self.nom['damping'] * self._rand_scale(0.6, 1.5)
        p['inertia_scale'] = self._rand_scale(0.8, 1.2)

        # ---- 致動器 (延遲與力矩上限) ----
        p['actuator_delay'] = np.random.uniform(0, 0.02 * self.level) # 0~20 ms
        p['torque_limit'] = self.nom['torque_limit'] * self._rand_scale(0.8, 1.1)
        p['motor_gain'] = self._rand_scale(0.9, 1.1)

        # ---- 感測雜訊 (依實測的分布) ----
        p['imu_noise_std'] = self.nom['imu_noise'] * self._rand_scale(0.5, 2.0)
        p['enc_quantize'] = self.nom['enc_res']
        p['sensor_delay'] = np.random.uniform(0, 0.01 * self.level)
        p['dropout_prob'] = np.random.uniform(0, 0.02 * self.level) # 資料遺失

        # ---- 外部擾動 ----
        p['ext_force'] = np.random.normal(0, 5.0 * self.level, size=3) # N
        p['ground_tilt'] = np.random.uniform(-0.05, 0.05) * self.level # rad
```

```

    return p

def set_level(self, level):
    """訓練過程中逐步提高難度 ( curriculum ) """
    self.level = np.clip(level, 0.0, 1.0)

# ----- 使用方式 -----
nominal_params = {
    'mass': 2.5, 'friction': 0.6, 'damping': 0.05,
    'torque_limit': 10.0, 'imu_noise': 0.01, 'enc_res': 0.001,
}
randomizer = DomainRandomizer(nominal_params, curriculum_level=0.2)

for episode in range(1_000_000):
    # 逐步提高隨機化強度 ( 前 20% 的訓練用於暖身 )
    randomizer.set_level(min(1.0, episode / 200_000))
    params = randomizer.sample()
    env.reset(params)          # 用這組參數重設模擬環境
    # ... 執行一回合的訓練 ...

```

三個值得注意的設計：

- **以標稱值為中心**：所有隨機化都是「標稱值 × 隨機倍率」，而非憑空取值。這確保涵蓋範圍是有意義的。
- **curriculum 逐步提高**：訓練初期 level 小、範圍窄；隨訓練進行逐步擴大。
- **包含感測缺陷與外部擾動**：不只是動力學參數，還有雜訊、延遲、資料遺失與外力——這些往往比物理參數更關鍵。

30.5.5 為何有效：一個直觀的解釋

領域隨機化為何能奏效？有兩個互補的解釋。

解釋一：把「未知參數」變成「觀測不到的干擾」。當策略無法依賴某個參數的特定值時，它被迫學會依即時的回授來調整——也就是學會了一個更接近「控制器」而非「開迴路播放」的行為。這與第 16 章「回授對抗未知干擾」的原理相通。

解釋二：隱含的系統辨識。若策略能觀察一段歷史（如過去 0.5 秒的狀態序列），它其實可以從行為中推斷出當前的參數——例如「我施加了這個力矩卻只動了這麼多，看來這次比較重」。這使策略具備了線上適應的能力。這正是第 28.9.3 節「教師—學生」架構要教會學生的東西。

30.6 系統辨識

30.6.1 與領域隨機化的互補

領域隨機化需要一個「標稱值」作為中心 (30.5.3 節) 。而取得這個值，就是**系統辨識** (system identification) 的工作。

兩者是**互補而非競爭**的：

系統辨識 → 找出參數的中心值
領域隨機化 → 在中心值周圍設定合理的範圍

先辨識、再隨機化，這是標準的流程。

若跳過辨識而盲目隨機化，可能整個範圍都涵蓋不到真實值 (30.5.3 節的原則一) ；若只辨識而不隨機化，則策略會過度依賴那個特定的值。

30.6.2 辨識什麼、怎麼辨識

第 13.10 節已介紹過動力學參數的辨識，此處整理模擬相關的重點：

表 30-8 常見的系統辨識項目

參數	辨識方法	難度
質量與質心	秤重、平衡點法	低
慣性張量	三線擺、CAD 估算	中
關節摩擦	等速運動下的力矩 (第 13.8.2 節)	中
致動器延遲	階躍指令與實際響應的時間差	低但常被忽略
力矩常數	電流與力矩的關係	低
接觸摩擦係數	斜面法 (第 21 章實作)	中 (且會變)
感測器雜訊	靜止時的統計量測	低，應必做
感測器延遲	與已知訊號的互相關	中

最容易被忽略但最有價值的三項：致動器延遲、感測器雜訊、感測器延遲。它們都很容易量測 (一個下午即可完成) ，但對 Sim-to-Real 的影響極大。

30.6.3 由軌跡資料反推參數

更系統性的作法是：讓真實機器人執行一段**激勵軌跡**，記錄輸入與輸出，再以最佳化反推參

數，使模擬的行為與實測吻合。

$$\min_{\theta} \sum_t ||x_{\text{real}}(t) - x_{\text{sim}}(t; \theta)||^2$$

參數辨識的最佳化形式。 θ 為待辨識的參數、 x 為狀態軌跡。

這與第 22.3 節的相機校正、第 25.6 節的圖最佳化是同一類問題——都是「調整參數使預測符合觀測」。

激勵軌跡的設計是關鍵（呼應第 19.10.2 節的持續激勵條件）：軌跡必須足夠「豐富」，才能把各個參數分離出來。若機器人只做單一方向的等速運動，就無法辨識出慣量（因為加速度為零）。

◎ 可微分模擬的新可能

第 14.8.3 節提過的**可微分模擬**，為系統辨識帶來了新工具。傳統的辨識需要反覆試錯或使用無梯度的最佳化；可微分模擬則能直接計算「模擬結果對參數的梯度」，使辨識能用梯度下降高效求解。這使得辨識**數十個參數**成為可能（傳統方法通常只能處理少數幾個）。但它的限制也如第 14.8.3 節所述——接觸的不連續性使梯度不可靠，因此目前主要用於接觸較少的場景。

30.7 常見模擬器的比較

30.7.1 主流選擇

第 14.8.2 節已比較過物理引擎，此處從**完整模擬器**的角度比較（模擬器 = 物理引擎 + 渲染 + 感測器模型 + 介面）：

表 30-9 常見機器人模擬器的比較

模擬器	物理引擎	渲染	ROS 2 整合	平行能力	主要優勢	典型用途
Gazebo	多種可選	中等	原生支援	低	ROS 生態最成熟	導航、SLAM、系統整合
Isaac Sim	PhysX (GPU)	極佳 (光追)	支援	極高 (GPU)	大規模平行、視覺逼真	強化學習、合成資料
MuJoCo	自有 (軟接觸)	簡單	需自行整合	高	接觸平滑可微、極快	控制研究、RL
Webots	ODE	良好	支援	低	易上手、教學友善	教學、競賽
PyBullet	Bullet	簡單	需自行	中	輕量、Python	快速原型、RL 入

模擬器	物理引擎	渲染	ROS 2 整合	平行能力	主要優勢	典型用途
			整合		友善	門
CoppeliaSim	多種可選	良好	支援	低	圖形化編輯、機構豐富	工業應用、教學

30.7.2 選擇的判準

選擇模擬器應依**主要用途**而非「哪個最好」：

- 做 ROS 系統整合與導航 → **Gazebo**。它與 ROS 2 的整合最深，感測器外掛最完整，且社群範例最多。
- 做大規模強化學習 → **Isaac Sim** 或 **MuJoCo**。前者能在單張 GPU 上跑數千個環境，後者速度極快且接觸模型平滑（有利於學習）。
- 做控制演算法研究 → **MuJoCo**。它的接觸模型可微且穩定，適合需要精確動力學的研究。
- 教學或快速原型 → **Webots** 或 **PyBullet**。前者圖形化易上手，後者用純 Python 幾行就能跑起來。
- 做視覺感知訓練 → **Isaac Sim**。它的光線追蹤渲染能產生接近照片級的合成影像。

一個常見的實務模式是「用兩個」：用 MuJoCo 或 Isaac 做大量的訓練（速度優先），再用 Gazebo 做系統整合驗證（生態優先）。因為第 29.3.2 節的解耦設計，控制器程式碼不需要修改。

30.7.3 模擬器與 ROS 2 的介接

模擬器與 ROS 2 的整合，本質上是讓模擬器**扮演硬體驅動節點的角色**——發布同樣的話題、訂閱同樣的指令：

```
真實硬體：驅動節點 → /scan, /odom, /joint_states ; ← /cmd_vel
模擬器：模擬節點 → /scan, /odom, /joint_states ; ← /cmd_vel
```

兩者對上層完全一樣。這正是第 29.3.2 節解耦設計的價值。

因此切換模擬與實機，理想上只需要改變啟動檔中的一行——這也是為何第 29.5.3 節的啟動檔要設計 `use\_sim` 這個參數。

一個重要的細節：**時間來源**。模擬中的時間可能比真實時間快或慢，因此所有節點必須使用**模擬時間**而非系統時鐘。ROS 2 透過 `/clock` 話題與 `use\_sim\_time` 參數處理這件事——忘記設

定它是初學者常見的錯誤，症狀是 TF 查詢一直失敗或行為異常。

30.8 大規模平行模擬

30.8.1 為何需要平行

第 30.1.1 節算過：十萬次抓取嘗試在真機上需要六天。即使模擬能快十倍，仍需十四小時。而第 31 章的強化學習往往需要數百萬次互動——這使平行化成為必要。

平行模擬的思想很直接：同時跑 N 個獨立的環境，每個環境各自進行。若 $N = 4096$ ，理論上可以把訓練時間縮短四千倍。

30.8.2 CPU 平行與 GPU 平行

平行化有兩種截然不同的作法：

CPU 平行：在多核心的 CPU 上開多個行程，每個跑一個模擬。這是傳統的作法。

- **優點**：實作簡單、可用任何模擬器、環境之間完全獨立。
- **缺點**：受限於 CPU 核心數（典型 8~64），且行程間的資料傳輸有開銷。

GPU 平行：把整個模擬移到 GPU 上，用向量化的方式同時計算數千個環境。這是近年的突破（如 Isaac Gym）。

- **優點**：可達數千甚至上萬個環境、且資料直接留在 GPU 上——訓練的神經網路也在 GPU 上，省去了資料搬移的開銷。
- **缺點**：需要 GPU 友善的物理實作（不是所有引擎都支援）、環境必須同構（相同的機器人與場景結構）、且除錯較困難。

◎ 為何 GPU 平行是一次典範轉移

在 Isaac Gym 之前，強化學習的瓶頸通常在**模擬**而非**訓練**——CPU 跑模擬跑不過 GPU 訓練網路。研究者必須用數十台電腦組成叢集才能收集足夠的資料。GPU 平行模擬改變了這件事：現在**單張 GPU 就能同時跑數千個環境**，且資料不需要在 CPU 與 GPU 之間搬移。這使得原本需要數天與一整個叢集的訓練，變成數小時與一台工作站。第 28.9 節腿式機器人的強化學習突破，很大程度上是這個技術突破的結果——不是演算法變聰明了，而是能試的次數多了千倍。

30.8.3 平行模擬的實作要點

三個實務上的關鍵：

要點一：環境的異質性。若所有環境都一樣，平行只是重複同樣的經驗，價值有限。**應搭配領域隨機化**——每個環境用不同的參數、不同的初始狀態、不同的場景配置。**平行 × 隨機化**，才能產生真正多樣的經驗。

要點二：非同步重置。各環境的回合長度不同（有的早早失敗、有的跑完全程）。若等所有環境都結束才一起重置，會浪費大量算力。正確的作法是**個別重置**——某個環境結束就立刻重來，不影響其他環境。

要點三：批次的資料流。所有環境的觀測應該以一個大批次（batch）送進神經網路，一次前向傳播處理全部——這正是 GPU 的強項。若逐一處理，GPU 的平行能力就浪費了。

```
# 平行環境的典型架構（概念性程式碼）
import numpy as np

class ParallelEnv:
    """N 個平行環境的向量化介面"""

    def __init__(self, num_envs=4096, randomizer=None):
        self.N = num_envs
        self.randomizer = randomizer
        # 所有環境的狀態存成一個大陣列（GPU 上則為 tensor）
        self.states = np.zeros((self.N, STATE_DIM))
        self.params = [None] * self.N
        self.step_count = np.zeros(self.N, dtype=int)

    def reset_all(self):
        for i in range(self.N):
            self._reset_one(i)
        return self.states.copy()

    def _reset_one(self, i):
        """個別重置：每個環境用不同的隨機參數（要點一）"""
        self.params[i] = self.randomizer.sample()
        self.states[i] = self._sample_initial_state()
        self.step_count[i] = 0

    def step(self, actions):
        """一次推進所有環境（批次處理，要點三）"""
        # 向量化的物理計算—GPU 上這是一次核心呼叫
        self.states = self._physics_step(self.states, actions, self.params)
        self.step_count += 1

        rewards = self._compute_rewards(self.states, actions)
        dones = self._check_termination(self.states, self.step_count)

        # 非同步重置：只重置已結束的環境（要點二）
```



```

    for i in np.where(dones)[0]:
        self._reset_one(i)

    return self.states.copy(), rewards, dones

# ----- 訓練迴圈 -----
env = ParallelEnv(num_envs=4096, randomizer=randomizer)
obs = env.reset_all()

for iteration in range(10000):
    # 一次前向傳播處理 4096 個環境的觀測
    actions = policy(obs) # shape: (4096, ACTION_DIM)
    obs, rewards, dones = env.step(actions)
    # ... 更新策略 ...

    # 每次迭代收集 4096 筆經驗——相當於 4096 台真實機器人
    if iteration % 100 == 0:
        print(f'迭代 {iteration}: 已收集 {(iteration+1)*4096:,} 筆經驗')

```

注意最後的計算：一萬次迭代 $\times$ 4096 個環境 = 4096 萬筆經驗。若在真機上收集（每筆 0.1 秒），需要 47 年。這就是平行模擬的價值。

30.9 漸進式驗證流程

30.9.1 不要一步跨到實機

開頭故事中「98% 變成 20%」的挫折，很大一部分來自**驗證流程的缺失**——直接從模擬跳到實機。正確的作法是**分階段漸進**，每一階段驗證不同的東西。

表 30-10 漸進式驗證的五個階段

階段	環境	驗證什麼	失敗時的診斷
一、單元測試	純程式（無物理）	邏輯正確性、邊界條件	程式錯誤
二、理想模擬	無雜訊、參數精確	演算法本身可行	演算法設計問題
三、隨機化模擬	領域隨機化開啟	對參數變異的強健性	策略過度依賴特定參數
四、硬體在環	真實控制器 + 模擬機器人	時序、通訊、延遲	軟體架構或即時性問題
五、實機受限測試	真機但低速、有防護	真實的物理與感測	未建模的效應
六、實機完整測試	真機、正常條件	整體性能	—

每一階段的價值在於「隔離變因」。若在階段三失敗，你知道是策略不夠強健（而非硬體問題）；若階段三通過但階段四失敗，問題就在通訊與時序。這使除錯有明確的方向。

◎ 硬體在環（HIL）：最被低估的一步

階段四的**硬體在環**常被跳過，但它能捕捉一類模擬永遠測不到的問題：**時序與通訊**。在這個階段，真實的控制器（如飛控板、工控機）連接到模擬的機器人——控制器不知道自己接的是假的。這能測出：控制迴路是否真的能在 1 ms 內完成、通訊是否會塞車、CPU 負載是否過高、以及第 29.10.2 節的執行器阻塞問題。這些在純軟體模擬中完全不會出現（因為模擬會等你算完），卻是實機失敗的常見原因。

30.9.2 量化落差

每一階段之間，應該**量化**性能的下降。這比「感覺變差了」有用得多：

$$\text{落差} = (\text{模擬性能} - \text{實機性能}) / \text{模擬性能}$$

以成功率為例：98% → 20% 的落差是 79.6%。

追蹤這個數字的變化，能判斷改善是否有效。例如：

- 加入領域隨機化後，落差由 80% 降到 30% → **策略的強健性改善了**。
- 做了系統辨識後，落差由 30% 降到 15% → **標稱參數更準了**。
- 加入感測雜訊模擬後，落差由 15% 降到 8% → **感知的強健性改善了**。

若某項改善沒有降低落差，就代表你找錯了方向——這是避免浪費時間的關鍵判準（呼應 30.4.2 節「哪些落差重要」）。

30.9.3 實機測試的安全原則

進入階段五時，必須有明確的安全措施（呼應第 7.9 節）：

- 降速執行**：以正常速度的 10~30% 開始，確認行為正常後再逐步提高。
- 限制工作空間**：用軟體邊界限制機器人的活動範圍。
- 人在迴路**：手持緊急停止，且操作者的注意力不可分散。
- 先用替代物**：抓取測試先用便宜的替代品（保麗龍塊而非玻璃杯）。
- 全程錄製**：用 rosbag 記錄所有資料（第 29.9.3 節）——失敗時才有資料可分析。

30.10 數位學生

30.10.1 與一般模擬的差異

數位學生 (Digital Twin) 近年成為工業界的熱門概念，但它常被誤用為「模擬」的同義詞。兩者有一個關鍵的差異：

模擬：一個獨立的虛擬環境，用於測試「如果.....會怎樣」

數位學生：與「特定的實體」持續同步的虛擬複本

關鍵在於「同步」。數位學生會持續接收實體的資料並更新自己。

這個差異帶來不同的用途：

表 30-11 模擬與數位學生的比較

面向	一般模擬	數位學生
對象	一類機器人 (通用模型)	特定的那一台
資料流	單向 (模擬 → 分析)	雙向持續同步
參數	標稱值或隨機	由實測持續更新
主要用途	開發、訓練、測試	監控、預測、最佳化
典型問題	「這個演算法可行嗎？」	「這台機器什麼時候該保養？」

30.10.2 三個實務應用

應用一：預測性維護。持續比對實體與學生的行為差異。若某個關節需要的力矩逐漸增加 (而學生模型顯示不該如此)，代表軸承或減速機可能在劣化——在故障發生前就發出警示。

應用二：離線編程與驗證。在學生上測試新的程式，確認無誤後才下載到實體。這對產線極為重要——停機測試的代價極高。

應用三：遠端監控與診斷。專家不需要到現場，透過學生就能看見機器的狀態並診斷問題。

30.10.3 建立數位學生的要件

一個有效的數位學生需要三樣東西：

- **準確的初始模型：**由 CAD 與系統辨識 (30.6 節) 建立。
- **持續的資料連結：**實體必須把感測資料傳給學生。這正是第 29 章 ROS 2 架構的用途之一——同樣的話題，一份給控制器、一份給學生。

- **參數的線上更新**：用實測資料持續修正模型（如第 24 章的估測技術）。

數位學生的價值與其同步的精度成正比。若模型與實體差距太大，比對就沒有意義——這使得 30.6 節的系統辨識成為數位學生的必要基礎。

30.11 由模擬邁向學習

30.11.1 本章的成果

本章建立了模擬的正確使用方式。你現在能夠：說明模擬的三個價值與四種用途、理解物理引擎的計算瓶頸與時間步長的取舍、依需求選擇適當的保真度、分析 Sim-to-Real Gap 的五個來源、實作領域隨機化與系統辨識、選擇適合的模擬器、運用大規模平行模擬、並設計漸進式的驗證流程。

最重要的是那個貫穿全章的觀念轉換：**模擬不必更真，策略必須更穩**。追求完美的模擬是一條報酬遞減的路；讓策略在**參數的整個合理範圍內**都能運作，才是根本的解法。

這個哲學與全書多處呼應——第 20 章用柔順取代精確、第 21 章用軟性夾爪取代精密定位、第 24 章用機率描述取代確定值。它們說的是同一件事：**與其消除不確定性，不如設計對它不敏感的系統**。

30.11.2 接下來：讓機器人自己學

本章建立的模擬環境，其最大的用途將在下一章展現。

第 31 章（機器學習與強化學習）將處理「機器人如何自己學會技能」。我們會看到：監督式學習如何用於感知（第 21.9 節的抓取姿勢偵測、第 22.8 節的物體辨識）、模仿學習如何從人類示範中學習、以及強化學習如何透過試錯自主發展行為。

而**強化學習正是本章模擬技術的最大受益者**——它需要數百萬次的試錯，這在真機上不可能，只有靠 30.8 節的大規模平行模擬才成為可行。第 28.9 節腿式機器人的突破、第 21.9.3 節的抓取資料集，都建立在這個基礎上。

屆時我們也會回收第 18 章的伏筆——**強化學習與最佳控制共用同一條 Bellman 方程**，差別只在於模型是否已知。

換句話說：第 30 章提供了**練習的場地**，第 31 章則教機器人**如何在其中學習**。

完整計算例題

例題 30-1 模擬速度與訓練時間

題目：某抓取任務每次嘗試需 4 秒。強化學習需要 200 萬次嘗試才能收斂。(a) 在真機上訓練需要多久？(b) 若模擬的 RTF 為 20，單環境訓練需要多久？(c) 若改用 GPU 平行，同時跑 2048 個環境且總 RTF 為 8000，需要多久？(d) 若加入領域隨機化使所需的嘗試次數增為 3 倍，重新計算 (c)。

解 (a)：真機訓練時間：

$$2,000,000 \times 4 \text{ 秒} = 8,000,000 \text{ 秒} = 2,222 \text{ 小時} = 92.6 \text{ 天}$$

而且這還沒算人工重置場景的時間。實務上不可行。

解 (b)：RTF = 20 代表模擬時間比真實時間快 20 倍：

$$8,000,000 / 20 = 400,000 \text{ 秒} = 111 \text{ 小時} = 4.6 \text{ 天}$$

仍然太慢——每次調整超參數就要等近五天。

解 (c)：總 RTF 為 8000 (2048 個環境，每個環境的等效 RTF 約 3.9)：

$$8,000,000 / 8,000 = 1,000 \text{ 秒} = 16.7 \text{ 分鐘} \checkmark$$

由 92.6 天縮短到 16.7 分鐘——這正是 GPU 平行模擬帶來的典範轉移 (30.8.2 節的知識框)。

解 (d)：領域隨機化使任務變難，需要更多次嘗試：

$$\begin{aligned} 2,000,000 \times 3 &= 6,000,000 \text{ 次} \\ 6,000,000 \times 4 / 8,000 &= 3,000 \text{ 秒} = 50 \text{ 分鐘} \checkmark \end{aligned}$$

仍在可接受範圍內。

關鍵的工程判讀：領域隨機化的代價是訓練時間增為 3 倍 (16.7 分鐘 → 50 分鐘)，但換來的是 **Sim-to-Real 的成功**。以本章開頭的故事為例，若不做隨機化，訓練再快也只有 20% 的實機成功率——快速地得到一個沒用的策略，並不是效率。

例題 30-2 落差的量化與歸因

題目：某團隊開發抓取策略，記錄了不同階段的成功率：理想模擬 96%、加入感測雜訊 91%、加入動力學隨機化 84%、實機測試 71%。(a) 計算各階段之間的性能下降。(b) 計算總落差。(c) 判斷主要的落差來源。(d) 若要進一步改善，建議下一步該做什麼。

解 (a)：逐階段的下降：

表 30-12 各階段的性能變化

階段	成功率	相對前一階段的下降	累積下降
理想模擬	96%	—	—
+ 感測雜訊	91%	5.2%	5.2%
+ 動力學隨機化	84%	7.7%	12.5%
實機測試	71%	15.5%	26.0%

解 (b)：總落差 (以理想模擬為基準)：

$$\text{落差} = (96 - 71)/96 = 26.0\%$$

相對於 30.4 節開頭故事的 79.6%，這已經是相當好的結果。

解 (c)：最大的單一下降出現在「隨機化模擬 → 實機」這一步 (15.5%)。這代表：

- 感測雜訊與動力學隨機化**確實有效**——它們各自吸收了 5.2% 與 7.7% 的落差 (若不做，這些下降會在實機階段一次爆發)。
- 但仍有 15.5% 的落差**未被模擬涵蓋**——存在某種隨機化沒有涵蓋到的因素。

解 (d)：建議的診斷步驟：

- 分析失敗案例：錄下實機失敗的 rosbag，逐一檢視。是抓空？滑落？還是碰撞？不同的失敗模式指向不同的原因。
- 擴大隨機化範圍：若失敗集中在特定的物體 (如較滑的)，可能是摩擦係數的範圍設得不夠寬。
- 檢查未隨機化的項目：是否有遺漏的因素？常見的是致動器延遲與視覺的光線變化。
- 做敏感度分析：在模擬中系統性地改變各參數，找出策略最敏感的那一個，優先擴大它的範圍。

一個實務判準：若擴大某個參數的隨機化範圍後，模擬成功率明顯下降但實機成功率上升，代表那個參數正是關鍵。反之若模擬與實機都沒變，代表方向錯了。

例題 30-3 保真度與計算成本的取舍

題目：某模擬環境的計算時間分配為：碰撞偵測 45 ms、約束求解 20 ms、動力學 8 ms、

渲染 35 ms、感測器模擬 12 ms (每個模擬步驟，步長 5 ms)。(a) 計算目前的 RTF。(b) 若任務不需要視覺，關閉渲染後的 RTF。(c) 若把碰撞網格由 5000 面簡化為 200 面 (碰撞時間降為 1/8)，重新計算。(d) 若目標是 $RTF \geq 50$ ，判斷是否達成並提出建議。

解 (a)：每步的總計算時間：

$$45 + 20 + 8 + 35 + 12 = 120 \text{ ms}$$

模擬一個 5 ms 的步長，需要 120 ms 的計算。

$$RTF = 5 \text{ ms} / 120 \text{ ms} = 0.0417$$

RTF 僅 0.042——模擬比現實慢 24 倍。這幾乎無法使用。

解 (b)：關閉渲染 (省下 35 ms)：

$$45 + 20 + 8 + 12 = 85 \text{ ms} \Rightarrow RTF = 5/85 = 0.0588$$

改善了 41%，但仍遠低於實用水準。

解 (c)：碰撞時間由 45 ms 降為 $45/8 = 5.6 \text{ ms}$ ：

$$5.6 + 20 + 8 + 12 = 45.6 \text{ ms} \Rightarrow RTF = 5/45.6 = 0.110$$

再改善 87%。碰撞簡化的效果比關閉渲染更顯著——這印證了 30.2.1 節「碰撞偵測通常是最大瓶頸」的說法。

解 (d)：目前 $RTF = 0.110$ ，距離目標 50 還差 455 倍。單靠上述優化不可能達成。三個建議：

- **平行化 (最有效)**：跑 512 個環境，總吞吐量即可達到 $0.110 \times 512 = 56.3$ —— 超過目標。這說明了為何 30.8 節的平行化如此重要：與其把單一環境優化到極限，不如同時跑很多個。
- **加大時間步長**：若接觸不多，步長可由 5 ms 加大到 10 ms，RTF 直接加倍。但需確認不會發生穿透 (第 14.6.3 節)。
- **簡化約束求解**：20 ms 是第二大項。減少迭代次數或降低接觸點數量可以改善，但會影響精度——這是保真度與速度的直接取捨。

工程結論：優化的優先順序應為「平行化 > 碰撞簡化 > 步長 > 關閉不需要的模組」。而最重要的判斷是：先確認任務真正需要什麼保真度 (30.3 節)，不需要的一律關閉。

程式範例：領域隨機化的效果驗證

以下程式以一個簡化的抓取任務為例，比較「無隨機化」與「有隨機化」訓練出的策略，在參數變動下的表現差異。

```
import numpy as np

# ===== 簡化的抓取模擬 =====
class SimpleGraspEnv:
    """
    極簡的抓取模型：夾爪以某個力夾住物體，
    若「摩擦力 × 夾持力」足以支撐重量則成功。
    這是第 21 章力封閉的一維簡化。
    """
    def __init__(s, mu=0.6, mass=0.5, sensor_bias=0.0,
                 actuator_gain=1.0, g=9.8):
        s.mu = mu                # 摩擦係數
        s.mass = mass            # 物體質量
        s.sensor_bias = sensor_bias # 感測器偏差 (估計的質量誤差)
        s.gain = actuator_gain    # 致動器增益誤差
        s.g = g

    def observe(s):
        """策略能看到的資訊 (帶偏差的質量估計)"""
        return s.mass * (1 + s.sensor_bias)

    def execute(s, commanded_force):
        """執行夾持，回傳是否成功"""
        actual_force = commanded_force * s.gain    # 致動器誤差
        friction_force = s.mu * actual_force        # 最大摩擦力
        weight = s.mass * s.g
        # 成功條件：摩擦力足以支撐重量，且不過度用力壓壞
        holds = friction_force >= weight
        crushes = actual_force > 50.0              # 超過 50 N 壓壞
        return holds and not crushes

# ===== 兩種策略 =====
class NaivePolicy:
    """在標稱參數下訓練——假設  $\mu$  恰好是 0.6"""
    def __init__(s, mu_assumed=0.6, safety=1.05):
        s.mu = mu_assumed
        s.safety = safety          # 只留 5% 餘裕

    def act(s, observed_mass):
        #  $F = mg / \mu$ ，只留很小的餘裕
        return observed_mass * 9.8 / s.mu * s.safety
```



```

class RobustPolicy:
    """在隨機化環境中訓練——學會保留足夠餘裕"""
    def __init__(s, mu_worst=0.35, safety=1.25):
        s.mu = mu_worst          # 假設最差的摩擦
        s.safety = safety         # 留 25% 餘裕

    def act(s, observed_mass):
        return observed_mass * 9.8 / s.mu * s.safety

# ===== 評估 =====
def evaluate(policy, n_trials=10000, randomize=True, seed=0):
    """在隨機的真實參數下測試策略"""
    rng = np.random.default_rng(seed)
    success = 0
    for _ in range(n_trials):
        if randomize:
            # 真實世界的參數分布
            env = SimpleGraspEnv(
                mu=rng.uniform(0.30, 0.90),          # 摩擦變動大
                mass=rng.uniform(0.3, 0.8),
                sensor_bias=rng.normal(0, 0.08),      # 8% 的估計誤差
                actuator_gain=rng.uniform(0.85, 1.10))
        else:
            env = SimpleGraspEnv()                   # 標稱參數
            if env.execute(policy.act(env.observe())):
                success += 1
    return success / n_trials

print("=== 兩種策略在不同環境下的成功率 ===")
naive, robust = NaivePolicy(), RobustPolicy()
print(f"{'策略':<20}{'理想環境':>12}{'隨機環境':>12}{'落差':>10}")
for name, pol in (("天真策略 (無隨機化)", naive),
                  ("強健策略 (有隨機化)", robust)):
    ideal = evaluate(pol, randomize=False)
    real = evaluate(pol, randomize=True)
    gap = (ideal - real)/ideal*100 if ideal > 0 else 0
    print(f"{name:<20}{ideal:11.1%}{real:11.1%}{gap:9.1f}%")
print("→ 天真策略在理想環境完美，但真實環境崩潰 (重現開頭故事)")

# ===== 逐項分析落差來源 =====
print("\n=== 落差的來源分析 (每次只隨機化一項) ===")
def eval_single(policy, vary, n=10000, seed=1):
    rng = np.random.default_rng(seed)
    kw = dict(mu=0.6, mass=0.5, sensor_bias=0.0, actuator_gain=1.0)
    ok = 0
    for _ in range(n):
        p = dict(kw)
        if vary == 'mu':      p['mu'] = rng.uniform(0.30, 0.90)
        if vary == 'mass':    p['mass'] = rng.uniform(0.3, 0.8)
        if vary == 'sensor':  p['sensor_bias'] = rng.normal(0, 0.08)
        if vary == 'actuator':p['actuator_gain'] = rng.uniform(0.85, 1.10)
        env = SimpleGraspEnv(*p)

```

```

        if env.execute(policy.act(env.observe())): ok += 1
    return ok/n

print(f"{'隨機化的項目':<16}{'天真策略':>12}{'強健策略':>12}")
for vary, label in (('none', '無 (理想)'), ('mu', '摩擦係數'),
                    ('mass', '質量'), ('sensor', '感測偏差'),
                    ('actuator', '致動器增益')):
    a = eval_single(naive, vary)
    b = eval_single(robust, vary)
    print(f"{label:<16}{a:11.1%}{b:11.1%}")
print("> 摩擦係數是最主要的落差來源 ( 呼應 30.4.1 節 ) ")

# ===== 隨機化範圍的取舍 ( 30.5.3 節 ) =====
print("\n=== 隨機化範圍對策略的影響 ===")
print(f"{'訓練時假設的最差  $\mu$ ':>20}{'餘裕':>8}"
      f"{'理想環境':>11}{'真實環境':>11}{'平均用力':>11}")
for mu_train, safety in ((0.60, 1.05), (0.50, 1.10),
                        (0.40, 1.20), (0.35, 1.25), (0.25, 1.40)):
    pol = RobustPolicy(mu_worst=mu_train, safety=safety)
    ideal = evaluate(pol, randomize=False)
    real = evaluate(pol, randomize=True)
    # 平均施力 ( 衡量是否過度保守 )
    avg_force = np.mean([pol.act(m) for m in np.linspace(0.3, 0.8, 100)])
    print(f"{mu_train:19.2f}{safety:8.2f}{ideal:10.1%}"
          f"{real:10.1%}{avg_force:10.1f} N")
print("> 假設越保守 · 真實成功率越高 · 但用力也越大 ( 可能壓壞物體 ) ")
print("> 這正是 30.5.3 節「範圍太窄涵蓋不到、太寬過度保守」的取舍")

# ===== 漸進式驗證的模擬 ( 30.9 節 ) =====
print("\n=== 漸進式驗證：逐階段加入不確定性 ===")
stages = [
    ("階段二：理想模擬", dict(mu=(0.6,0.6), sb=0.00, ag=(1.0,1.0))),
    ("階段三 a：+感測雜訊", dict(mu=(0.6,0.6), sb=0.08, ag=(1.0,1.0))),
    ("階段三 b：+致動器", dict(mu=(0.6,0.6), sb=0.08, ag=(0.85,1.10))),
    ("階段三 c：+摩擦變動", dict(mu=(0.3,0.9), sb=0.08, ag=(0.85,1.10))),
]
def eval_stage(policy, cfg, n=10000, seed=2):
    rng = np.random.default_rng(seed)
    ok = 0
    for _ in range(n):
        env = SimpleGraspEnv(
            mu=rng.uniform(*cfg['mu']),
            mass=rng.uniform(0.3, 0.8),
            sensor_bias=rng.normal(0, cfg['sb']),
            actuator_gain=rng.uniform(*cfg['ag']))
        if env.execute(policy.act(env.observe())): ok += 1
    return ok/n

print(f"{'階段':<20}{'天真策略':>12}{'強健策略':>12}{'下降':>10}")
prev_a = prev_b = None
for label, cfg in stages:

```

```

a, b = eval_stage(naive, cfg), eval_stage(robust, cfg)
drop = f"{{(prev_a-a)/prev_a*100:.1f}}%" if prev_a else "-"
print(f"{{label:<20}}{{a:11.1%}}{{b:11.1%}}{{drop:>10}}")
prev_a, prev_b = a, b
print("> 逐階段測試能明確指出「哪一項因素造成最大的下降」")
print("> 本例中摩擦變動是關鍵—應優先擴大其隨機化範圍")

```

請試著修改：（一）把 RobustPolicy 的 safety 提高到 1.6，觀察是否開始出現「壓壞物體」的失敗。（二）加入一個「自適應策略」——它能觀察前一次的結果並調整力道，比較其表現。（三）把真實環境的摩擦範圍改為 0.2~0.5（比訓練時假設的更差），觀察強健策略是否仍然有效，驗證 30.5.3 節「涵蓋不到就無效」的原則。

實作：建立模擬環境與驗證流程

本章實作以 Gazebo 或 PyBullet 為主。若已完成第 29 章的 ROS 2 環境，可直接延續。

- 環境建置**：安裝模擬器（建議 Gazebo 搭配 ROS 2，或 PyBullet 用於快速實驗）。載入一個現成的機器人模型並確認能運行。
- RTF 量測**：量測你的模擬環境的即時因子。用 `ros2 topic hz /clock` 或模擬器內建的統計功能。記錄下來作為後續優化的基準。
- 碰撞簡化實驗**：把機器人的碰撞模型由精細網格改為簡化的幾何（膠囊、盒子），重新量測 RTF。驗證 30.2.1 節與例題 30-3 的結論——你應該會看到顯著的提升。
- 時間步長實驗**：逐步加大物理步長（1、2、5、10、20 ms），記錄 RTF 與行為的變化。找出「開始出現穿透或不穩」的臨界步長。
- 感測器雜訊模擬**：為模擬的感測器加入雜訊（多數模擬器支援）。用實測的統計值（第 24 章實作量測過的）設定參數。
- 系統辨識**：對你的實體機器人（或一個已知的模型）做簡單的辨識——量測質量、致動器延遲、感測器雜訊。**特別是延遲**，用階躍指令與實際響應的時間差即可測得。
- 領域隨機化實作**：實作 30.5.4 節的隨機化器，在每次重設環境時套用。從窄範圍開始，逐步擴大。
- 敏感度分析**：系統性地改變單一參數（其餘固定），記錄性能變化。繪出「參數 vs 性能」曲線，找出**最敏感的參數**。這決定了隨機化的優先順序。
- 漸進式驗證**：依 30.9.1 節的五個階段逐步測試你的控制器或策略。在每個階段記錄性能，

繪出「階段 vs 性能」的下降曲線。

10. 硬體在環（若有硬體）：把真實的控制器（如 Arduino 或工控機）接到模擬器，驗證時序與通訊。觀察是否出現純軟體模擬中沒有的問題。

11. （進階）平行模擬：實作多環境的平行模擬（可用 Python 的 multiprocessing 或 PyBullet 的多實例）。量測總吞吐量並與單環境比較。

表 30-13 Sim-to-Real 驗證記錄表（前兩列為範例）

階段 / 設定	性能指標	相對下降	推測原因	改善措施
理想模擬	成功率 94%	—	—	—
+ 感測雜訊	成功率 88%	6.4%	位置估計誤差	加入雜訊訓練
（自行填寫其餘階段）				

國中小也看得懂：為什麼電動玩得好不代表真的會開車

你有沒有玩過賽車電動遊戲？在遊戲裡你可能是高手，每次都第一名。但如果真的讓你去開卡丁車，你一定會撞牆——為什麼？

因為**遊戲不是真的**。遊戲裡沒有風、輪胎不會磨損、地上不會有小石頭、你也感覺不到車子在震動。遊戲把很多東西「簡化」了。

機器人的工程師也用同樣的方法：他們先在電腦裡蓋一個「假的世界」，讓機器人在裡面練習。這樣機器人摔一萬次也不會壞、而且電腦可以跑得比真實世界快一百倍！

但問題來了——在假世界裡練得很好的機器人，到了真實世界常常就不行了。

有一個學姊就遇到這種事：她的機器手臂在電腦裡抓東西成功率 98%，但真的去抓，一百次只成功二十次！

為什麼呢？因為電腦裡的瓶子**永遠一樣滑**，但真實的瓶子可能沾了灰、桌子可能有一點油、夾子用久了會變鬆.....真實世界有**太多小細節**，永遠算不完。

那怎麼辦？聰明的科學家想出一個很妙的辦法：**故意把假世界弄亂！**

每一次練習，就隨機改變一次設定——這次瓶子很滑、下次很粗糙；這次很重、下次很輕；這次燈很亮、下次很暗。**逼機器人在幾百萬種不同的世界裡都要能成功。**

這樣一來，當它到了真實世界，就會想：「喔，這個世界我雖然沒見過，但也沒什麼特別的嘛！」然後就成功了。

這個道理其實對你也適用：如果你只在一種情況下練習，換個場地就不會了；但如果你在各種情況下都練過——大場地、小場地、有風、沒風——那你就真的學會了。

叫獸提醒與常見錯誤

- 誤區一：以為「模擬夠真就能上實機」。真實參數本身就在變動，過度依賴特定參數的策略必定脆弱。
- 誤區二：面對 Sim-to-Real 失敗時只想提升模擬保真度。應優先檢視策略的強健性——這是報酬率更高的方向。
- 誤區三：盲目地隨機化而未先做系統辨識。若隨機化的範圍涵蓋不到真實值，就完全沒有意義。
- 誤區四：隨機化範圍設得太寬。任務會變得過難而學不會，或使策略過度保守（例題的過度用力）。
- 誤區五：只隨機化動力學而忽略感測缺陷。多數 Sim-to-Real 失敗的原因在感測而非物理。
- 誤區六：用高精度網格作為碰撞模型。這可能使模擬慢一到兩個數量級，應改用簡化的幾何。
- 誤區七：不分用途地追求最高保真度。四種用途對保真度的需求截然不同，應依需求分別決定。
- 誤區八：忘記設定 `use_sim_time`。模擬時間與系統時鐘不同步，會導致 TF 查詢失敗與行為異常。
- 誤區九：跳過硬體在環測試。時序、通訊與 CPU 負載的問題在純軟體模擬中完全不會出現。
- 誤區十：直接從模擬跳到實機全速測試。應遵循漸進式驗證，並在實機階段降速、限制範圍、全程錄製。
- 誤區十一：平行模擬時所有環境用相同參數。這只是重複同樣的經驗，應搭配領域隨機化才有價值。
- 誤區十二：把數位孿生當成一般模擬的同義詞。前者的關鍵在於與特定實體的持續同步。

課堂討論

1. 30.4.3 節主張「模擬不必更真，策略必須更穩」。請討論這個原則的適用邊界——在什麼情況下，提升模擬保真度反而是正確的選擇？
2. 領域隨機化把「未知參數」變成「觀測不到的干擾」。請討論這與第 16 章「回授對抗未知干擾」、第 24 章「用機率描述不確定性」的異同。
3. 例題 30-1 顯示 GPU 平行模擬把訓練時間由 92.6 天縮短到 16.7 分鐘。請討論這個「量變引起質變」的現象——它如何改變了研究的方式？
4. 30.9.1 節的漸進式驗證強調「隔離變因」。請討論這個科學方法的原則，在本書其他章節的除錯情境中如何應用（提示：第 29 章的 QoS 診斷、第 24 章的創新序列檢驗）。
5. 本章的哲學「與其消除不確定性，不如設計對它不敏感的系統」在全書多次出現。請找出至少四處，並歸納它們的共同結構與各自的實現方式。

章末摘要

- 模擬的三個價值：解除成本、風險與時間的限制；另有「全知的可觀測性」這個常被低估的優勢。
- 模擬有四種用途——演算法除錯、控制器驗證、學習訓練、視覺訓練——各自對保真度的需求截然不同。
- 物理引擎的四個模組中，碰撞偵測通常是最大瓶頸；應使用簡化的碰撞幾何而非高精度網格。
- 即時因子（RTF）是模擬速度的指標；不同用途的需求由 HIL 的必須為 1 到 RL 的越高越好。
- 保真度有三個獨立的維度：動力學、感測、視覺；應依需求分別決定，不必全部追求最高。
- 最容易被忽略的保真度是感測器缺陷——多數 Sim-to-Real 失敗的原因在感測而非物理。
- Sim-to-Real Gap 的五個來源：接觸與摩擦（最嚴重）、致動器動態、感測器缺陷、未建模的物理、環境差異。
- 判斷落差是否重要的準則是「它會不會改變決策」；應先做敏感度分析再投入資源。
- 核心觀念：模擬不必更真，策略必須更穩——追求完美模擬是報酬遞減的路徑。
- 領域隨機化的思想是「不模擬真實世界，而模擬所有可能的世界」，使真實世界成為其中的

一個樣本。

- 隨機化的三類對象：動力學、感測、視覺與場景；優先順序建議為感測雜訊與延遲 > 摩擦與質量 > 視覺。
- 隨機化範圍的取舍：太窄涵蓋不到、太寬過度保守；應以系統辨識的結果為中心，並採漸進式擴大。
- 領域隨機化有效的兩個解釋：把未知參數變成干擾（迫使依賴回授）、以及隱含的線上系統辨識。
- 系統辨識與領域隨機化互補：先辨識找出中心值，再隨機化設定範圍。
- 最容易量測卻最常被忽略的三項辨識：致動器延遲、感測器雜訊、感測器延遲。
- 模擬器的選擇應依用途：ROS 整合用 Gazebo、大規模 RL 用 Isaac Sim 或 MuJoCo、教學用 Webots 或 PyBullet。
- GPU 平行模擬是一次典範轉移——單張 GPU 可跑數千環境且資料不需搬移，這是近年 RL 突破的物質基礎。
- 平行模擬應搭配領域隨機化與非同步重置，否則只是重複同樣的經驗。
- 漸進式驗證分六階段，每階段隔離不同的變因；硬體在環能捕捉純軟體模擬測不到的時序問題。
- 數位學生與一般模擬的關鍵差異在於「與特定實體的持續同步」，用途為監控、預測與最佳化。

觀念題與計算題

觀念題

1. 說明模擬解除的三個限制，並解釋「可觀測性」為何是被低估的優勢。
2. 列舉模擬的四種用途，並說明各自對保真度的需求為何不同。
3. 說明物理引擎的四個模組，並解釋為何碰撞偵測通常是最大瓶頸。
4. 定義即時因子，並比較五種用途對它的需求。
5. 說明保真度的三個維度，並解釋為何應分別決定。
6. 為何說「感測器缺陷是最容易被忽略的保真度」？請舉三個具體的例子。

7. 列舉 Sim-to-Real Gap 的五個來源，並依嚴重程度排序。
8. 說明判斷「哪些落差重要」的準則，並舉例說明。
9. 解釋「模擬不必更真，策略必須更穩」的意義，並說明其理由。
10. 說明領域隨機化的核心思想，以及它與傳統思路的根本差異。
11. 列舉領域隨機化的三類對象，並說明實務上的優先順序。
12. 說明隨機化範圍設定的取捨，以及三個實用的原則。
13. 說明領域隨機化為何有效的兩個解釋。
14. 說明系統辨識與領域隨機化的互補關係，以及為何順序不可顛倒。
15. 列舉三項最容易量測卻常被忽略的辨識項目。
16. 比較至少四種模擬器的特性，並說明選擇的判準。
17. 說明 GPU 平行模擬相對於 CPU 平行的優勢，以及它為何是一次典範轉移。
18. 說明平行模擬的三個實作要點。
19. 列舉漸進式驗證的六個階段，並說明硬體在環能捕捉什麼問題。
20. 區分數位學生與一般模擬，並列舉數位學生的三個應用。

計算題

1. 某任務每次嘗試需 6 秒，強化學習需要 150 萬次嘗試。(a) 真機訓練需多久？(b) $RTF = 15$ 的單環境模擬需多久？(c) 若用 1024 個環境平行且總 RTF 為 5000，需多久？(d) 若領域隨機化使所需次數增為 2.5 倍，重新計算 (c)。
2. 某團隊記錄的成功率為：理想模擬 92%、加入感測雜訊 87%、加入動力學隨機化 79%、實機 68%。(a) 計算各階段的下降百分比。(b) 計算總落差。(c) 指出最大的單一下降出現在何處。(d) 提出兩個改善的方向。
3. 某模擬的每步計算時間為：碰撞 60 ms、約束求解 25 ms、動力學 10 ms、渲染 40 ms、感測 15 ms，步長 4 ms。(a) 計算 RTF 。(b) 關閉渲染後的 RTF 。(c) 若碰撞簡化使其降為 1/10，重新計算。(d) 若要達到總 $RTF \geq 100$ ，計算所需的平行環境數。
4. 某抓取任務的物體質量在 0.2~1.0 kg 之間，摩擦係數在 0.35~0.85 之間。策略以「假設最差情況」的方式設計。(a) 若假設 $\mu = 0.85$ (最樂觀)，求質量 1.0 kg 時所需的夾持力，並判斷在 $\mu = 0.35$ 時是否會失敗。(b) 若假設 $\mu = 0.35$ (最保守)，求同樣情況下的夾持

力。(c) 計算兩者的比值。(d) 若夾持力超過 40 N 會壓壞物體，判斷保守策略是否可行。

動手做與專題延伸

本章實作：模擬與驗證環境

建立一套完整的模擬與驗證流程，這將是第 31 章學習訓練的基礎。

- **階段一：**建立基本的模擬環境，載入機器人模型並確認能運行；量測 RTF 作為基準。
- **階段二：**優化模擬速度——簡化碰撞幾何、調整時間步長、關閉不需要的模組。記錄每項優化的效果。
- **階段三：**實作領域隨機化器，支援動力學、感測與場景三類參數，並具備 curriculum 功能。
- **階段四：**實作漸進式驗證的自動化流程——一鍵跑完各階段並產生性能比較表。
- **階段五：**實作平行模擬，量測吞吐量並與單環境比較。

專題延伸

- **敏感度分析地圖：**對一個具體的任務（如抓取或導航），系統性地掃描各個物理參數（摩擦、質量、延遲、雜訊），記錄性能的變化。繪出「參數 vs 性能」的曲線族，並用斜率量化敏感度。這張圖能直接告訴你：該優先隨機化哪些參數、以及該優先辨識哪些參數。這是本章方法論最實用的產出。
- **隨機化範圍的最佳化：**以程式範例的抓取任務為基礎，系統性地改變隨機化的範圍寬度（從極窄到極寬），記錄「模擬成功率」與「真實環境成功率」兩條曲線。你會看到一個明確的取舍——模擬成功率隨範圍變寬而下降，但真實成功率先上升後下降（過寬時策略變得過度保守）。找出那個最佳點，並討論它與真實參數分布的關係。
- **進階：完整的 Sim-to-Real 專案。**選一個具體的任務（建議：用循跡車走一條有轉彎的路線，或用手臂抓取三種不同的物體）。（一）在模擬中開發並訓練；（二）做系統辨識並量化真實參數；（三）加入領域隨機化重新訓練；（四）依 30.9 節的六階段逐步驗證，每階段記錄性能；（五）繪出完整的落差曲線並歸因。這個專題會讓你親身體會本章的每一個概念——尤其是「哪些改善有效、哪些沒效」這個最寶貴的工程經驗。

■ 參考文獻與延伸閱讀

- Tobin, J. et al., "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," IROS 2017 (領域隨機化的奠基論文) 。
- Peng, X. B. et al., "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization," ICRA 2018 (動力學隨機化的代表性工作) 。
- Makoviychuk, V. et al., "Isaac Gym: High Performance GPU-Based Physics Simulation For Robot Learning," NeurIPS 2021 (GPU 平行模擬的突破) 。
- Todorov, E., Erez, T. and Tassa, Y., "MuJoCo: A Physics Engine for Model-Based Control," IROS 2012 (MuJoCo 的設計理念) 。
- Koenig, N. and Howard, A., "Design and Use Paradigms for Gazebo," IROS 2004 (Gazebo 的原始論文) 。
- Zhao, W., Queralta, J. P. and Westerlund, T., "Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: A Survey," SSCI 2020 (Sim-to-Real 的完整綜述) 。
- Muratore, F. et al., "Robot Learning From Randomized Simulations: A Review," Frontiers in Robotics and AI, 2022 (隨機化模擬的系統性回顧) 。
- Chebotar, Y. et al., "Closing the Sim-to-Real Loop: Adapting Simulation Randomization with Real World Experience," ICRA 2019 (用實機資料自動調整隨機化範圍) 。
- Kaspar, M. et al., "Sim2Real Transfer for Reinforcement Learning without Dynamics Randomization," IROS 2020 (以系統辨識為主的替代路線) 。
- Gazebo、Isaac Sim、MuJoCo、PyBullet 官方文件：各模擬器的使用與最佳實務。

第 31 章

機器學習與強化學習

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為兩個任務的對照：上格是六軸手臂精確地定位到一個標記點，旁註「逆運動學：兩頁公式，微米級精度」；下格是一件皺成一團的 T 恤與一雙手，旁註「摺衣服：至今無機器人能及五歲小孩」。中央以一條垂直的分界線區隔「模型」與「學習」兩條路徑，左側標註「寫得出方程式 → 建模、推導、保證」，右側標註「寫不出方程式 → 從經驗中找規律」。右半為 Bellman 方程的雙重身分：同一條方程式，左邊標「最佳控制：模型已知，用算的」、右邊標「強化學習：模型未知，用試的」。下緣文字：模型提供保證，學習提供泛化。

叫獸開講 「算逆運動學到微米級，為什麼摺不好一件 T 恤？」

叫獸在白板上寫下兩個任務，請全班判斷哪一個比較難。

任務一：計算一支六軸手臂的逆運動學，誤差不超過 0.01 公釐。

任務二：把一件皺成一團的 T 恤攤平摺好。

學生們幾乎都選了任務一——畢竟那要解三角函數、處理八組解、還要避開奇異點。

「錯了。」叫獸說，「任務一，第 10 章我們用兩頁的公式就解決了，而且**精度可以到微米**。任務二，目前全世界沒有任何一台機器人能做得像五歲小孩那樣好。」

教室安靜了。小好問：「為什麼？摺衣服看起來很簡單啊。」

「因為衣服沒有方程式。」叫獸拿起一件 T 恤揉成一團，「你能寫出它的動力學嗎？布料有無限多的自由度、會皺、會摩擦、會黏在一起。第 13 章的 $M(q)\ddot{q} + C\dot{q} + G = \tau$ 在這裡完全派不上用場。」

他把衣服攤在桌上：「前面三十章，我們的方法都是「先建模型，再解方程」。這套方法極為強大——但它有一個前提：問題必須寫得出方程式。」

小翰舉手：「那寫不出來的怎麼辦？」

「那就**不要寫**。」叫獸說，「換一條路——讓機器人自己從經驗中找出規律。這就是機器學習。」

他在白板上畫了一條分界線，左邊寫「模型」、右邊寫「學習」：「這兩條路不是互相取代，而是分工。左邊處理你**理解得夠清楚**的部分——運動學、動力學、控制律，這些有嚴格的保證。右邊處理你**寫不出來但有很多例子**的部分——什麼樣的地方適合抓、皺成什麼樣的布該怎麼拉。」

最後他補了一句，指向第 18 章的板書痕跡：「而且今天你會發現一件有趣的事。**強化學習**跟我們在第 18 章學的最佳控制，用的是同一條方程式——Bellman 方程。差別只在於：第 18 章我們知道世界的規則所以能用算的，強化學習則是不知道規則，只好用試的。」

本章為什麼重要

前面三十章建立的是一套**模型導向**的方法論：建立數學模型、推導控制律、證明穩定性。這

套方法在有明確物理規律的問題上極為成功——第 17 章的計算力矩控制能達到微米級精度、第 18 章的 LQR 有最佳性保證、第 19 章的 Lyapunov 方法能嚴格證明不會失控。

但這套方法有一個前提：**問題必須能被建模**。而真實世界中有大量的問題無法滿足這個前提：

- **難以建模的物體**：布料、繩索、液體、顆粒、生物組織。
- **難以建模的接觸**：第 14.7 節與第 21.2.3 節反覆提及的摩擦與接觸不確定性。
- **難以描述的判斷**：「這個物體看起來該從哪裡抓」、「這個地形該怎麼踩」。
- **組合爆炸的決策**：雜亂桌面上該先移開哪一個物體。

機器學習提供了另一條路：不建模，而是從資料中學習規律。近年機器人領域的重大突破——第 21.9 節的抓取姿勢偵測、第 28.9 節的四足運動控制、以及第 32 章的大型模型——全部來自這條路線。

但本章要傳達的核心訊息不是「學習取代模型」，而是**兩者的分工**。正如第 21.9.5 節的混合式抓取、第 25.10.3 節的 SLAM 前後端、第 28.9.4 節的模型與學習比較所示：**模型提供保證，學習提供泛化**。理解何時該用哪一個，是本章最重要的收穫。

學習目標

- 能說明模型導向與資料導向方法的適用邊界。
- 能區分監督式、非監督式與強化學習三種典範。
- 能說明監督式學習在機器人感知中的應用與資料需求。
- 能說明模仿學習的原理、優勢與其複合誤差問題。
- 能說明 DAgger 如何解決分布偏移。
- 能定義馬可夫決策過程的五個要素。
- 能說明強化學習與最佳控制的關係並指出關鍵差異。
- 能區分價值型與策略型方法及其適用場合。
- 能說明獎勵函數設計的困難與獎勵駭客現象。
- 能說明策略梯度與 PPO 的基本原理。
- 能說明從模擬到實機的學習流程與其成功要件。

- 能為特定的機器人問題選擇適當的學習方法。

先備知識

第 18 章的 Bellman 方程與價值函數（本章直接建立在其上，務必先複習）、第 30 章的模擬與領域隨機化、第 22.8 節的深度學習概念、第 21.9 節的學習式抓取、第 28.9 節的腿式運動學習。

數學方面需要：機率的基本概念、梯度的直覺。**不需要**深度學習的完整背景——本章聚焦於「機器人如何學習」，而非神經網路的細節。若需要補充，可先閱讀附錄 E。

核心名詞中英對照

表 31-1 本章核心名詞

中文	英文	說明
監督式學習	Supervised Learning	由帶標籤的資料學習輸入到輸出的映射
非監督式學習	Unsupervised Learning	由無標籤資料中發現結構
強化學習	Reinforcement Learning, RL	透過與環境互動並依獎勵學習
模仿學習	Imitation Learning	由專家示範中學習策略
行為複製	Behavior Cloning	以監督式學習直接模仿示範動作
分布偏移	Distribution Shift	執行時的狀態分布偏離訓練資料
複合誤差	Compounding Error	小誤差逐步累積導致大偏離
馬可夫決策過程	Markov Decision Process, MDP	強化學習的標準數學框架
策略	Policy	由狀態決定動作的規則
獎勵	Reward	衡量行為好壞的純量訊號
回報	Return	未來獎勵的折扣累加
折扣因子	Discount Factor, γ	衡量未來獎勵重要性的係數
價值函數	Value Function	由某狀態出發的期望回報
Q 函數	Q-function / Action-Value	在某狀態採取某動作的期望回報
探索與利用	Exploration vs Exploitation	嘗試新動作與採用已知好動作的取捨
策略梯度	Policy Gradient	直接對策略參數求梯度的方法
獎勵塑形	Reward Shaping	設計輔助獎勵以加速學習
獎勵駭客	Reward Hacking	找到獎勵的漏洞而非完成任務

中文	英文	說明
樣本效率	Sample Efficiency	達到目標性能所需的互動次數

31.1 模型與學習的分界

31.1.1 模型導向方法的成功與邊界

前面三十章的方法論可以概括為三步：**建模** → **推導** → **驗證**。它的成功建立在物理定律的普適性上——牛頓定律不會因為換一台機器人而改變。

這帶來三個無可取代的優勢：

- **保證**：第 19 章能嚴格證明「這個控制器不會失控」。學習方法給不出這種保證。
- **可解釋**：當出問題時，可以逐項檢查——是模型參數錯了？增益不對？還是感測失準？
- **資料效率**：不需要任何訓練資料。一台新機器人裝好、量好參數，控制器立刻能用。

但它的邊界也很明確：**問題必須寫得出方程式**。

表 31-2 模型導向方法的適用邊界

問題類型	能否建模	範例	本書章節
剛體運動學	完全可以	手臂的正逆運動學	第 9、10 章
剛體動力學	可以（有誤差）	力矩計算、軌跡規劃	第 13~15 章
線性控制	可以且有保證	PID、LQR	第 16、18 章
接觸與摩擦	部分可以	抓取、行走	第 20、21、28 章
變形物體	極困難	布料、繩索、食物	—
視覺理解	幾乎不可能	「這是什麼物體」	第 22 章
常識判斷	不可能	「杯子該正放」	第 32 章

31.1.2 學習方法的價值與代價

學習方法的思路完全相反：不問「規律是什麼」，只問「資料告訴我們什麼」。

價值：能處理無法建模的問題、能從經驗中改善、能泛化到新情況。

代價：需要大量資料、沒有保證、難以解釋、且在訓練分布之外的行為不可預測。

◎ 一個關鍵的判準

面對一個機器人問題，該用模型還是學習？一個實用的判準是問三個問題：一、你能寫出方程式嗎？能 → 用模型（它更準、更可靠、且不需要資料）。二、你有大量的資料或能低成本地產生嗎？有 → 學習可行（第 30 章的模擬正是為此）。三、失敗的代價有多大？極高（如載人系統）→ 至少關鍵部分必須用有保證的方法。多數實務系統的答案是「兩者都要」——用模型處理有物理規律的部分、用學習處理需要判斷的部分。

31.1.3 三種混合的模式

實務上模型與學習的結合有三種常見模式，本書前面各章都出現過：

表 31-3 模型與學習的三種混合模式

模式	作法	本書中的例子
學習做前端、模型做後端	學習負責辨識與判斷，模型負責計算與驗證	第 25.10.3 節的 SLAM、第 21.9.5 節的抓取
模型做骨架、學習做修正	模型提供標稱行為，學習補償未建模的部分	第 17 章的前饋加回授（概念相同）
模型產生資料、學習做泛化	用模型或模擬產生大量標註資料訓練網路	第 21.9.3 節的解析式標註

31.2 三種學習典範

31.2.1 由「資料長什麼樣」區分

機器學習傳統上分為三種典範，其差別在於訓練資料的形式：

表 31-4 三種學習典範的比較

典範	資料形式	學什麼	機器人上的典型應用
監督式學習	(輸入, 正確答案) 配對	輸入到輸出的映射	物體辨識、抓取點預測、深度估計
非監督式學習	只有輸入，無標籤	資料的內在結構	特徵學習、異常偵測、分群
強化學習	(狀態, 動作, 獎勵) 序列	能最大化長期回報的策略	運動控制、操作技能、決策

三者的關鍵差異在於「回饋的形式」：

- 監督式學習得到的是**正確答案**——「這張圖是杯子」。回饋最豐富、學習最快。
- 強化學習得到的是**評分**——「這次抓取得 1 分」。它不告訴你正確答案是什麼，只說結果好

不好。這使學習困難得多。

- 非監督式學習完全沒有回饋——只能從資料本身的結構中找規律。

31.2.2 機器人領域的特殊性

機器人上的機器學習，與影像辨識或自然語言處理有三個重要的差異：

差異一：資料昂貴。網路上有數十億張標註好的影像，但沒有任何現成的「機器人抓取經驗」資料集能與之相比。每一筆資料都需要真實的物理互動——這正是第 30 章模擬如此重要的原因。

差異二：動作有後果。影像辨識錯了只是一個錯誤的標籤；機器人動作錯了可能打翻杯子、撞壞設備、甚至傷人。這使「試錯」的代價極高。

差異三：序列決策。辨識是一次性的判斷；機器人的動作則會改變自己接下來看到的世界。這個「動作影響後續狀態」的特性，正是強化學習與監督式學習的根本差異。

◎ 為何機器人學習比影像辨識困難

2012 年之後，深度學習在影像辨識上取得了驚人的進展。許多人期待同樣的突破會迅速發生在機器人上——但實際上慢得多。三個原因：**一、資料量差了好幾個數量級**（ImageNet 有 1400 萬張標註影像，而最大的機器人資料集也只有數百萬筆互動，且多來自模擬）；**二、無法用網路上的資料**（機器人需要的是「動作與後果」的配對，而不是圖片與標籤）；**三、錯誤的代價不對稱**（辨識錯 1% 無妨，但機器人 1% 的機率打翻東西就無法部署）。理解這三點，就能理解為何機器人領域的學習方法必須格外重視樣本效率與安全性。

31.3 監督式學習在機器人上的應用

31.3.1 最成熟的應用：感知

監督式學習在機器人上最成功的應用是**感知**——因為感知本質上就是「由輸入預測輸出」的映射問題。

本書前面已經多次提及：

- **物體偵測與分類**（第 22.8 節）：輸入影像，輸出物體的類別與位置。
- **位姿估計**（第 22.8 節）：輸入影像，輸出物體的六自由度位姿。
- **抓取姿勢偵測**（第 21.9 節）：輸入深度圖，輸出可行的抓取位姿與成功率預測。
- **語意分割**（第 25.10.1 節）：輸入影像，輸出每個像素的類別——用於剔除動態物體。

- 深度估計（第 22.4 節）：由單張影像預測深度。

31.3.2 資料從哪裡來

監督式學習的瓶頸永遠是**標註資料**。機器人領域有四個來源：

表 31-5 機器人學習資料的四個來源

來源	成本	真實性	規模	典型用途
人工標註	高	高	小（千~萬）	驗證集、微調
模擬產生	極低	中（有落差）	極大（百萬+）	預訓練、大規模訓練
解析式標註	低	中	大	抓取品質評分（第 21.9.3 節）
自我監督	中	高	中	機器人自行嘗試並記錄結果

實務上的標準作法是**混合**：用模擬產生大量資料做預訓練（學會通用的特徵）、再用少量的真實資料微調（校正 Sim-to-Real 的落差）。這正是第 30 章模擬技術的直接應用。

31.3.3 一個常被忽略的陷阱

監督式學習有一個在機器人上特別危險的問題：它假設訓練資料與實際使用時的資料來自同一個分布。

但機器人的部署環境往往與訓練時不同——不同的光線、不同的物體、不同的背景。當這個假設被打破時，模型的預測**可能錯得很離譜，而且它自己不知道**（會給出很高的信心分數）。

因應之道有三：

- **訓練時涵蓋多樣性**：這正是第 30.5 節領域隨機化的視覺版本。
- **輸出不確定性**：讓模型除了預測之外，也回報自己的信心。低信心時交由其他機制處理（如改用保守的策略、或請求人類介入）。
- **執行時的驗證**：用幾何或物理的方法驗證學習的輸出——這正是第 21.9.5 節「學習產生候選、解析驗證品質」的混合式架構。

31.4 模仿學習

31.4.1 從示範中學習

模仿學習（Imitation Learning）的思路極為直觀：與其讓機器人自己摸索，不如讓人示範

幾次，機器人照著學。

這在機器人上特別有吸引力，因為：

- 不需要設計獎勵函數（31.7 節會說明這有多困難）。
- 樣本效率高：幾十次示範可能就夠，而強化學習需要數百萬次嘗試。
- 安全：不需要在真機上做危險的隨機探索。
- 直觀：非專家也能「教」機器人——這對第 20.5.3 節的手動導引示教是自然的延伸。

31.4.2 行為複製及其致命缺陷

最簡單的模仿學習稱為**行為複製**（Behavior Cloning）：把示範資料當成監督式學習的訓練集。

訓練資料： $\{(s_1, a_1), (s_2, a_2), \dots\}$

學習目標： $\pi(s) \approx a_{\text{expert}}(s)$

這就是一個標準的監督式學習問題。實作極為簡單。

但它有一個致命的缺陷，稱為**複合誤差**（compounding error）：

◎ 複合誤差：為何模仿學習會逐漸偏離

假設機器人學會了 99% 準確地模仿專家。第一步它有 1% 的誤差，來到一個專家從未到過的狀態。在這個新狀態下，它的預測更不準（因為訓練資料裡沒有），於是誤差變成 3%。下一步來到更陌生的狀態，誤差變成 10%.....**誤差會像滾雪球一樣累積**。這正是為何行為複製訓練出的自駕車，在直線上開得很好，卻會在第一個彎道逐漸偏離車道並最終衝出去——**因為專家從來沒有偏離過，所以資料中沒有「偏離後該怎麼修正」的範例**。

數學上可以證明：若每一步的誤差率為 ϵ ，在長度 T 的軌跡上，總誤差會以 $O(\epsilon T^2)$ 成長——與時間的平方成正比，而非線性。

31.4.3 DAgger：讓專家糾正錯誤

DAgger（Dataset Aggregation）是解決複合誤差的經典方法。它的核心洞見是：

問題的根源：訓練資料只涵蓋「專家走過的狀態」

解法：讓資料涵蓋「機器人自己會走到的狀態」

關鍵是改變資料的來源。

具體流程為：

1. 用現有的示範資料訓練一個初始策略。
2. 讓機器人自己執行（它會犯錯並偏離）。
3. 在它走過的每一個狀態上，詢問專家「你會怎麼做」。
4. 把這些新的（狀態, 專家動作）加入資料集。
5. 重新訓練，回到步驟 2。

這個流程的精髓在於步驟三：專家不是重新示範一遍，而是在機器人犯錯後的狀態上給出正確答案。這使資料集逐漸涵蓋「偏離後如何修正」——正是行為複製所缺少的。

DAgger 的代價是需要專家持續參與（而非一次性的示範），這在實務上可能很昂貴。因此有多種變體嘗試降低專家的負擔——例如只在模型不確定時才詢問。

31.4.4 近年的發展

模仿學習近年有三個重要的進展，都與第 32 章的大型模型密切相關：

進展一：擴散策略（Diffusion Policy）。傳統的行為複製只能學到「平均的動作」——若專家有多種合理的做法（如繞過障礙可左可右），平均之後可能兩邊都不是。擴散模型能表示多峰的動作分布，解決了這個問題。

進展二：大規模的示範資料集。多個實驗室合作收集跨機器人、跨任務的示範資料（如 Open X-Embodiment），使「預訓練一個通用的操作模型」成為可能——這正是第 32 章的主題。

進展三：遙操作介面的改善。用 VR 手把或第 20.5.3 節的手動導引收集示範，大幅降低了資料收集的門檻。

◎ 模仿學習與強化學習的分工

兩者不是競爭關係，而是常常搭配使用。典型的流程是：先用模仿學習取得一個「還可以」的初始策略（避免強化學習初期完全隨機的危險探索），再用強化學習微調到最佳。這個組合兼顧了樣本效率與最終性能——模仿學習提供了好的起點，強化學習則能超越示範者的水準（因為它不受限於專家的表現）。第 28.9 節的腿式運動、以及 AlphaGo 的訓練流程，都採用了這個架構。

31.5 強化學習的框架

31.5.1 馬可夫決策過程

強化學習的標準數學框架是**馬可夫決策過程** (Markov Decision Process, MDP)，由五個要素構成：

表 31-6 MDP 的五個要素

要素	符號	意義	機器人上的對應
狀態空間	S	所有可能的狀態	關節角度、速度、感測讀值
動作空間	A	所有可能的動作	關節力矩、速度指令
轉移機率	$P(s' s,a)$	在 s 做 a 後到達 s' 的機率	機器人動力學 (第 13 章)
獎勵函數	$R(s,a)$	行為好壞的評分	由設計者定義 (31.7 節)
折扣因子	$\gamma \in [0,1)$	未來獎勵的重要性	典型 0.95~0.99

第三列值得特別注意：轉移機率 $P(s'|s,a)$ 就是**系統的動力學**。第 13、14 章我們花了整整兩章推導它；而在強化學習中，它被假設為**未知**——這正是兩者最根本的差異。

31.5.2 目標：最大化回報

強化學習的目標是找出一個**策略** $\pi(a|s)$ ，使**期望回報**最大：

$$G = r_0 + \gamma \cdot r_1 + \gamma^2 \cdot r_2 + \gamma^3 \cdot r_3 + \dots$$

回報 (return) 是未來所有獎勵的折扣累加。

折扣因子 γ 有兩個作用：**數學上**使無限長的序列收斂；**直覺上**代表「未來的獎勵不如現在的**值錢**」。

$\gamma = 0.99 \rightarrow$ 有效視野約 100 步

$\gamma = 0.95 \rightarrow$ 有效視野約 20 步

$\gamma = 0.9 \rightarrow$ 有效視野約 10 步

有效視野 $\approx 1/(1-\gamma)$ 。這是一個實用的判準—— γ 應依任務所需的規劃長度來選。

γ 的選擇有一個實務上的取舍：太小則策略「短視」（只顧眼前）、太大則學習困難（因為要考慮的未來太長，訊號被稀釋）。

31.5.3 價值函數與 Q 函數

與第 18.3.2 節完全相同，我們定義**價值函數**：

$$V^{\pi}(s) = E[G \mid s_0 = s, \text{依策略 } \pi \text{ 行動}]$$

由狀態 s 出發、依策略 π 行動，能得到的期望回報。

強化學習還常用另一個相關的函數——**Q 函數**（動作價值函數）：

$$Q^\pi(s,a) = E[G \mid s_0 = s, a_0 = a, \text{之後依 } \pi \text{ 行動}]$$

在狀態 s 先做動作 a 、之後依 π 行動的期望回報。

Q 函數比 V 函數實用的地方在於：它直接告訴你該做哪個動作：

$$\text{最佳策略：}\pi^*(s) = \operatorname{argmax}_a Q^*(s,a)$$

只要有了 Q^* ，最佳策略就是「選 Q 值最大的動作」——不需要知道環境的模型。這正是 Q 函數在無模型情境下的價值。

31.6 伏筆回收：強化學習與最佳控制

31.6.1 同一條方程式

現在可以回收第 18 章的伏筆。強化學習的核心方程式是：

$$V^*(s) = \max_a [R(s,a) + \gamma \cdot \sum_{s'} P(s'|s,a) \cdot V^*(s')]$$

Bellman 最佳方程（強化學習版本）。

而第 18.4.1 節的最佳控制版本是：

$$V_k(x) = \min_u [L(x,u) + V_{k+1}(f(x,u))]$$

Bellman 方程（最佳控制版本）。

兩者是同一條方程式。差別只在三個表面的地方：

表 31-7 最佳控制與強化學習的對照（完整版）

最佳控制（第 18 章）	強化學習（本章）	說明
狀態 x	狀態 s	相同
控制輸入 u	動作 a	相同
代價 L （最小化）	獎勵 r （最大化）	符號相反，本質相同
系統模型 $f(x,u)$	轉移機率 $P(s' s,a)$	確定性 vs 隨機
模型已知	模型未知	最關鍵的差異
價值函數 $V(x)$	價值函數 $V(s)$	相同概念
動態規劃（計算）	時序差分學習（取樣）	求解方式不同
有限或無限時域	折扣因子 γ	處理無限未來的方式

31.6.2 關鍵的差異：模型是否已知

表中的第五列是唯一本質的差異，而它導致了完全不同的求解方式：

最佳控制知道模型，因此可以**計算**：對每一個狀態，直接算出「做這個動作會到哪裡、值多少」。第 18.4.2 節的動態規劃就是這樣由後往前推的。

強化學習不知道模型，因此只能**取樣**：實際去做一個動作，看看發生了什麼、得到多少獎勵，再用這個經驗更新估計。

最佳控制： $V(s) \leftarrow \max_a [R + \gamma \sum P(s'|s,a) \cdot V(s')]$ （用模型算期望值）

強化學習： $V(s) \leftarrow V(s) + \alpha [r + \gamma V(s') - V(s)]$ （用一次經驗更新）

後者稱為**時序差分學習 (TD Learning)**。中括號中的量稱為**TD 誤差**——「實際看到的」與「原本預期的」之差。

◎ TD 誤差：一個優美的想法

時序差分學習的核心是一個極簡的想法：我原本以為這個狀態值 $V(s)$ ，但走了一步之後看到獎勵 r 且來到了值 $V(s')$ 的狀態，所以它其實應該值 $r + \gamma V(s')$ 。兩者的差 $\delta = r + \gamma V(s') - V(s)$ 就是我的估計錯了多少，據此修正即可。這個想法的美妙之處在於：不需要等到回合結束才知道結果，走一步就能學一點。這與人類的學習方式驚人地相似——你不需要下完整盤棋才知道剛才那步是好是壞，走完立刻就有感覺。神經科學研究甚至發現，大腦中多巴胺神經元的活動模式與 TD 誤差高度吻合。

31.6.3 這個關聯的三個啟示

理解兩者的同源，帶來三個實用的啟示：

啟示一：有模型就別用強化學習。若你能寫出動力學方程（如第 13 章的機械手臂），用第 18 章的 LQR 或 MPC 會快得多、準得多、且有保證。強化學習是「沒有模型時的備案」，不是更先進的替代品。

啟示二：可以用模型加速學習。若有一個近似的模型，可以用它產生虛擬經驗來訓練，減少真實互動的次數。這稱為**模型基礎的強化學習 (Model-based RL)**，樣本效率通常高出一到兩個數量級。

啟示三：第 18 章的直覺仍然適用。維數詛咒、價值函數的意義、bang-bang 特性——這些在強化學習中同樣存在。第 18.4.3 節提到的「用函數近似逼近價值函數」，正是深度強化學習

的核心作法。

31.7 獎勵函數的設計

31.7.1 最困難也最被低估的一環

強化學習的教科書往往聚焦於演算法，但實務上真正困難的是獎勵函數的設計。原因是：

演算法有標準答案（PPO、SAC 等都是現成的）

獎勵函數沒有一它必須為每個任務重新設計

而獎勵函數決定了機器人會學到什麼。

一個常見的經驗是：同一個演算法，換一個獎勵函數，結果可能天差地別。許多「演算法無效」的案例，實際上是獎勵設計的問題。

31.7.2 稀疏獎勵與稠密獎勵

獎勵函數有兩種基本形式，各有優劣：

表 31-8 稀疏獎勵與稠密獎勵的比較

面向	稀疏獎勵	稠密獎勵
定義	只在成功時給獎勵	每一步都給部分獎勵
範例	抓到物體 +1，否則 0	距離目標越近獎勵越高
設計難度	低（不易出錯）	高（容易誤導）
學習難度	極高（探索困難）	低（有明確方向）
風險	可能永遠學不到	可能學到錯的東西
適用	簡單任務、有好的探索機制	複雜任務、需要引導

稀疏獎勵的困難在於探索：若隨機動作幾乎不可能成功，機器人就永遠得不到任何正回饋，也就無從學起。這稱為探索問題。

稠密獎勵的困難在於設計：你必須把「什麼叫做進步」量化——而這往往比想像中困難。下一節的獎勵駭客正是這個困難的體現。

31.7.3 獎勵駭客

獎勵駭客（Reward Hacking）是強化學習中最惡名昭彰的問題：機器人找到了獲得高獎勵的方法，但那不是你想要的行為。

◎ 三個經典的獎勵駭客案例

案例一：抓取任務。獎勵設為「夾爪與物體的距離越近越好」。結果機器人學會了把夾爪貼在物體旁邊來回摩擦——距離永遠很近，獎勵拿滿，但從未抓起任何東西。**案例二：行走任務。**獎勵設為「質心的前進距離」。結果機器人學會了往前撲倒——一次撲倒能前進很多，摔倒後重置再撲一次。**案例三：清潔任務。**獎勵設為「偵測到的髒污減少」。結果機器人學會了把攝影機轉開不看髒的地方。這三個案例的共通點是：演算法完美地達成了你「寫下的」目標，而不是你「想要的」目標。

因應獎勵駭客有三個原則：

- **盡量使用稀疏或半稀疏的獎勵：**越接近「真正的目標」，被鑽漏洞的空間越小。
- **加入懲罰項：**對不希望的行為明確扣分（如「摔倒 -10」、「關節超限 -5」、「能量消耗」）。
- **觀察行為而非只看分數：**訓練時務必實際看機器人在做什麼。分數上升不代表學對了——這是最重要也最常被省略的一步。

31.7.4 實務上的獎勵設計

以第 28 章的四足行走為例，一個典型的獎勵函數包含多項：

```
def compute_reward(state, action, next_state):
    """四足機器人行走的典型獎勵函數"""
    r = 0.0

    # ---- 主要目標：追蹤指令速度 ----
    vel_error = np.linalg.norm(next_state.base_vel[:2] - cmd_vel[:2])
    r += 1.0 * np.exp(-vel_error**2 / 0.25)      # 越接近指令速度越高

    # ---- 姿態獎勵：保持身體水平與高度 ----
    r += 0.5 * np.exp(-abs(next_state.base_height - 0.35)**2 / 0.01)
    r -= 0.5 * abs(next_state.base_roll)        # 懲罰側傾
    r -= 0.5 * abs(next_state.base_pitch)

    # ---- 平滑性：懲罰劇烈的動作（呼應第 15 章的急跳度） ----
    r -= 0.01 * np.sum(action**2)               # 力矩大小
    r -= 0.05 * np.sum((action - prev_action)**2) # 動作變化率

    # ---- 能耗：懲罰無效的功 ----
    power = np.sum(np.abs(action * next_state.joint_vel))
    r -= 0.001 * power
```

```
# ---- 安全約束：明確懲罰不希望的狀態 ----
if next_state.base_height < 0.15: r -= 10.0 # 趴下
if abs(next_state.base_roll) > 1.0: r -= 10.0 # 翻覆
r -= 1.0 * np.sum(next_state.joint_limit_violation)

# ---- 步態品質：鼓勵抬腳（避免拖行） ----
r += 0.1 * np.sum(next_state.foot_clearance > 0.03)

return r
```

注意這個獎勵函數的結構：一個主要目標（速度追蹤）、數個輔助項（姿態、平滑、能耗）、以及明確的懲罰（翻覆、超限）。這是實務上的典型配置。

各項的權重需要大量的調整——這是強化學習中最耗時的工作之一，也是為何第 30 章的快速模擬如此重要（每次調整都要重新訓練）。

31.8 主要的強化學習方法

31.8.1 兩大 family

強化學習的演算法可分為兩大類，其差別在於學什麼：

價值型方法 (Value-based)：學習 Q 函數，動作由 argmax 決定。

- 代表：Q-learning、DQN（深度 Q 網路）。
- 優點：樣本效率較高（可重複使用歷史經驗）、原理直觀。
- 限制：只適用於離散的動作空間——因為 argmax 需要列舉所有動作。

策略型方法 (Policy-based)：直接學習策略 $\pi(a|s)$ ，不經過價值函數。

- 代表：REINFORCE、PPO、TRPO。
- 優點：能處理連續動作（機器人的力矩、速度都是連續的）、可表示隨機策略。
- 限制：樣本效率較低、訓練較不穩定。

機器人幾乎都用策略型或其變體，原因很簡單——關節力矩是連續的，無法列舉。

31.8.2 Actor-Critic：兩者的結合

Actor-Critic 架構結合了兩者的優點，是目前的主流：

Actor (演員) : 策略網路 $\pi(a|s)$, 負責決定動作

Critic (評論家) : 價值網路 $V(s)$ 或 $Q(s,a)$, 負責評估好壞

Actor 做決定、Critic 給回饋——這個分工使學習更穩定。

為何需要 Critic ? 純策略梯度需要等到回合結束才知道結果 (變異極大) ; 有了 Critic , 每一步都能得到「這個動作比預期好還是壞」的即時評估——這正是 31.6.2 節 TD 誤差的應用。

31.8.3 PPO : 機器人上最常用的演算法

PPO (Proximal Policy Optimization) 是目前機器人強化學習的事實標準。第 28.9 節的四足運動、多數的 Isaac Gym 範例, 用的都是它。

它解決的核心問題是: 策略更新的步伐該多大?

更新太小 → 學得太慢

更新太大 → 策略突然變差, 且可能再也回不來

後者是強化學習特有的危險——因為策略變差後, 收集到的資料也跟著變差, 形成惡性循環。

PPO 的解法極為務實: 限制每次更新的幅度, 不要讓新策略與舊策略差太多。

$$L = E[\min(\text{ratio} \cdot \hat{A}, \text{clip}(\text{ratio}, 1-\epsilon, 1+\epsilon) \cdot \hat{A})]$$

PPO 的裁剪目標函數。ratio 是新舊策略的機率比、 $\hat{A}$ 是優勢估計、 ϵ 典型取 0.2。clip 函數限制了更新的幅度。

PPO 之所以成為標準, 不是因為它最先進, 而是因為它「夠好且好調」——超參數不敏感、實作簡單、在各種任務上都能穩定運作。這是工程上極為可貴的特質 (呼應第 26 章 Pure Pursuit 「參數少」的價值) 。

表 31-9 常見強化學習演算法的比較

演算法	類型	動作空間	樣本效率	穩定性	機器人適用性
DQN	價值型	離散	中	中	低 (動作需離散化)
DDPG	Actor-Critic	連續	高	低 (易發散)	中
TD3	Actor-Critic	連續	高	中	中高
SAC	Actor-Critic	連續	高	高	高 (真機訓練常用)
PPO	策略型	連續	低	高	極高 (模擬訓練標準)

選擇的判準: 若在模擬中訓練 (樣本便宜), 用 PPO——穩定且好調。若必須在真機上訓練 (樣本昂貴), 用 SAC——樣本效率高出許多。

31.8.4 樣本效率的問題

強化學習最大的實務障礙是**樣本效率極低**。以 PPO 訓練一個四足行走策略為例，典型需要 1 億到 10 億步的互動。

$$10 \text{ 億步} \times 0.02 \text{ 秒/步} = 2000 \text{ 萬秒} \approx 231 \text{ 年 (真機)}$$

這就是為何強化學習幾乎必須在模擬中進行——第 30 章的內容不是選配，而是必要條件。

三個提升樣本效率的方向：

- **模型基礎的方法**：用學到的模型產生虛擬經驗（31.6.3 節的啟示二）。
- **離線策略方法 (Off-policy)**：如 SAC，能重複使用歷史經驗，效率高出 10~100 倍。
- **從示範出發**：先用模仿學習取得初始策略（31.4.4 節的知識框），大幅減少探索的需求。

31.9 完整的學習流程

31.9.1 七個步驟

整合前面各節，一個完整的機器人強化學習專案包含七個步驟：

1. **問題形式化**：定義狀態空間、動作空間、以及成功的判準。**這一步決定了後續一切——狀態若不含必要的資訊（如未觀測的接觸狀態），策略再好也學不會。**
2. **建立模擬環境（第 30 章）**：包含機器人模型、環境、感測器模擬。優化速度以支援大規模訓練。
3. **設計獎勵函數（31.7 節）**：主要目標加輔助項加懲罰。這一步通常需要多次迭代。
4. **設定領域隨機化（第 30.5 節）**：先做系統辨識取得標稱值，再設定範圍。
5. **訓練**：以 PPO 或 SAC 在平行環境中訓練。**務必同時觀察行為與分數。**
6. **在模擬中驗證**：測試泛化能力——用訓練時沒見過的地形、物體、擾動。
7. **部署到實機（第 30.9 節）**：漸進式驗證，由低速與受限環境開始。

其中**步驟一與步驟三最耗時間**，卻最常被低估。多數初學者急著跑演算法，結果在錯誤的問題形式化或獎勵設計上浪費數週。

31.9.2 觀測空間的設計

步驟一中最關鍵的決定是：**策略能看到什麼**。這比想像中重要得多。

以四足機器人為例，典型的觀測包含：

表 31-10 四足機器人的典型觀測空間

類別	內容	維度	說明
本體感覺	關節角度與速度	24	編碼器直接讀取，最可靠
	軀幹角速度與重力方向	6	由 IMU 取得 (第 24 章)
指令	期望的前進與轉向速度	3	由上層或遙控器給定
歷史	前幾步的動作與觀測	可變	使策略能推斷未觀測的狀態
外部感知	地形高度圖 (可選)	大	第 22 章的深度感知

第四列的「歷史」極為關鍵。單一時刻的觀測無法分辨「地面是硬的還是軟的」、「負載是輕還是重」——但一段歷史可以 (呼應第 30.5.5 節的解釋二)。這正是第 28.9.3 節「教師—學生」架構中，學生能從有限感測推斷隱藏狀態的原因。

31.9.3 除錯的方法

強化學習的除錯特別困難——它可能「沒有錯誤訊息但就是學不起來」。五個實用的診斷步驟：

- **先確認環境正確**：手動下達已知正確的動作，確認機器人的行為與獎勵符合預期。**環境有錯是最常見的問題。**
- **簡化任務**：先讓它學會最簡單的版本 (如「站著不動」而非「行走」)。若連這都學不會，問題在環境或獎勵，不在演算法。
- **檢查獎勵的量級**：各項的數值範圍應該相當。若某一項是 100 而另一項是 0.01，後者形同不存在。
- **觀察行為而非分數**：錄影看機器人在做什麼。獎勵駭客 (31.7.3 節) 只有這樣才發現得了。
- **檢查觀測的正規化**：神經網路對輸入的尺度敏感。關節角度 (± 3) 與速度 (± 20) 的量級差異應該正規化。

31.10 學習方法的限制與展望

31.10.1 四個根本的限制

儘管成果驚人，學習方法仍有四個根本的限制，理解它們才能正確地使用：

限制一：沒有保證。第 19 章能證明「這個控制器一定穩定」；學習方法只能說「在測試的一萬個案例中成功了 99%」。對安全關鍵的系統，這個差異是決定性的。

限制二：分布外行為不可預測。策略在訓練涵蓋的範圍內表現良好，但遇到完全沒見過的情況時，行為可能任意奇怪——而且它不會告訴你「我不確定」。

限制三：難以除錯與修改。若一個學好的策略有某個特定的缺陷（如「遇到玻璃就撞上去」），你無法像改程式一樣直接修正——只能重新設計獎勵並重新訓練，而這可能破壞其他已經學好的行為。

限制四：樣本效率。31.8.4 節算過，即使有模擬，訓練仍需數小時到數天。每一次獎勵函數的調整都要重來。

31.10.2 因應的架構

面對這些限制，實務上的因應是分層與混合：

表 31-11 學習與模型的分層架構

層級	方法	理由
安全監督層	模型式的規則檢查	必須有保證——如力矩極限、工作空間、緊急停止
高階決策	學習或大型模型	需要判斷與泛化（第 32 章）
中層技能	學習（RL 或模仿）	難以建模但有大量資料
低階控制	模型式（PID、WBC）	有明確物理規律且需要保證

最上層的安全監督層是關鍵：無論學習的策略輸出什麼，都必須經過模型式的檢查——超出力矩極限就限制、要撞到人就停止。這使系統即使在學習出錯時也不會造成災難。

這正是第 21.9.5 節「學習產生候選、解析驗證品質」的推廣——學習可以犯錯，但錯誤必須被攔截。

31.10.3 近年的方向

三個值得關注的發展：

方向一：大規模預訓練。與其為每個任務從頭訓練，不如先在大量的資料上預訓練一個通用的模型，再針對特定任務微調。這正是第 32 章的主題，也是目前最受矚目的方向。

方向二：安全強化學習。在學習過程中明確地納入安全約束——例如用第 19 章的 Lyapunov 函數或控制屏障函數 (CBF) 限制探索的範圍。這試圖為學習方法補上「保證」這一塊。

方向三：離線強化學習。從已收集的資料中學習，完全不需要新的互動。這對真機訓練極有價值——可以用歷史的操作紀錄訓練，而不需要冒險探索。

31.11 由學習邁向理解

31.11.1 本章的成果

本章建立了機器人學習的完整圖像。你現在能夠：判斷何時該用模型、何時該用學習、區分三種學習典範、理解模仿學習的複合誤差問題與 DAgger 的解法、掌握 MDP 框架與價值函數、設計獎勵函數並避免獎勵駭客、選擇適當的強化學習演算法、並設計完整的學習流程與安全架構。

我們也回收了第 18 章最重要的伏筆：**強化學習與最佳控制共用同一條 Bellman 方程**，差別只在於「模型是否已知」。這個認識帶來三個實用的啟示——有模型就別用強化學習、可以用模型加速學習、以及第 18 章的所有直覺仍然適用。

最重要的是那個貫穿全章的分工原則：**模型提供保證，學習提供泛化**。這不是誰取代誰，而是各自處理擅長的部分——與第 17 章「模型處理已知、回授處理未知」、第 25 章「學習做前端、幾何做後端」，是同一種工程智慧的不同體現。

31.11.2 接下來：從技能到任務

但本章的方法有一個共同的限制：**它們學的都是「技能」，而非「任務」**。

一個強化學習訓練出的策略，可能極為擅長「走路」或「抓取圓柱體」。但若你對它說「**幫我把桌上的杯子拿到廚房，順便看看冰箱裡還有沒有牛奶**」——它完全無法理解，因為：

- 它不懂**語言**。
- 它不會**分解任務**（要先導航、再辨識、再抓取、再導航、再放置）。
- 它沒有**常識**（杯子要正著拿、牛奶在冰箱裡、開冰箱要拉門把）。
- 它無法**處理未預期的情況**（若杯子是滿的怎麼辦？）。

第 32 章（大型模型與任務規劃）將處理這個層次的問題。近年大型語言模型與視覺語言模

型的進展，使機器人第一次具備了「理解指令並自行分解任務」的能力。我們會探討：如何把語言連接到動作、如何確保生成的計畫是可執行的、以及這些模型帶來的新問題（它們會**自信地說錯話**，而在機器人上這意味著危險的動作）。

第 33 章 (具身人工智慧) 則是整個第五篇的收束，將回到第 1 章的 Moravec 悖論——**為何對電腦而言下棋容易而摺衣服困難？智慧是否需要身體？**

換句話說：第 31 章讓機器人**學會做一件事**，第 32 章讓它**知道該做哪些事**。

完整計算例題

例題 31-1 折扣因子與有效視野

題目：某機器人的控制頻率為 50 Hz。(a) 若 $\gamma = 0.99$ ，求有效視野的步數與對應的時間。(b) 若任務需要規劃未來 5 秒的行為，求所需的 γ 。(c) 若 $\gamma = 0.9$ ，計算 100 步之後的獎勵權重。(d) 討論 γ 過大或過小的後果。

解 (a)：有效視野的常用估計為 $1/(1-\gamma)$ ：

$$\text{有效視野} = 1/(1 - 0.99) = 100 \text{ 步}$$

步數。

$$\text{時間} = 100 \text{ 步} / 50 \text{ Hz} = 2.0 \text{ 秒}$$

策略大約只「看到」未來 2 秒。

解 (b)：5 秒對應的步數為 $5 \times 50 = 250$ 步：

$$1/(1 - \gamma) = 250 \Rightarrow \gamma = 1 - 1/250 = 0.996$$

所需的折扣因子。

注意 γ 對視野的敏感度：由 0.99 提高到 0.996（只差 0.006），視野卻由 2 秒延長到 5 秒。**這是一個高度非線性的關係**——這也解釋了為何 γ 的微調常有出乎意料的效果。

解 (c)：第 100 步的獎勵權重為 γ^{100} ：

$$0.9^{100} = 2.66 \times 10^{-5}$$

權重只有 0.0027%——形同不存在。

這說明 $\gamma = 0.9$ 的策略**完全看不到 100 步之後的事**。若任務需要長期規劃（如「先繞路避開

障礙，雖然短期較慢但長期較快」)，這個設定必定失敗。

解 (d)：兩端的後果：

- γ 太小 (如 0.9)：策略「短視」，只顧眼前的獎勵。在需要犧牲短期利益的任務中會失敗 (如必須先後退才能繞過障礙)。
- γ 太大 (如 0.999)：有效視野達 1000 步，價值函數的估計變異極大、學習極為緩慢。且遠期的獎勵訊號被稀釋，難以分辨哪個動作真正有貢獻 (信用分配問題)。

實務建議：由任務所需的規劃長度反推 γ ，而非盲目取一個「常見值」。多數機器人任務的合理範圍是 0.95~0.995。

例題 31-2 複合誤差的估算

題目：某模仿學習訓練出的策略，單步的動作誤差率為 $\varepsilon = 0.02$ (2%)。(a) 依線性模型估計 100 步後的累積誤差。(b) 依複合誤差的 $O(\varepsilon T^2)$ 模型重新估計。(c) 比較兩者。(d) 若要把 100 步的失敗率控制在 10% 以下，求所需的單步誤差率。

解 (a)：線性模型假設誤差獨立累加：

$$\text{累積誤差} \approx \varepsilon \times T = 0.02 \times 100 = 2.0$$

但機率不能超過 1——這顯示線性模型在此已不適用。

解 (b)：複合誤差模型的失敗率上界為：

$$\text{失敗率} \approx \varepsilon \cdot T^2 = 0.02 \times 100^2 = 200$$

遠大於 1，代表幾乎必定失敗。

(嚴格地說，理論給出的是「期望的總代價上界為 $O(\varepsilon T^2)$ 」，實務上表現為「軌跡幾乎必定偏離」。))

解 (c)：兩者的比值為 $T = 100$ 倍。這正是複合誤差的可怕之處——它不是線性累加，而是與時間的平方成正比。

直觀的理解：每一次小偏離都會把機器人帶到「訓練資料中沒有的狀態」，而在那裡它的表現更差，於是偏離更多——這是一個正回饋。

解 (d)：要求 $\varepsilon \cdot T^2 \leq 0.1$ ，其中 $T = 100$ ：

$$\varepsilon \leq 0.1/10000 = 1 \times 10^{-5}$$

單步誤差率必須低於 0.001%。

工程判讀：這個要求極為嚴苛，純粹的行為複製幾乎不可能達成。這正是 31.4.3 節 DAgger 存在的理由——它改變了資料的分布，使「偏離後如何修正」也在訓練資料中，從而把誤差成長由 $O(\varepsilon T^2)$ 降為 $O(\varepsilon T)$ 。

另一個實務上的因應是縮短 T ：把長任務分解為多個短片段，每段結束時重新定位或由更高層的規劃器介入。這正是第 32 章任務分解的價值之一。

例題 31-3 訓練資源的估算

題目：某四足行走任務，PPO 需要 5 億步的互動才能收斂。模擬環境的單環境速度為每秒 2000 步。(a) 單環境訓練需要多久？(b) 若用 4096 個平行環境 (GPU)，總吞吐量為每秒 300 萬步，需要多久？(c) 若獎勵函數需要調整 20 次，每次都要重新訓練，求總時間。(d) 討論這對開發流程的影響。

解 (a)：單環境：

$$5 \times 10^8 / 2000 = 250,000 \text{ 秒} = 69.4 \text{ 小時} \approx 2.9 \text{ 天}$$

近三天。

解 (b)：平行環境：

$$5 \times 10^8 / 3 \times 10^6 = 167 \text{ 秒} \approx 2.8 \text{ 分鐘}$$

不到三分鐘——這正是第 30.8.2 節 GPU 平行的威力。

解 (c)：二十次迭代：

$$\text{單環境：} 2.9 \text{ 天} \times 20 = 58 \text{ 天 (約兩個月)}$$

$$\text{平行環境：} 2.8 \text{ 分鐘} \times 20 = 56 \text{ 分鐘 (不到一小時)}$$

差距約 1500 倍。

解 (d)：這個差距根本地改變了開發流程：

- **單環境 (兩個月)：**每次調整獎勵都要等三天，一個學期只能試十幾次。研究者必須極為謹慎地設計每一次實驗，且無法探索大量的想法。
- **平行環境 (一小時)：**可以在一個下午試十次不同的獎勵設計。這使「快速迭代」成為可能

——可以先粗略地試，看到結果再修正。

這正是第 30.8.2 節「量變引起質變」的具體體現。它不只是「快一點」，而是使一種完全不同的工作方式成為可能——從「深思熟慮的少數實驗」轉變為「快速迭代的大量嘗試」。第 28.9 節腿式機器人的突破，很大程度上來自這個轉變。

程式範例：從 Bellman 方程到 Q-learning

以下程式用一個極簡的網格世界，展示「已知模型的動態規劃」與「未知模型的 Q-learning」如何求解同一個問題——具體呈現第 18 章與本章的關係。

```
import numpy as np

# ===== 環境：一個 4x4 的網格世界 =====
class GridWorld:
    """
    機器人在 4x4 網格中移動，目標是到達右下角。
    有一個「陷阱」格子會給予懲罰。
    這是 MDP 的最小範例（31.5.1 節）。
    """
    def __init__(s, size=4):
        s.size = size
        s.goal = (size-1, size-1)
        s.trap = (1, 2)
        s.actions = [(0,1), (0,-1), (1,0), (-1,0)] # 右左下上
        s.n_states = size * size
        s.n_actions = 4

    def idx(s, pos): return pos[0]*s.size + pos[1]
    def pos(s, idx): return (idx // s.size, idx % s.size)

    def step(s, state_idx, action_idx):
        """執行動作，回傳（下一狀態，獎勵，是否結束）"""
        r0, c0 = s.pos(state_idx)
        dr, dc = s.actions[action_idx]
        r, c = np.clip(r0+dr, 0, s.size-1), np.clip(c0+dc, 0, s.size-1)
        nxt = s.idx((r, c))
        if (r, c) == s.goal: return nxt, 10.0, True
        if (r, c) == s.trap: return nxt, -10.0, True
        return nxt, -0.1, False # 每步小懲罰（鼓勵快速到達）

# ===== 方法一：動態規劃（第 18 章，已知模型）=====
def value_iteration(env, gamma=0.95, tol=1e-6):
    """
    價值迭代：用 Bellman 方程直接「計算」（31.6.2 節）
    前提：知道 step() 函式，也就是知道模型
    """
    V = np.zeros(env.n_states)
```

```

    iters = 0
    while True:
        delta = 0.0
        for si in range(env.n_states):
            if env.pos(si) in (env.goal, env.trap): continue
            # Bellman 最佳方程 :  $V(s) = \max_a [ R + \gamma \cdot V(s') ]$ 
            q_values = []
            for ai in range(env.n_actions):
                nxt, r, done = env.step(si, ai)
                q_values.append(r + (0 if done else gamma*V[nxt]))
            v_new = max(q_values)
            delta = max(delta, abs(v_new - V[si]))
            V[si] = v_new
        iters += 1
        if delta < tol: break
    return V, iters

# ===== 方法二：Q-learning (本章·未知模型) =====
def q_learning(env, episodes=3000, alpha=0.1, gamma=0.95,
               eps_start=1.0, eps_end=0.05, seed=0):
    """
    Q-learning: 只能「試」·不能「算」(31.6.2 節)
    它不使用 env 的內部結構·只透過 step() 取得經驗
    """
    rng = np.random.default_rng(seed)
    Q = np.zeros((env.n_states, env.n_actions))
    steps_used = 0
    for ep in range(episodes):
        #  $\epsilon$ -greedy 的探索率逐步下降 (探索與利用的取捨)
        eps = eps_end + (eps_start-eps_end)*np.exp(-3.0*ep/episodes)
        si = 0 # 由左上角出發
        for t in range(50):
            # 選動作：以機率 eps 隨機探索
            if rng.random() < eps:
                ai = rng.integers(env.n_actions)
            else:
                ai = int(np.argmax(Q[si]))
            nxt, r, done = env.step(si, ai)
            steps_used += 1
            # TD 更新 (31.6.2 節的核心)
            target = r + (0 if done else gamma*np.max(Q[nxt]))
            td_error = target - Q[si, ai]
            Q[si, ai] += alpha * td_error
            si = nxt
            if done: break
    return Q, steps_used

# ===== 比較 =====
env = GridWorld()
print("=== 方法一：動態規劃 (已知模型·第 18 章) ===")
V_dp, iters = value_iteration(env)
print(f"收斂所需迭代次數: {iters}")
print("價值函數 V(s):")
for r in range(env.size):
    print(" " + " ".join(f"{V_dp[env.idx((r,c))]:7.3f}" for c in range(env.size)))

```

```

print("\n=== 方法二：Q-learning ( 未知模型 · 本章 ) ===")
Q_rl, steps = q_learning(env)
V_rl = Q_rl.max(axis=1)
print(f"所需的環境互動步數：{steps:,}")
print("由 Q 導出的價值函數 max_a Q(s,a):")
for r in range(env.size):
    print(" " + " ".join(f"{V_rl[env.idx((r,c))]:7.3f}" for c in range(env.size)))

# 比較兩者的差異
mask = np.array([env.pos(i) not in (env.goal, env.trap)
                  for i in range(env.n_states)])
err = np.abs(V_dp[mask] - V_rl[mask]).max()
print(f"\n 兩者的最大差異：{err:.4f}")
print(f"動態規劃：{iters} 次迭代 ( 每次掃過所有狀態 · 需要模型 ) ")
print(f"Q-learning：{steps:,} 步互動 ( 不需模型 · 但要試很多次 ) ")
print("→ 同一條 Bellman 方程 · 兩種求解方式 ( 31.6 節 ) ")

# ===== 導出的策略 =====
print("\n=== 兩者導出的策略 ( →↔↑↓ · G=目標 X=陷阱 ) ===")
arrows = ['→', '↔', '↓', '↑']
def show_policy(Q_or_V, name, use_q=True):
    print(f"{name}:")
    for r in range(env.size):
        row = []
        for c in range(env.size):
            si = env.idx((r,c))
            if (r,c) == env.goal: row.append(' G'); continue
            if (r,c) == env.trap: row.append(' X'); continue
            if use_q:
                ai = int(np.argmax(Q_or_V[si]))
            else:
                qs = []
                for a in range(env.n_actions):
                    nxt, rr, dn = env.step(si, a)
                    qs.append(rr + (0 if dn else 0.95*Q_or_V[nxt]))
                ai = int(np.argmax(qs))
            row.append(' ' + arrows[ai])
        print(" " + " ".join(row))

show_policy(V_dp, "動態規劃的策略", use_q=False)
show_policy(Q_rl, "Q-learning 的策略", use_q=True)

# ===== 折扣因子的影響 ( 例題 31-1 ) =====
print("\n=== 例題 31-1：折扣因子與有效視野 ===")
print(f"{'γ':>8}{ '有效視野(步)':>14}{ '@50Hz(秒)':>12}{ '100 步後權重':>14}")
for g in (0.9, 0.95, 0.99, 0.996, 0.999):
    horizon = 1/(1-g)
    print(f"{g:8.3f}{horizon:13.0f}{horizon/50:11.2f}{g**100:14.2e}")
print("→ γ 由 0.99 到 0.996 只差 0.006 · 視野卻由 2 秒延長到 5 秒")

```

```

# ===== 探索率的影響 =====
print("\n=== 探索策略對學習的影響 ===")
print(f"{'探索設定':<22}{'最終誤差':>12}{'說明':>4}")
for label, e0, e1 in (("完全不探索 ( $\epsilon=0$ )", 0.0, 0.0),
                      ("少量探索 ( $\epsilon=0.1$ )", 0.1, 0.1),
                      ("遞減探索 ( $1 \rightarrow 0.05$ )", 1.0, 0.05),
                      ("持續高探索 ( $\epsilon=0.5$ )", 0.5, 0.5)):
    Qt, _ = q_learning(env, episodes=3000, eps_start=e0, eps_end=e1, seed=1)
    Vt = Qt.max(axis=1)
    e = np.abs(V_dp[mask] - Vt[mask]).max()
    print(f"{'label':<22}{'e':12.4f}")
print("> 完全不探索會困在局部；遞減探索（先探索後利用）效果最好")

# ===== 獎勵設計的影響（31.7 節）=====
print("\n=== 獎勵設計對行為的影響 ===")
class ModifiedGrid(GridWorld):
    """把每步的懲罰改掉·觀察行為變化"""
    def __init__(s, step_penalty=-0.1, size=4):
        super().__init__(size); s.pen = step_penalty
    def step(s, si, ai):
        r0, c0 = s.pos(si)
        dr, dc = s.actions[ai]
        r, c = np.clip(r0+dr,0,s.size-1), np.clip(c0+dc,0,s.size-1)
        nxt = s.idx((r,c))
        if (r,c) == s.goal: return nxt, 10.0, True
        if (r,c) == s.trap: return nxt, -10.0, True
        return nxt, s.pen, False

print(f"{'每步懲罰':>10}{'到達目標的步數':>16} 行為")
for pen in (0.0, -0.1, -1.0, -3.0):
    e2 = ModifiedGrid(step_penalty=pen)
    V2, _ = value_iteration(e2)
    # 依策略走一遍·計算步數
    si, n = 0, 0
    for _ in range(50):
        qs = []
        for a in range(e2.n_actions):
            nxt, rr, dn = e2.step(si, a)
            qs.append(rr + (0 if dn else 0.95*V2[nxt]))
        si, _, done = e2.step(si, int(np.argmax(qs)))
        n += 1
        if done: break
    behav = ("不急著到達" if pen == 0 else
            "正常" if pen >= -1.0 else "急於結束（可能冒險）")
    print(f"{'pen':10.1f}{'n':16d} {behav}")
print("> 獎勵的細節直接決定了行為——這正是 31.7 節的核心")

```

請試著修改：（一）把陷阱的懲罰由 -10 改為 -100 ，觀察策略是否變得更保守（繞更遠的路避開）。（二）把 Q-learning 的學習率 α 設為 0.9 ，觀察是否變得不穩定。（三）加入一個

「牆壁」使某些格子不可通行，觀察兩種方法是否都能找到繞路。

實作：訓練你的第一個機器人策略

本章實作以模擬環境為主。建議延續第 30 章建立的環境，或使用現成的 Gymnasium 與 Isaac Gym 範例。

1. **環境理解**：先用現成的環境（如 Gymnasium 的 Pendulum 或 CartPole）跑通流程。**手動下達動作**，觀察狀態與獎勵，確認自己理解了 MDP 的五個要素。
2. **動態規劃 vs Q-learning**：實作本章的程式範例，親眼看見兩者求解同一問題的差異。記錄各自所需的計算量。
3. **PPO 訓練**：用現成的實作（如 Stable-Baselines3）訓練一個簡單的任務。記錄學習曲線。
4. **折扣因子實驗**：改變 γ （0.9、0.95、0.99、0.999），觀察對學習速度與最終行為的影響。驗證例題 31-1 的分析。
5. **獎勵設計實驗**：對同一個任務設計三種不同的獎勵函數（稀疏、稠密、含懲罰項），比較學習速度與最終行為。**特別觀察是否出現獎勵駭客**。
6. **觀測空間實驗**：故意移除某些觀測（如速度資訊），觀察策略是否還能學會。接著加入歷史觀測，驗證 31.9.2 節「歷史使策略能推斷隱藏狀態」的說法。
7. **行為複製**：手動（或用一個已知的控制器）產生一批示範資料，用監督式學習訓練一個策略。**觀察複合誤差**——策略是否在執行一段時間後逐漸偏離。
8. **DAGger 實作**：對上一步的策略實作 DAGger——讓它自己跑、在它走過的狀態上用專家控制器標註、重新訓練。比較改善的幅度。
9. **領域隨機化**：把第 30.5 節的隨機化加入訓練，比較「有無隨機化」的策略在參數變動下的表現。
10. **（進階）四足或機械手臂**：若有 Isaac Gym 或類似環境，訓練一個四足行走或手臂到達的策略。這會讓你體會 31.8.4 節樣本效率的問題。
11. **（進階）安全層實作**：在學習的策略外加上一層模型式的安全檢查（力矩限制、工作空間限制），驗證即使策略輸出異常也不會造成危險。

表 31-12 強化學習實驗記錄表（前兩列為範例）

實驗項目	設定	結果	觀察	下一步
折扣因子	$\gamma = 0.9$	收斂快但行為短視	不會繞路避障	提高至 0.99
獎勵設計	只有距離獎勵	在目標旁徘徊	獎勵駭客	改用稀疏 + 懲罰
(自行填寫其餘實驗)				

國中小也看得懂：兩種學習方式

你是怎麼學會騎腳踏車的？

是先讀完一本《腳踏車力學原理》，把公式都背熟，然後一次就騎成功嗎？當然不是！你是一直摔、一直試——摔一次就知道「剛才那樣不行」，慢慢地身體就記住了怎麼保持平衡。

而且很有趣的是：你到現在都說不出來自己是怎麼做到的。如果有人問你「車子往左倒的時候，龍頭要轉幾度？」你根本答不出來——但你就是會騎。

機器人也有這兩種學習方式：

第一種：用算的。 像我們前面學的，先寫出公式，再算出答案。這個方法很準，而且保證不會出錯。但它有一個條件——你必須寫得出公式。

第二種：用試的。 讓機器人自己一直嘗試，做對了就給它「獎勵」（加分），做錯了就扣分。試了幾百萬次之後，它就會找出最好的方法了。

那什麼時候要用哪一種呢？很簡單：

能寫出公式的，就用算的——像是「手臂要轉幾度才能碰到那個點」，這個算得出來，而且算得超級準。

寫不出公式的，就用試的——像是「怎麼把一件皺成一團的衣服摺好」。衣服是軟的、會皺、會黏在一起，沒有人寫得出它的公式。這種就只能讓機器人自己去試。

最後告訴你一件很妙的事：教機器人最難的部分，不是寫程式，而是「決定怎麼給分」。

有一次，科學家想教機器人抓東西，就設定「夾子離物體越近，分數越高」。結果機器人學會了什麼？它把夾子貼在物體旁邊一直摩來摩去！分數拿滿了，但一次都沒抓起來過。

所以啊，你要很小心地想清楚「什麼才是真正想要的」——不然機器人會找到你想不到的

偷懶方法。這其實跟養小孩很像對不對？

叫獸提醒與常見錯誤

- 誤區一：以為學習方法比模型方法「更先進」。兩者是分工——有物理規律且需要保證的部分，模型永遠是更好的選擇。
- 誤區二：能建模卻用強化學習。若能寫出動力學，LQR 或 MPC 會更快、更準且有保證（31.6.3 節的啟示一）。
- 誤區三：用行為複製處理長時序任務。複合誤差以 $O(\epsilon T^2)$ 成長，必須用 DAgger 或縮短任務長度。
- 誤區四：折扣因子隨便取一個常見值。應由任務所需的規劃長度反推（有效視野 $\approx 1/(1-\gamma)$ ）。
- 誤區五：只看獎勵曲線而不看行為。獎勵上升不代表學對了——獎勵駭客只有實際觀察才發現得了。
- 誤區六：獎勵函數各項的量級不平衡。若一項是 100 而另一項是 0.01，後者形同不存在。
- 誤區七：把稠密獎勵設計得太具體。越接近「你想要的最終結果」越安全，越是中間步驟的引導越容易被鑽漏洞。
- 誤區八：在連續動作空間使用 DQN。價值型方法需要 $\arg\max$ ，只適用於離散動作。
- 誤區九：忽略觀測空間的設計。若狀態不含必要資訊（或缺少歷史），策略再好也學不會。
- 誤區十：在真機上做大規模的隨機探索。樣本效率使其不可行且危險——應在模擬中訓練（第 30 章）。
- 誤區十一：忘記正規化觀測。神經網路對輸入尺度敏感，量級差異大的觀測必須正規化。
- 誤區十二：把學習的策略直接部署而無安全層。必須有模型式的檢查攔截異常輸出（31.10.2 節）。

課堂討論

1. 31.6 節顯示強化學習與最佳控制共用 Bellman 方程，差別只在模型是否已知。請討論：這個認識如何改變你對「該用哪種方法」的判斷？
2. 獎勵駭客的三個案例中，機器人都「完美地達成了寫下的目標」。請討論這個現象對人工智

慧安全的啟示，並嘗試設計一個不容易被鑽漏洞的抓取獎勵函數。

3. 例題 31-3 顯示平行模擬使開發流程由「兩個月」變成「一小時」。請討論這種「量變引起質變」還出現在本書的哪些地方，以及它對研究方法的影響。
4. 31.10.1 節指出學習方法「沒有保證」且「分布外行為不可預測」。請討論：在什麼應用中這是可接受的？在什麼應用中絕對不行？並設計對應的安全架構。
5. 本章與第 21、25、28 章都得出「學習做判斷、模型做保證」的結論。請歸納這個分工原則的共同結構，並討論它是否有例外。

章末摘要

- 模型導向方法的優勢是保證、可解釋與資料效率；其邊界是「問題必須寫得出方程式」。
- 學習方法能處理無法建模的問題，代價是需要資料、沒有保證、且分布外行為不可預測。
- 模型與學習的三種混合模式：學習做前端、模型做骨架、模型產生資料。
- 三種學習典範的差別在於回饋的形式：正確答案（監督式）、評分（強化）、無回饋（非監督式）。
- 機器人學習的三個特殊性：資料昂貴、動作有後果、以及序列決策（動作改變後續狀態）。
- 監督式學習在機器人上最成功的應用是感知；其資料來源有人工標註、模擬、解析式標註與自我監督。
- 監督式學習的陷阱是分布偏移——部署環境與訓練不同時，模型可能自信地犯錯。
- 模仿學習從示範中學習，不需設計獎勵、樣本效率高、且安全。
- 行為複製的致命缺陷是複合誤差：誤差以 $O(\epsilon T^2)$ 成長，因為偏離後會進入訓練資料未涵蓋的狀態。
- DAgger 的核心是改變資料來源——在機器人自己走過的狀態上詢問專家，使資料涵蓋「如何修正」。
- MDP 的五個要素：狀態、動作、轉移機率、獎勵、折扣因子；其中轉移機率就是系統動力學。
- 有效視野 $\approx 1/(1-\gamma)$ ； γ 應由任務所需的規劃長度反推，而非取一個常見值。
- 強化學習與最佳控制共用同一條 Bellman 方程，唯一的本質差異是「模型是否已知」。
- 模型已知則可「計算」（動態規劃），未知則只能「取樣」（時序差分學習）。

- TD 誤差 $\delta = r + \gamma V(s') - V(s)$ 是「實際看到的」與「原本預期的」之差，走一步就能學一點。
- 獎勵函數的設計是實務上最困難的一環；稀疏獎勵探索困難、稠密獎勵容易被鑽漏洞。
- 獎勵駭客：機器人完美達成「寫下的」目標而非「想要的」目標；防範之道是使用接近真實目標的獎勵並實際觀察行為。
- 價值型方法只適用於離散動作；機器人的連續動作空間需要策略型或 Actor-Critic 方法。
- PPO 是模擬訓練的事實標準（穩定好調）；SAC 則因樣本效率高而適合真機訓練。
- 學習方法的四個限制：沒有保證、分布外不可預測、難以修改、樣本效率低。
- 因應之道是分層架構：低階用模型式控制、中階用學習、最上層必須有模型式的安全監督。

觀念題與計算題

觀念題

1. 說明模型導向方法的三個優勢與其適用邊界。
2. 列舉四類難以建模的機器人問題，並說明為何模型方法在此失效。
3. 比較三種學習典範的資料形式與回饋，並說明為何強化學習比監督式學習困難。
4. 說明機器人學習相對於影像辨識的三個特殊性。
5. 列舉監督式學習在機器人感知上的五個應用。
6. 說明機器人學習資料的四個來源及其優缺點。
7. 什麼是分布偏移？它在機器人上為何特別危險？有哪三種因應之道？
8. 說明模仿學習的四個優勢。
9. 解釋複合誤差的成因，並說明為何誤差以 $O(\epsilon T^2)$ 而非 $O(\epsilon T)$ 成長。
10. 說明 DAgger 的核心洞見與其四個步驟。
11. 列舉 MDP 的五個要素，並指出哪一個對應機器人的動力學。
12. 說明折扣因子的兩個作用，以及有效視野的估計方式。
13. 區分價值函數與 Q 函數，並說明後者為何在無模型情境下特別實用。
14. 說明強化學習與最佳控制的關係，並指出唯一的本質差異。
15. 解釋 TD 誤差的意義，並說明時序差分學習相對於動態規劃的差異。

16. 說明「有模型就別用強化學習」的理由，以及模型能如何加速學習。
17. 比較稀疏獎勵與稠密獎勵的優缺點。
18. 什麼是獎勵駭客？請說明三個案例的共通點與三個防範原則。
19. 說明價值型與策略型方法的差異，以及為何機器人幾乎都用後者。
20. 說明 PPO 解決的核心問題，以及它成為標準的原因。
21. 列舉學習方法的四個根本限制，並說明因應的分層架構。

計算題

1. 某機器人控制頻率 100 Hz。(a) 若 $\gamma = 0.995$ ，求有效視野的步數與時間。(b) 若任務需規劃未來 8 秒，求所需的 γ 。(c) 計算 $\gamma = 0.95$ 時第 200 步的獎勵權重。(d) 由此討論 γ 的選擇原則。
2. 某模仿學習策略的單步誤差率 $\epsilon = 0.005$ 。(a) 依 $O(\epsilon T^2)$ 模型估計 50 步與 200 步後的累積誤差。(b) 計算兩者的比值並說明。(c) 若要求 200 步的失敗率低於 5%，求所需的 ϵ 。(d) 討論達成此要求的兩種途徑。
3. 某任務用 PPO 需要 3 億步收斂。單環境速度為每秒 1500 步、平行環境 (2048 個) 總速度為每秒 200 萬步。(a) 分別計算兩者的訓練時間。(b) 若獎勵函數需調整 15 次，求總時間。(c) 計算兩者的比值。(d) 討論這對研究方式的影響。
4. 某網格世界的獎勵設定為：到達目標 +20、掉入陷阱 -20、每步 -0.5， $\gamma = 0.9$ 。(a) 若最短路徑為 6 步，計算沿最短路徑的回報。(b) 若繞路避開陷阱需 10 步，計算其回報。(c) 判斷策略會選擇哪一條。(d) 若把每步的懲罰改為 -2.0，重新判斷並討論。

動手做與專題延伸

本章實作：學習框架

建立一套機器人學習的實驗框架，這將是第 32 章的基礎。

- 階段一：實作 MDP 的基本框架與動態規劃，驗證第 18 章與本章的等價性。
- 階段二：實作 Q-learning 與 ϵ -greedy 探索，比較與動態規劃的差異。
- 階段三：接上現成的 PPO 實作 (Stable-Baselines3)，在標準環境上訓練。
- 階段四：實作行為複製與 DAgger，量化複合誤差與改善幅度。

- **階段五**：整合第 30 章的領域隨機化，訓練一個可轉移的策略。

專題延伸

- **Bellman 方程的兩種求解方式對照**：以一個中等規模的網格世界（如 10×10 含障礙），並排實作價值迭代與 Q-learning。繪出「所需計算量 vs 問題規模」的曲線，並記錄兩者收斂到相同解所需的資源。接著把環境改為「模型未知」（不允許動態規劃存取 step 函式），驗證只有 Q-learning 仍然可行。**這個專題能讓第 18 章與第 31 章的關係變得具體。**
- **獎勵駭客的實驗**：刻意設計三個「有漏洞」的獎勵函數（如只獎勵距離、只獎勵速度、只獎勵接觸次數），訓練並錄影記錄機器人學到的行為。你很可能會親眼看到 31.7.3 節描述的那些荒謬行為。接著逐步修正獎勵函數，記錄行為如何改變。**這個專題的教育價值極高——它會讓你永遠記得「要小心你許的願」。**
- **進階：模仿學習與強化學習的組合**。選一個中等難度的任務（如二維的到達或避障）。
 - （一）先用一個已知的控制器（如第 26 章的 DWA）產生示範資料；
 - （二）用行為複製訓練並量測複合誤差；
 - （三）用 DAgger 改善；
 - （四）以模仿學習的結果為初始策略，再用 PPO 微調；
 - （五）比較四種方式的最終性能與所需的樣本數。**你應該會看到組合方式兼具了樣本效率與最終性能**，驗證 31.4.4 節知識框的說法。

參考文獻與延伸閱讀

- Sutton, R. S. and Barto, A. G., Reinforcement Learning: An Introduction, 2nd ed., MIT Press（強化學習的權威教材，線上公開；第 3~6 章對應本章）。
- Levine, S., CS285 Deep Reinforcement Learning 課程講義（UC Berkeley，線上公開）（機器人強化學習的最佳課程資源之一）。
- Schulman, J. et al., "Proximal Policy Optimization Algorithms," arXiv 2017（PPO 的原始論文）。
- Haarnoja, T. et al., "Soft Actor-Critic: Off-Policy Maximum Entropy Deep RL," ICML 2018（SAC，真機訓練的主流選擇）。
- Ross, S., Gordon, G. and Bagnell, D., "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," AISTATS 2011（DAgger 與複合誤差的理論分析）。

- Chi, C. et al., "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion," RSS 2023 (擴散策略，模仿學習的近年突破) 。
- Hwangbo, J. et al., "Learning Agile and Dynamic Motor Skills for Legged Robots," Science Robotics, 2019 (腿式機器人的強化學習突破，呼應第 28.9 節) 。
- Kober, J., Bagnell, J. A. and Peters, J., "Reinforcement Learning in Robotics: A Survey," IJRR, 2013 (機器人強化學習的完整綜述) 。
- Amodei, D. et al., "Concrete Problems in AI Safety," arXiv 2016 (獎勵駭客與 AI 安全的系統性討論) 。
- Stable-Baselines3 與 Gymnasium 官方文件：強化學習演算法與環境介面的標準實作。

第 32 章

大型模型與任務規劃

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為一句話的拆解：中央是「幫我把桌上的杯子拿到廚房」這句話，以四條放射線連向四個問號框——「哪一個杯子？」（畫面中三個不同的杯子）、「廚房在哪？」（地圖上只有座標沒有標籤）、「要哪些步驟？」（六個空白的步驟框）、「什麼是常識？」（杯子要正著拿的圖示）。右半為 SayCan 的機制圖：上方一列是語言模型對五個技能的評分（長條圖）、中間一列是可供性的評分、下方是兩者相乘的結果，並以箭頭標出「Say 高但 Can 低」的技能如何被排除。下緣文字：語言模型知道世界通常是怎樣的，但不知道現在這裡是怎樣的。

叫獸開講 「幫我把桌上的杯子拿到廚房」——這句話難在哪裡？

實驗室裡那台服務機器人已經很能幹了。它會導航、會避障、會辨識物體、會抓取——第 22 到 31 章教的東西，它樣樣都會。

叫獸對著麥克風說：「幫我把桌上的杯子拿到廚房。」

機器人靜止不動。螢幕上跳出一行錯誤：`Unknown command`。

「問題在哪裡？」叫獸問。小翰說：「語音辨識沒開？」叫獸搖頭：「辨識沒問題，它聽到了每一個字。問題是它不知道這句話要做什麼。」

他在白板上把這句話拆開：「你想想，要完成這件事，它必須知道多少事情？

第一，什麼是『杯子』？它的視覺模型或許認得杯子，但如果桌上有三個杯子呢？我說的是哪一個？

第二，什麼是『廚房』？地圖上只有座標，沒有『廚房』這個標籤。

第三，要做哪些步驟？導航到桌邊、辨識杯子、規劃抓取、抓起來、導航到廚房、放下。這六步是我說的嗎？不是，是它要自己想出來的。

第四，什麼是常識？杯子要正著拿（不然裡面的東西會灑）、要放在平面上（不能放在地上）、經過門要先確認門是開的。」

小好說：「所以要把這些全部寫成規則？」

「以前確實是這樣做的。」叫獸點頭，「而且做了三十年。但你有沒有發現一個問題——你永遠寫不完。今天寫了『杯子』的規則，明天要處理『碗』；今天處理了『廚房』，明天要處理『陽台』。這就是第 1 章那個 Moravec 悖論的另一面：常識太多了，多到無法窮舉。」

他停頓一下：「但最近三年，事情有了根本的變化。大型語言模型讀了幾乎整個網際網路——它知道杯子要正著拿、知道牛奶在冰箱裡、知道『廚房』通常有水槽。那些我們寫不完的常識，它從文字中學到了。」

最後他補了一個關鍵的警告：「但這帶來一個新的、而且很危險的問題。語言模型會非常有自信地說錯話。在聊天時，說錯了頂多是笑話；在機器人上，說錯了就是一個真實的動作——可能打翻東西、可能撞到人。今天我們要學的，一半是怎麼用它，另一半是怎麼防它。」

本章為什麼重要

經過第 22 ~ 31 章，機器人已經具備了完整的**執行能力**：它能感知、移動、抓取、學習技能。但這些能力都停留在「**技能**」的層次——它會抓杯子，但不知道「什麼時候該抓杯子」。

任務層級的智慧是最後一塊拼圖。它要回答的問題是：

- **理解**：使用者說的話是什麼意思？「那個」指的是哪一個？
- **分解**：這個任務要拆成哪些步驟？順序如何？
- **常識**：哪些事情是「理所當然」但沒有明說的？
- **應變**：若某一步失敗了，該怎麼辦？

這些問題困擾了機器人學三十年。傳統的方法是**手工編寫規則與知識庫**，但如叫獸所說——**常識太多，寫不完**。

近三年，**大型語言模型 (LLM)** 與**視覺語言模型 (VLM)** 帶來了根本的改變。它們從海量的文字與影像中，隱含地學到了大量的常識與任務知識。這使機器人第一次能夠「理解」自然語言的指令，並自行分解任務。

但這也帶來了新的風險。**語言模型會產生看似合理但實際錯誤的內容 (幻覺)**——在聊天中這是笑話，在機器人上則是危險的動作。因此本章不只講「如何使用」，更要講**如何約束**：如何確保生成的計畫是**可執行、可驗證、且安全的**。

學習目標

- 能說明符號接地問題及其為何是機器人智慧的核心困難。
- 能說明傳統符號式任務規劃的原理與其三個限制。
- 能說明 PDDL 的基本結構並撰寫簡單的領域描述。
- 能說明大型語言模型的基本原理與其在機器人上的能力來源。
- 能區分語言模型作為規劃器的三種使用方式。
- 能說明程式碼生成作為機器人介面的優勢。
- 能說明可供性與價值函數如何約束語言模型的輸出。
- 能說明視覺語言模型如何連接感知與語言。

- 能說明視覺語言動作模型的架構與其意義。
- 能分析大型模型在機器人上的四類風險並設計防護。
- 能設計一個結合語言模型與傳統規劃的混合架構。
- 能評估任務規劃系統的正确性與安全性。

先備知識

第 26 章的路徑規劃（本章的任務規劃是其上一層）、第 31 章的學習方法與其限制、第 22 章的視覺感知、第 29 章的軟體架構、第 1 章的 Moravec 悖論與自主性光譜。

不需要深度學習或自然語言處理的專業背景。本章聚焦於「大型模型如何用於機器人」，而非模型本身的訓練細節。若對神經網路完全陌生，可先閱讀附錄 E。

核心名詞中英對照

表 32-1 本章核心名詞

中文	英文	說明
符號接地問題	Symbol Grounding Problem	如何把符號與真實世界連結
任務規劃	Task Planning	決定達成目標所需的動作序列
任務與運動規劃	Task and Motion Planning, TAMP	同時處理離散決策與連續運動
PDDL	Planning Domain Definition Language	描述規劃問題的標準語言
前提條件	Precondition	動作可執行所需滿足的條件
效果	Effect	動作執行後世界狀態的改變
大型語言模型	Large Language Model, LLM	以海量文字訓練的生成式模型
視覺語言模型	Vision-Language Model, VLM	同時處理影像與文字的模型
視覺語言動作模型	Vision-Language-Action, VLA	直接輸出機器人動作的多模態模型
提示工程	Prompt Engineering	設計輸入以引導模型輸出的技術
思維鏈	Chain-of-Thought	引導模型逐步推理的提示方式
可供性	Affordance	環境或物體所提供的行動可能性
幻覺	Hallucination	模型產生看似合理但錯誤的內容
接地	Grounding	把語言連結到真實的感知與動作
技能庫	Skill Library	機器人可執行的原子動作集合
閉環回饋	Closed-loop Feedback	以執行結果修正後續計畫

中文	英文	說明
語意地圖	Semantic Map	含物體類別與空間關係的地圖
零樣本	Zero-shot	未經該任務訓練即可執行

32.1 符號接地問題

32.1.1 三十年的老問題

機器人要理解「把杯子拿到廚房」，必須解決一個哲學層次的問題：「杯子」這兩個字，如何連結到眼前那個具體的物體？

這稱為**符號接地問題**（Symbol Grounding Problem），由 Harnad 於 1990 年提出。它的核心是：

◎ 符號接地問題

符號本身沒有意義。「杯子」這兩個字，對電腦而言只是一串位元組。若我們用其他符號來定義它（「杯子是一種容器」），那「容器」又是什麼？這會陷入無窮的循環——**符號只能用符號解釋，永遠接觸不到真實世界**。要打破這個循環，符號必須**接地**（grounded）到感知與行動：「杯子」意味著「看起來像這樣的東西」、以及「可以這樣抓起來的東西」。

這個問題對機器人特別關鍵，因為機器人**必須行動**。一個純語言的系統可以在符號之間打轉；但機器人必須把「杯子」變成一個具體的抓取姿態、把「廚房」變成一個具體的座標。

32.1.2 三個層次的接地

實務上，接地問題可以分解為三個層次：

表 32-2 符號接地的三個層次

層次	問題	所需技術	本書章節
物體接地	「杯子」對應畫面中的哪個區域？	物體偵測與分割	第 22.8 節
空間接地	「桌上」「廚房」對應什麼座標？	語意地圖、空間關係推理	第 25 章 + 本章
動作接地	「拿」對應什麼運動指令？	技能庫、抓取規劃	第 21、26 章

第三列最容易被忽略但最關鍵。「拿起杯子」這個符號，最終必須變成一連串的關節力矩——而中間的每一層轉換，都是前面三十一章的內容。

32.1.3 為何近年有了突破

符號接地問題困擾了三十年，為何近年突然有了進展？三個原因：

原因一：視覺模型能辨識開放詞彙。傳統的物體偵測只能辨識訓練過的固定類別（如 COCO 的 80 類）。近年的視覺語言模型（如 CLIP）能處理任意的文字描述——你可以問「畫面中哪個是藍色的馬克杯」，即使它從未見過這個特定的類別。這直接解決了物體接地。

原因二：語言模型內含大量常識。它從文字中學到了「杯子通常在桌上」、「廚房有水槽」、「牛奶要冷藏」。這些常識過去必須手工編寫，現在是免費附贈的。

原因三：語言成為通用的介面。過去每個模組都有自己的資料格式；現在語言可以作為模組之間共同語言——視覺模型輸出「桌上有一個紅色杯子」，語言模型讀懂它並輸出「移動到桌子、抓取紅色杯子」。

32.2 傳統的符號式任務規劃

32.2.1 STRIPS 與 PDDL

在大型模型出現之前，任務規劃有一套成熟的形式化方法。其核心是把世界描述為一組符號命題，把動作描述為前提條件與效果。

最經典的表示法是 STRIPS，其現代標準是 PDDL（Planning Domain Definition Language）。一個動作的定義包含三部分：

動作名稱（參數）

前提條件（Preconditions）：什麼情況下可以執行

效果（Effects）：執行後世界如何改變

這個結構極為簡潔，卻能表達複雜的任務。

```
;; PDDL 領域定義：一個簡化的取放任務
(define (domain robot-pickplace)
  (:requirements :strips :typing)

  (:types robot object location)

  (:predicates
    (at ?r - robot ?l - location)      ; 機器人在某位置
    (on ?o - object ?l - location)     ; 物體在某位置
    (holding ?r - robot ?o - object)   ; 機器人正握著物體
```

```

(hand-empty ?r - robot)          ; 手是空的
(reachable ?l - location))        ; 該位置可到達

;; 動作一：移動
(:action move
 :parameters (?r - robot ?from ?to - location)
 :precondition (and (at ?r ?from) (reachable ?to))
 :effect (and (at ?r ?to) (not (at ?r ?from))))

;; 動作二：拿起
(:action pick
 :parameters (?r - robot ?o - object ?l - location)
 :precondition (and (at ?r ?l) (on ?o ?l) (hand-empty ?r))
 :effect (and (holding ?r ?o)
              (not (on ?o ?l))
              (not (hand-empty ?r))))

;; 動作三：放下
(:action place
 :parameters (?r - robot ?o - object ?l - location)
 :precondition (and (at ?r ?l) (holding ?r ?o))
 :effect (and (on ?o ?l)
              (hand-empty ?r)
              (not (holding ?r ?o))))

;; 問題定義：把杯子從桌子拿到廚房
(define (problem cup-to-kitchen)
  (:domain robot-pickplace)
  (:objects
   r1 - robot
   cup - object
   table kitchen - location)
  (:init
   (at r1 table)
   (on cup table)
   (hand-empty r1)
   (reachable table)
   (reachable kitchen))
  (:goal (on cup kitchen)))

```

規劃器讀入這兩個檔案，會自動輸出動作序列：

pick(r1, cup, table) → move(r1, table, kitchen) → place(r1, cup, kitchen)

這正是叫獸開場時說的「六個步驟」中的核心三步——而它是自動推導出來的，不是人寫的。

32.2.2 規劃的搜尋

PDDL 規劃器的工作，本質上是在狀態空間中搜尋——與第 26 章的路徑規劃是同一類問題：

表 32-3 路徑規劃與任務規劃的對照

面向	路徑規劃 (第 26 章)	任務規劃 (本章)
狀態	機器人的位形 (x, y, θ)	世界的符號狀態 (哪些命題為真)
動作	連續的運動	離散的動作 (pick、place、move)
狀態空間	連續、低維	離散、極高維 (2^n 個命題組合)
演算法	A^* 、RRT	A^* 、啟發式搜尋 (如 FF、Fast Downward)
主要困難	高維連續空間	組合爆炸

兩者的演算法思想相通——都用啟發式搜尋。差別在於任務規劃的狀態空間是離散的，且會因為命題數量而組合爆炸 (n 個命題有 2^n 種狀態)。

32.2.3 傳統方法的三個限制

符號式規劃在理論上優雅、且有**正確性的保證** (規劃器找到的計畫必定滿足所有前提條件)。但它有三個致命的實務限制：

限制一：知識工程的負擔。每一個物體、每一個動作、每一個相關的性質，都必須手工編碼。一個真實的家庭環境有數百種物體與數千種可能的狀態——**這個工作永遠做不完**。

限制二：無法處理常識。「杯子要正著拿」、「熱的東西要小心」、「玻璃會碎」——這些常識若要編碼，數量是無窮的。這正是第 1 章 Moravec 悖論在任務層次的體現。

限制三：符號從哪來。PDDL 假設世界狀態已經是符號形式 (`(on cup table)`)。但機器人看到的是**像素**——由像素到符號的轉換，正是 32.1 節的接地問題，而傳統方法對此無解。

◎ 為何符號式規劃仍然重要

儘管有上述限制，符號式規劃**並未過時**——它在本章的架構中扮演關鍵角色。原因是它提供了大型模型所缺乏的東西：**正確性的保證**。一個 PDDL 規劃器產生的計畫，**保證每一步的前提條件都被滿足**；而語言模型產生的計畫可能包含「拿起一個不存在的杯子」這種錯誤。因此現代的架構常是**混合式**：用語言模型處理理解與常識 (它擅長的)，用符號式規劃驗證計畫的可行性 (它擅長的)。這與第 21.9.5 節的抓取、第 25.10.3 節的 SLAM、第 31.10.2 節的安全層，是同一種分工。

32.3 大型語言模型的能力來源

32.3.1 它到底在做什麼

大型語言模型的訓練目標極為簡單：**預測下一個詞**。給定「今天天氣很」，預測下一個詞可

能是「好」、「熱」、「糟」。

這個看似平凡的目標，在**極大規模**的訓練下產生了意想不到的能力。原因是：**要準確地預測下一個詞，模型必須理解語意、掌握常識、甚至進行推理。**

「杯子從桌上掉下來，結果它____」

要預測「破了」，模型必須知道：杯子易碎、地板是硬的、重力會使它加速

預測下一個詞的目標，隱含地要求了對世界的理解。

32.3.2 對機器人有價值的三種能力

大型語言模型有許多能力，但對機器人特別有價值的是三種：

能力一：常識知識。這是最直接的價值。模型知道：

- **物體的性質：**杯子可以裝液體、刀子很鋒利、玻璃會碎。
- **空間的常識：**廚房有水槽、書在書架上、鞋子在門口。
- **行為的規範：**熱的東西要小心、杯子要正著拿、開門要先確認沒人。

能力二：任務分解。給定一個高階目標，模型能拆解為步驟。這正是叫獸開場提到的「第三」——六個步驟不是人說的，是機器人要自己想出來的。

能力三：語意理解與指代消解。「把那個紅色的拿過來」、「放在旁邊」——這些模糊的指代，模型能依上下文推斷。

32.3.3 三個根本的限制

但語言模型也有三個對機器人特別危險的限制：

限制一：幻覺。模型會產生看似合理但完全錯誤的內容，且帶著高度的自信。它可能建議「打開櫃子取出杯子」——但那個櫃子根本不存在。

限制二：沒有物理直覺。模型從文字學習，而文字中很少描述物理細節。它知道「杯子會破」，但不知道「這個角度施力 5 牛頓會不會破」。第 13 到 21 章的所有物理知識，它幾乎都不具備。

限制三：不知道機器人能做什麼。模型可能規劃出「跳上桌子」或「用兩隻手同時操作」——而這台機器人可能根本沒有腿或只有一隻手臂。

◎ 一個關鍵的認識

語言模型知道「世界通常是怎樣的」，但不知道「現在這裡是怎樣的」，也不知道「這台機器人能做什麼」。這三者的落差，正是本章後續所有技術要處理的問題——32.5 節的可供性用來告訴它「現在能做什麼」、32.6 節的視覺語言模型用來告訴它「現在看到什麼」、而 32.9 節的安全層則負責攔截它的錯誤。把語言模型當成一個「見多識廣但沒到過現場的顧問」，是最貼切的理解。

32.4 語言模型作為任務規劃器

32.4.1 三種使用方式

把語言模型用於機器人任務規劃，有三種漸進的方式：

表 32-4 語言模型的三種使用方式

方式	輸出形式	優點	缺點
自然語言步驟	「先走到桌子，然後拿起杯子」	最直觀	難以自動執行（需再解析）
技能序列	goto(table); pick(cup); goto(kitchen)	可直接執行	受限於預定義的技能
程式碼生成	完整的 Python 程式（含迴圈與判斷）	最靈活、可表達複雜邏輯	更容易出錯、需嚴格驗證

32.4.2 方式一與二：從語言到技能

最務實的作法是**方式二**：定義一組**技能庫**（機器人能執行的原子動作），要求語言模型只用這些技能組合出計畫。

提示（Prompt）的典型結構

SYSTEM_PROMPT = """

你是一台服務機器人的任務規劃器。你只能使用以下技能：

```
goto(location)      - 移動到指定地點
pick(object)        - 拿起物體（手必須是空的）
place(location)     - 把手中的物體放下
open(container)     - 打開容器
close(container)    - 關閉容器
say(text)           - 說話
```

目前可見的物體：{visible_objects}

目前可到達的地點：{known_locations}

機器人狀態：{robot_state}

規則：

1. 只能使用上列技能，不可發明新技能。
2. 只能操作「目前可見的物體」清單中的物體。
3. `pick` 之前手必須是空的；`place` 之前手必須有東西。
4. 若指令無法達成，回覆 `CANNOT`：加上原因。

請輸出計畫，每行一個技能呼叫，不要有其他文字。

"""

USER = "幫我把桌上的杯子拿到廚房"

模型的輸出

```
PLAN = """
goto(table)
pick(cup)
goto(kitchen)
place(counter)
"""
```

這個提示設計包含四個關鍵元素：

- **技能清單**：明確限定模型能用什麼——防止它發明機器人做不到的動作（32.3.3 節的限制三）。
- **當前狀態**：告訴它現在看到什麼、能去哪裡——彌補「不知道現在這裡是怎樣」的落差。
- **規則**：明確的前提條件——這其實是把 PDDL 的知識用自然語言表達。
- **失敗的出口**：允許它說「做不到」——**這極為重要**，否則模型會硬生出一個錯誤的計畫。

32.4.3 方式三：程式碼作為介面

程式碼生成是近年最受矚目的方式（如 Google 的 Code as Policies）。它的核心洞見是：

◎ 為何程式碼是好的介面

程式語言恰好具備任務規劃所需要的一切結構：順序（一行接一行）、條件（if 判斷）、迴圈（重複處理多個物體）、函式（技能的呼叫與組合）、以及變數（記住中間結果）。而純粹的技能序列（方式二）只能表達「順序」——若任務是「把桌上**所有**的杯子都收起來」，技能序列必須事先知道有幾個杯子，程式碼卻可以用一個 for 迴圈處理。此外，語言模型在程式碼上的訓練資料極為豐富且結構嚴謹，因此生成程式碼的品質往往優於生成自然語言的計畫。

```
# 語言模型生成的程式碼 (Code as Policies 的風格)
# 指令：「把桌上所有的杯子都放到水槽裡，但如果杯子是滿的就先倒掉」

def execute_task():
    goto('table')
    cups = detect_objects('cup')          # 視覺模組回傳所有杯子

    if len(cups) == 0:
        say('桌上沒有看到杯子')
        return

    for cup in cups:
        pick(cup)
        # 用視覺語言模型判斷杯子是否是滿的
        if vlm_query(f'這個杯子裡有液體嗎?', get_image()):
            goto('sink')
            pour_out()                    # 倒掉
            goto('sink')
            place('sink')
            goto('table')                # 回來拿下一個

    say(f'已經收好 {len(cups)} 個杯子')
```

注意這段程式碼展現的能力：迴圈處理未知數量的物體、條件判斷、以及呼叫視覺語言模型做即時的判斷（`vlm\_query`）。這是純技能序列完全做不到的。

32.4.4 思維鏈：讓它先想再做

一個簡單卻極為有效的技巧是**思維鏈**（Chain-of-Thought）：要求模型在輸出計畫之前，先寫出推理過程。

直接輸出計畫 → 錯誤率較高

先推理再輸出計畫 → 錯誤率顯著降低

這與人類「先想清楚再動手」是同樣的道理。

```
# 加入思維鏈的提示
PROMPT_WITH_COT = """
在輸出計畫之前，請先分析：
1. 使用者的目標是什麼？
2. 目前的狀態如何？有哪些相關的物體？
3. 需要哪些步驟？每一步的前提條件是否滿足？
4. 有什麼可能出錯的地方？

分析完成後，再輸出計畫。
```

```

格式：
分析：<你的推理>
計畫：
<技能呼叫，每行一個>
"""

# 模型的輸出
"""
分析：
1. 目標是把杯子移到廚房。
2. 目前機器人在起始點，杯子在桌上，手是空的。
   可見物體：cup, book, plate。可到達：table, kitchen, door。
3. 步驟：先到桌邊（因為 pick 需要在物體所在位置）、
   拿起杯子（手是空的，前提滿足）、移動到廚房、放下。
4. 可能的問題：杯子若是滿的，移動時可能灑出。
   但指令未提及，先假設是空的。

計畫：
goto(table)
pick(cup)
goto(kitchen)
place(counter)
"""

```

思維鏈還有一個額外的好處：**可解釋性**。當計畫出錯時，可以從推理過程中看出它是在哪一步想錯了——這對除錯極為有價值，也部分彌補了 31.10.1 節提到的「學習方法難以除錯」的問題。

32.5 可供性：把語言接地到能力

32.5.1 問題：模型不知道自己能做什麼

32.3.3 節指出，語言模型的第三個限制是「不知道機器人能做什麼」。這在實務上會產生兩類錯誤：

- **能力錯誤**：規劃出機器人做不到的動作（如「爬上桌子」，但這是一台輪型機器人）。
- **情境錯誤**：規劃出當下不可行的動作（如「拿起杯子」，但杯子在關著的櫃子裡）。

解法的核心概念是「**可供性**」（affordance）——這個詞源自心理學家 Gibson，意指「環境提供給行動者的行動可能性」。

語言模型說：「這件事「應該」做」（依常識）

可供性說：「這件事「現在能不能」做」（依實況）

兩者結合 → 可執行的計畫

這是 SayCan 這類方法的核心思想。

32.5.2 SayCan：說的 × 能的

Google 的 SayCan 提出了一個極為優雅的框架。對每一個候選技能，計算兩個分數並相乘：

$$\text{分數(技能)} = P_{\text{LLM}}(\text{技能} \mid \text{指令}) \times P_{\text{affordance}}(\text{成功} \mid \text{當前狀態})$$

「Say」是語言模型的意見、「Can」是實際的可行性。

兩個分數的來源不同：

- $P_{\text{LLM}}(\text{Say})$ ：語言模型評估「這個技能對達成目標有多相關」。它有常識但不知道現場狀況。
- $P_{\text{affordance}}(\text{Can})$ ：由價值函數提供——這正是第 31 章強化學習訓練出的 Q 函數！它評估「在當前的視覺觀測下，這個技能有多大機率成功」。

這個設計的巧妙之處在於兩者的互補：

表 32-5 Say 與 Can 的互補

情境	Say (語言模型)	Can (可供性)	相乘後	結果
合理且可行	高	高	高	執行 ✓
合理但不可行	高	低	低	跳過 (如杯子在關著的櫃中)
可行但不相關	低	高	低	跳過 (如「打開電視」與任務無關)
既不合理也不可行	低	低	低	跳過

第二列是關鍵：語言模型說「拿杯子」很合理，但視覺告訴我們杯子在關著的櫃子裡，抓取的成功率極低——相乘後這個選項被排除，模型會轉而選擇「打開櫃子」。系統因此自動學會了正確的順序，而不需要有人寫下這條規則。

32.5.3 可供性從哪裡來

可供性分數有三種來源，各有優缺：

表 32-6 可供性的三種來源

來源	作法	優點	缺點
學習的價值函數	用第 31 章的 RL 訓練 Q 函數	準確、反映真實能力	需要大量訓練
幾何檢查	用第 26 章的規劃器檢查可達性	有保證、快速	只涵蓋幾何，不含成功率
視覺語言模型	問 VLM 「這個動作現在可行嗎」	零樣本、易實作	可能不準

實務上常三者並用：先用幾何檢查快速排除不可能的（如超出工作空間）、再用價值函數評估成功率、必要時用 VLM 補充判斷。

32.5.4 閉環：讓執行結果回饋到規劃

閉環的規劃有一個根本問題：**若某一步失敗了，後續的計畫全部失效**。例如規劃了「拿杯子 → 去廚房 → 放下」，但抓取失敗了——機器人仍會空手走到廚房並做出放下的動作。

閉環的架構在每一步執行後檢查結果，並據此更新計畫：

```
def execute_with_feedback(instruction, max_steps=20):
    """閉環的任務執行 ( Inner Monologue 的架構 ) """
    history = []

    for step in range(max_steps):
        # 1. 取得當前的觀測
        image = camera.capture()
        scene = vlm_describe(image)          # 視覺語言模型描述場景
        robot_state = get_robot_state()

        # 2. 由語言模型決定下一步 ( 考慮歷史與當前狀態 )
        prompt = build_prompt(instruction, scene, robot_state, history)
        candidates = llm_propose_skills(prompt)  # 產生候選技能

        # 3. 用可供性過濾 ( SayCan 的核心 )
        scored = []
        for skill in candidates:
            p_say = candidates[skill]          # LLM 的分數
            p_can = affordance_model(skill, image)  # 可供性分數
            scored.append((skill, p_say * p_can))
        best_skill, score = max(scored, key=lambda x: x[1])

        # 4. 安全檢查 ( 32.9 節 )
        if not safety_check(best_skill, robot_state):
            history.append(f'{best_skill}: 未通過安全檢查')
            continue
        if score < THRESHOLD:
```

```

say('我不確定該怎麼做，請給我更多指示')
return False

# 5. 執行並取得結果
success = execute_skill(best_skill)

# 6. 把結果加入歷史——這是「閉環」的關鍵
result = '成功' if success else '失敗'
history.append(f'{best_skill}: {result}')

if not success:
    # 讓語言模型看到失敗，它會自行調整策略
    # 例如：抓取失敗 → 它可能會提議「換一個角度再試」
    continue

# 7. 檢查是否完成
if llm_check_done(instruction, scene, history):
    return True

return False

```

這個架構稱為「內心獨白」(Inner Monologue) ——機器人在每一步之後「告訴自己」發生了什麼，並依此調整。它的價值在於：

- 能從失敗中恢復：抓取失敗時，模型看到「pick(cup): 失敗」，可能會提議換個角度或先移開遮擋物。
- 能處理環境變化：若有人把杯子移走了，下一步的場景描述會反映這件事。
- 能處理意外：發現杯子是滿的、發現門是鎖的——這些都能即時反應。

32.6 視覺語言模型

32.6.1 連接看見與說出

前面的架構有一個環節尚未說明：機器人如何把「看到的」變成「語言模型能讀的」？這就是視覺語言模型 (Vision-Language Model, VLM) 的工作。

VLM 能同時處理影像與文字，執行三類任務：

表 32-7 VLM 在機器人上的三類用途

用途	輸入	輸出	機器人上的應用
場景描述	影像	文字描述	告訴語言模型「現在看到什麼」
視覺問答	影像 + 問題	答案	「杯子是滿的嗎？」「門開著嗎？」
開放詞彙定位	影像 + 物體描述	位置（框或遮罩）	「藍色馬克杯」在畫面的哪裡

第三列直接解決了 32.1.2 節的物體接地問題。傳統的物體偵測只認得訓練過的固定類別；VLM 則能處理任意的文字描述——「左邊那個有把手的杯子」、「最大的那個盒子」。

32.6.2 開放詞彙的意義

這個能力的價值極大。比較兩種作法：

傳統：訓練一個 80 類的偵測器 → 只認得那 80 類
VLM：用文字查詢 → 可以認得任何能用語言描述的東西

這是質的差異，而非量的差異。

具體的影響：一台家用機器人不可能事先訓練所有可能遇到的物體（每個家庭的東西都不同）。有了開放詞彙的能力，使用者可以直接說「拿我的藍色水壺」——即使系統從未見過那個特定的水壺。

32.6.3 VLM 的限制

但 VLM 在機器人上有三個重要的限制：

限制一：空間關係的精度不足。VLM 能說「杯子在盤子旁邊」，但難以精確判斷「杯子在盤子右邊 5 公分處」。對抓取而言，這個精度遠遠不夠（第 21 章的容差只有數公釐）。因此實務上仍需傳統的幾何方法計算精確位姿。

限制二：對物理性質不敏感。它能認出「這是玻璃杯」，但無法判斷「這個杯子有多重」、「這個表面有多滑」。第 30 章的 Sim-to-Real 落差在此重現。

限制三：推論成本高。大型 VLM 的單次推論可能需要數百毫秒到數秒——這遠慢於控制迴路。因此它只能用於低頻的決策層（第 3 章的時間尺度金字塔的最上層），而非即時控制。

◎ VLM 與傳統視覺的分工

這再次體現了本書反覆出現的分工原則。**VLM 負責「這是什麼、該怎麼理解」**——開放詞彙的辨識、語意的判斷、常識的應用。**傳統視覺負責「它精確地在哪裡」**——第 22 章的相機模型、位姿

估計、深度計算。一個典型的流程是：VLM 回答「藍色杯子在畫面的這個區域」，再用傳統的方法（點雲配準或 PnP）算出它精確到公釐的六自由度位姿。**語意用學習、幾何用模型**——與第 25.10.3 節的 SLAM 前後端完全相同的思維。

32.7 視覺語言動作模型

32.7.1 更激進的一步

前面各節的架構是**模組化**的：VLM 看、LLM 想、技能庫做。近年出現了一個更激進的方向——把三者合而為一。

視覺語言動作模型（Vision-Language-Action, VLA）直接由「影像 + 語言指令」輸出「機器人動作」：

傳統模組化：影像 → VLM → 文字 → LLM → 技能 → 控制器 → 動作

VLA：（影像，指令）→ 單一模型 → 動作

中間的所有轉換都被吸收進一個模型中。

32.7.2 為何可能有效

這個作法初看違反直覺——它放棄了模組化的所有好處。但它有三個潛在的優勢：

優勢一：避免資訊損失。模組化的每一次轉換都會丟失資訊。「杯子在畫面右側」這句話，遠不如原始影像所含的資訊豐富。端到端的模型能保留全部。

優勢二：從網路知識中遷移。VLA 通常以預訓練的 VLM 為基礎（已經理解了視覺與語言），再用機器人資料微調。這使它能**把網路上學到的常識，直接帶到動作層**。例如它可能從未見過「拿起恐龍玩具」的機器人示範，但因為它認得恐龍玩具，就能泛化過去。

優勢三：不需要設計介面。模組化架構需要人為定義技能庫、狀態表示、以及模組間的協定。VLA 完全跳過這些設計。

32.7.3 RT-2 與其後續

Google 的 RT-2（Robotic Transformer 2）是這個方向的代表。它的核心技巧極為巧妙：

◎ 把動作當成文字

RT-2 的關鍵洞見是：**把機器人的動作編碼成「詞」**。例如把每個關節的位移離散化為 256 個區間，每個區間對應一個特殊的詞元（token）。這樣一來，「輸出動作」就變成了「輸出一串文

字」——而這正是語言模型最擅長的事。於是整個預訓練好的視覺語言模型，可以幾乎不做架構修改就用於機器人控制。這個技巧使得「網路上的知識」與「機器人的動作」能在同一個模型中共存——RT-2 展示了它能理解「把水果放到能裝水的容器裡」，即使訓練資料中沒有這個組合。

後續的發展包括 **Open X-Embodiment** (跨機構、跨機器人的大規模資料集) 與各種開源的 VLA 模型。這個方向目前發展極快。

32.7.4 目前的限制

但 VLA 距離實用仍有距離，四個主要的限制：

表 32-8 VLA 目前的四個限制

限制	說明	影響
推論速度	大型模型的單次推論需數十至數百毫秒	只能做低頻控制 (3~10 Hz)
資料需求	需要大量的真實機器人示範資料	收集成本極高
沒有保證	無法驗證輸出動作的安全性	必須有外部的安全層
精度不足	離散化的動作解析度有限	難以做精密操作 (第 20 章的裝配)

因應之道是分層：VLA 負責高層的、粗略的決策 (3~10 Hz)，底層仍用傳統的控制器 (1 kHz) 執行精確的運動。這正是第 27、28 章分層架構思想的延續。

32.8 任務與運動規劃的整合

32.8.1 符號與幾何的落差

32.2 節的符號式規劃有一個根本的問題：它假設每個符號動作都能執行。

但實際上不一定。考慮這個計畫：

pick(cup) → place(cup, shelf)

符號上完全合理——前提條件都滿足。

但實際執行時可能發現：手臂構不到那個位置 (超出工作空間，第 12 章)、路徑被擋住 (第 26 章的規劃無解)、或抓取姿態會使手臂與桌子碰撞。

符號層不知道這些幾何的細節，而幾何層不知道任務的目標——這個落差稱為 **TAMP 問題** (Task and Motion Planning)。

32.8.2 為何困難

TAMP 的困難在於兩層必須互相依賴：

要選符號動作 → 必須知道它幾何上可行

要檢查幾何可行 → 必須先選定符號動作

這是一個雞生蛋的循環——與第 25 章 SLAM 的結構相同。

更麻煩的是連續參數的選擇。「把杯子放到架子上」這個符號動作，實際上包含無限多種可能——放在架子的哪個位置？手臂用什麼姿態？而這個選擇會影響後續動作的可行性（若放得太靠內側，下一次可能拿不到）。

32.8.3 常見的解法

TAMP 有三類主要的解法：

表 32-9 TAMP 的三類解法

方法	作法	優點	缺點
先符號後幾何	先規劃符號序列，逐一檢查幾何可行性，失敗則回溯	實作簡單	回溯可能很慢
交錯式	符號搜尋的每一步都呼叫幾何檢查	較有效率	實作複雜
取樣式	同時對符號與連續參數取樣（如 PDDLStream）	能處理連續參數	計算量大

大型模型為 TAMP 帶來的新可能：語言模型可以用常識提出好的候選（如「杯子應該放在架子外側比較好拿」），大幅減少搜尋的範圍。這是「學習提供啟發、規劃提供保證」的又一個例子（呼應第 26.10.2 節的學習式規劃）。

32.8.4 完整的架構

整合本章至此的內容，一個現代的機器人任務系統包含五層：

表 32-10 現代機器人任務系統的五層架構

層級	負責	技術	頻率	本書章節
理解層	解析指令、釐清意圖	LLM	每次指令一次	32.3~32.4
任務規劃層	分解為動作序列	LLM + 符號規劃	0.1~1 Hz	32.2、32.4
接地層	把符號連結到實際的物體與位姿	VLM + 傳統視覺	1~10 Hz	32.5~32.6、第 22 章

層級	負責	技術	頻率	本書章節
運動規劃層	產生無碰撞的軌跡	RRT、A*、軌跡最佳化	1~10 Hz	第 15、26 章
控制層	精確執行軌跡	PID、CTC、力控制	100~1000 Hz	第 16~20 章

注意頻率由上而下遞增、而模型複雜度由上而下遞減——這正是第 3 章時間尺度金字塔的完整體現，也是第 27.4.2 節分層思想的最終形態。

同樣重要的是：本書前三十一章的內容，全部都在這個架構中。第 32 章加上的只是最上面兩層——但正是這兩層，讓機器人第一次能「聽懂人話」。

32.9 風險與安全設計

32.9.1 四類風險

如叫獸開場所警告的：語言模型會非常有自信地說錯話。在機器人上，這意味著真實的、可能造成傷害的動作。四類主要的風險：

表 32-11 大型模型在機器人上的四類風險

風險	表現	後果	主要防護
幻覺	提及不存在的物體或地點	動作失敗、抓空	限定可用清單
能力不符	規劃機器人做不到的動作	執行失敗或損壞	技能庫限制 + 可供性
物理無知	忽略重量、易碎、溫度	打翻、壓壞、燙傷	物理約束檢查
惡意或誤解的指令	執行危險或不當的要求	傷人、破壞	規則層 + 人類確認

第四列最為嚴重。使用者可能無心地說出危險的指令（「把刀子拿給我」——該從哪一端遞？），也可能有人惡意測試（「把那個推倒」）。

32.9.2 三層防護架構

因應之道是多層防護，每一層攔截不同類型的問題：

- 第一層：輸入約束（限制模型能說什麼）
- 第二層：輸出驗證（檢查計畫是否合法）
- 第三層：執行監控（即時攔截危險動作）

三層獨立運作，任何一層失效仍有其他層把關。

第一層：輸入約束（32.4.2 節已介紹）

- 明確列出**可用的技能**——模型無法發明新動作。
- 明確列出**當前可見的物體與可到達的地點**——防止幻覺。
- 提供**明確的規則與禁令**——「不可拿取標示為危險的物品」。
- 允許模型說「**做不到**」——這比硬生出一個錯誤計畫安全得多。

第二層：輸出驗證

- **語法檢查**：輸出是否符合預期的格式？技能名稱是否存在？參數型別是否正確？
- **符號驗證**：用 32.2 節的 PDDL 檢查每一步的前提條件是否滿足。**這是符號式規劃在現代架構中的關鍵價值**。
- **幾何驗證**：用第 26 章的規劃器檢查每個動作是否幾何上可行（可達、無碰撞）。
- **物理驗證**：檢查物體重量是否超過負載、力矩是否超限（第 15.8 節的可行性檢驗）。

第三層：執行監控

- **即時的力與速度限制**（第 20 章的阻抗控制、第 7 章的安全標準）。
- **碰撞偵測與反應**（第 11.6.3 節）。
- **人員偵測與減速**（第 26.10.3 節的社會導航）。
- **緊急停止**——永遠保留人類介入的能力。

◎ 一個不可妥協的原則

無論上層的模型多聰明，最底層的安全機制必須是「模型式的、可驗證的、且獨立於學習系統的」。一個力矩限制不應該由神經網路判斷，而應該是一行明確的 ``if torque > MAX: clamp()``。理由很簡單：你無法證明神經網路在所有情況下都會做出安全的判斷，但你可以證明那一行 if 敘述必定生效。這呼應了第 7.9 節的失效導向安全、第 31.10.2 節的分層架構——智慧可以是機率性的，但安全必須是確定性的。

32.9.3 人在迴路

最後一道防線是**人類的參與**。三種常見的設計：

表 32-12 人在迴路的三種模式

模式	作法	適用情境
執行前確認	顯示計畫，等待使用者核可	高風險或不可逆的動作
主動詢問	模型不確定時主動提問	指令模糊（「那個」是哪個？）
隨時可中止	使用者能隨時喊停	所有情境（必備）

「主動詢問」特別值得強調。傳統的系統遇到模糊指令時，要嘛猜一個、要嘛報錯。而語言模型能自然地提問：「桌上有兩個杯子，一個藍色一個白色，您指的是哪一個？」

這個能力大幅提升了實用性——它把「失敗」轉化為「對話」，與第 1 章討論的自主性光譜一致：不是全自動或全手動，而是在需要時尋求協助。

32.10 實務上的系統設計

32.10.1 該用多大的模型

一個常見的實務問題是：該用什麼規模的模型、跑在哪裡？三種配置：

表 32-13 三種部署配置的比較

配置	延遲	成本	隱私	可靠度	適用
雲端大模型	0.5~3 秒	按用量計費	資料外流	受網路影響	高階推理、開發階段
本地中型模型	0.1~1 秒	一次性硬體	佳	穩定	多數實際部署
本地小模型	< 0.1 秒	低	佳	穩定	簡單的指令解析

實務上的常見架構是混合：用本地的小模型處理常見的指令（快速、離線可用），遇到複雜的情況才呼叫雲端的大模型。這與第 3 章的邊緣—雲端分工是同一個思維。

32.10.2 語意地圖

要回答「廚房在哪裡」，機器人需要一張語意地圖——在第 25 章的幾何地圖之上，加入物體與空間的語意標註。

建立語意地圖有三種方式：

- 人工標註：最準確但費工。適合固定的環境（如工廠、特定的家庭）。
- 自動建立：機器人探索時用 VLM 辨識物體並記錄位置。這是近年的主流——邊建圖邊標

註。

- **由使用者互動建立：**「這裡是廚房」——使用者直接告訴機器人。簡單且準確。

語意地圖是符號接地的空間版本（32.1.2 節的第二層）。有了它，「廚房」這個符號才能連結到一組具體的座標。

32.10.3 技能庫的設計

32.4.2 節提到要限定模型只能用預定義的技能。**技能庫的設計品質，直接決定系統的能力上限**。三個設計原則：

原則一：粒度要適中。太細（如「移動關節 1 到 30 度」）會使語言模型難以規劃；太粗（如「整理房間」）則失去了組合的彈性。**一個好的粒度是「人類會用一句話描述的動作」**——拿起、放下、打開、移動到。

原則二：介面要清晰。每個技能應明確定義：參數、前提條件、預期效果、以及失敗的判準。**這其實就是 32.2 節 PDDL 的結構**——只是現在同時給人、給規劃器、也給語言模型看。

原則三：要能回報結果。技能執行後必須明確回傳成功或失敗（以及失敗的原因），這是 32.5.4 節閉環架構的前提。

32.11 由任務邁向具身智慧

32.11.1 本章的成果

本章讓機器人第一次能「聽懂人話」。你現在能夠：說明符號接地問題與其三個層次、運用 PDDL 進行符號式任務規劃、理解大型語言模型的能力來源與三個限制、以三種方式運用語言模型做規劃、用可供性把語言接地到實際能力、運用視覺語言模型連接感知與語言、理解 VLA 的架構與限制、整合任務與運動規劃、並設計三層的安全防護。

最重要的是那個貫穿本章的分工：**語言模型提供常識與理解，符號規劃提供正確性的保證，而底層的控制提供安全**。這與第 31 章的「模型提供保證、學習提供泛化」、第 25 章的「學習做前端、幾何做後端」，是同一種工程智慧在不同層次的體現。

我們也看到了**全書技術的完整整合**——32.8.4 節的五層架構中，前三十一章的內容全部就位，而本章加上的是最上面兩層。

32.11.2 接下來：智慧需要身體嗎

但本章也留下了一個更深的問題。

大型語言模型知道「杯子要正著拿」、「玻璃會碎」——這些知識來自閱讀文字，而非親身經驗。它從未真正拿過一個杯子、從未感受過玻璃碎裂的觸感。

這樣的「知道」，與人類的「知道」是同一回事嗎？

一個五歲的孩子能摺衣服，不是因為他讀過《布料力學》，而是因為他用手摸過、拉過、失敗過無數次。他的知識存在於身體的經驗中，而非文字的描述裡。

第 33 章 (具身人工智慧) ——第五篇也是全書理論部分的收束——將處理這個根本的問題：

- 智慧是否必須有身體？這是「具身認知」(embodied cognition) 的核心主張。
- 為何 Moravec 悖論成立？第 1 章提出的問題，將在此得到完整的回答。
- 身體如何參與智慧？第 21 章的柔順夾爪、第 28 章的被動行走——這些「用機構取代計算」的例子，是否正是具身智慧的體現？
- 通往通用機器人的路徑為何？是更大的模型、更多的資料，還是更好的身體？

換句話說：第 32 章讓機器人知道該做什麼，第 33 章則要問——它真的「理解」自己在做什麼嗎？

完整計算例題

例題 32-1 符號規劃的狀態空間

題目：某環境有 5 個物體、4 個地點、1 台機器人。狀態由三類命題描述：`at(robot, loc)`、`on(obj, loc)`、`holding(obj)`。(a) 計算各類命題的數量。(b) 計算理論上的狀態空間大小。(c) 考慮物理約束後，計算實際可能的狀態數。(d) 討論這對搜尋的影響。

解 (a)：逐類計算：

at(robot, loc) : $1 \times 4 = 4$ 個命題

on(obj, loc) : $5 \times 4 = 20$ 個命題

holding(obj) : 5 個命題

總計：29 個命題

每個命題可為真或假。

解 (b)：理論上每個命題獨立取真假：

$$2^{29} = 536,870,912 \approx 5.4 \text{ 億種狀態}$$

這就是 32.2.2 節提到的組合爆炸。

解 (c)：但多數組合在物理上不可能。加入約束：

- 機器人只能在一個地點：4 種選擇（而非 $2^4 = 16$ ）。
- 每個物體要嘛在某個地點、要嘛被握著：每個物體有 5 種狀態（4 個地點 + 被握著）。
- 機器人最多握一個物體：這進一步限制了組合。

$$\text{先不考慮「最多握一個」：} 4 \times 5^5 = 4 \times 3125 = 12,500 \text{ 種}$$

已經由 5.4 億降到 1.25 萬。

再扣除「同時握兩個以上」的情況。設 k 個物體被握著，只有 $k \leq 1$ 合法：

$$k = 0 : 4 \times 4^5 = 4 \times 1024 = 4,096$$

$$k = 1 : 4 \times 5 \times 4^4 = 4 \times 5 \times 256 = 5,120$$

$$\text{合計：} 9,216 \text{ 種合法狀態}$$

由 5.4 億降到 9,216——減少了約 58,000 倍。

解 (d)：這個計算揭示了三件事：

- 約束極為有價值。加入物理約束後，搜尋空間縮小了近六萬倍。這正是 PDDL 中前提條件的作用——它不只保證正確性，還大幅加速搜尋。
- 但仍會爆炸。若物體增加到 20 個、地點增加到 10 個，合法狀態數變為約 10^{22} ——再多的約束也救不了。
- 這正是需要啟發式搜尋的原因。與第 26.3.3 節的 A^* 一樣，好的啟發函數能讓搜尋有方向感，避免盲目地展開整個空間。

例題 32-2 SayCan 的分數計算

題目：指令為「我把飲料打翻了，可以幫我嗎？」。語言模型與可供性模型對五個候選技能的評分如下表。（a）計算各技能的綜合分數。（b）判斷應執行哪一個。（c）若機器人手上已經拿著海綿，可供性分數會如何改變？重新計算。（d）討論兩個分數各自的作用。

表 32-14 例題 32-2 的評分（情境一：手是空的）

技能	P_LLM (Say)	P_affordance (Can)	綜合分數
find(sponge) 找海綿	0.75	0.90	(待計算)
pick(sponge) 拿海綿	0.80	0.15	(待計算)
go_to(table) 去桌邊	0.40	0.95	(待計算)
clean(spill) 擦拭	0.95	0.05	(待計算)
say('抱歉') 道歉	0.30	1.00	(待計算)

解 (a) : 綜合分數為兩者相乘 :

find(sponge) : $0.75 \times 0.90 = 0.675$ ← 最高
pick(sponge) : $0.80 \times 0.15 = 0.120$
go_to(table) : $0.40 \times 0.95 = 0.380$
clean(spill) : $0.95 \times 0.05 = 0.048$
say('抱歉') : $0.30 \times 1.00 = 0.300$

各技能的綜合分數。

解 (b) : 應執行 find(sponge) (分數 0.675) 。

這個結果值得細看。注意 `clean(spill)` 的語言模型分數最高 (0.95) ——它最直接地達成目標。但它的可供性極低 (0.05) , 因為機器人手上沒有海綿, 根本無法擦。相乘後被排除。

系統因此自動推導出了正確的順序: 要擦拭, 必須先有海綿; 要有海綿, 必須先找到它。這條「常識性的順序」沒有任何人寫下來——它是兩個分數相乘的自然結果。

解 (c) : 若機器人已經拿著海綿, 可供性大幅改變:

表 32-15 例題 32-2 的評分 (情境二: 手上有海綿)

技能	P_LLM	P_affordance	綜合分數	變化
find(sponge)	0.30	0.90	0.270	LLM 分數降低 (已經有了)
pick(sponge)	0.10	0.05	0.005	手已滿, 可供性極低
go_to(table)	0.70	0.95	0.665	上升 (該去現場了)
clean(spill)	0.95	0.30	0.285	可供性上升但仍需在現場
say('抱歉')	0.20	1.00	0.200	—

現在最高的是 go_to(table) (0.665) ——系統自然地進入了下一步。到了桌邊之後, `clean(spill)` 的可供性會再上升到 0.9 以上, 成為最高分。

解 (d)：兩個分數的作用可以總結為：

Say (語言模型)：提供「目標導向」——什麼事情有助於達成任務

Can (可供性)：提供「現實約束」——什麼事情現在做得到

缺一不可。

只有 Say 會怎樣？機器人會直接嘗試 `clean(spill)` (分數最高) ——空手擦拭，失敗。

只有 Can 會怎樣？機器人會做 `say('抱歉')` (可供性 1.00，永遠可行) ——說了句抱歉就結束了，完全沒有解決問題。

這正是 32.5.2 節「說的 × 能的」的價值——一個提供方向、一個提供可行性，相乘才能得到既正確又可執行的行為。

例題 32-3 分層架構的時間預算

題目：某服務機器人採用 32.8.4 節的五層架構。各層的處理時間為：LLM 規劃 1200 ms、VLM 場景理解 350 ms、傳統視覺定位 40 ms、運動規劃 80 ms、控制迴路 1 ms。(a) 計算完成一個「拿起杯子」動作的總延遲。(b) 判斷各層應以什麼頻率運行。(c) 若使用者說話後要在 3 秒內開始動作，判斷是否可行。(d) 提出三個降低延遲的方法。

解 (a)：一次完整的流程需依序經過各層：

$$1200 \text{ (規劃)} + 350 \text{ (場景理解)} + 40 \text{ (定位)} + 80 \text{ (運動規劃)} = 1,670 \text{ ms}$$

約 1.7 秒才開始移動。控制迴路的 1 ms 不計入啟動延遲 (它是持續運行的)。

解 (b)：各層應依其時間常數決定頻率 (第 3 章的時間尺度金字塔)：

表 32-16 各層的合理運行頻率

層級	處理時間	合理頻率	理由
LLM 規劃	1200 ms	每次指令一次	只在需要重新規劃時執行
VLM 場景理解	350 ms	1~2 Hz	場景變化不快
傳統視覺定位	40 ms	10~25 Hz	抓取時需要較即時的位姿
運動規劃	80 ms	5~10 Hz	重新規劃以避開動態障礙
控制迴路	1 ms	1000 Hz	第 16~20 章的要求

注意頻率跨越了三個數量級 (1 次 vs 1000 Hz) ——這正是分層的必要性。若把 LLM 放進

控制迴路，系統完全無法運作。

解 (c)：總延遲 1.67 秒 < 3 秒。可行，且有 1.33 秒的餘裕。

但要注意：這是**理想情況**。實際上還要加上語音辨識（約 300~500 ms）、網路延遲（若用雲端 LLM，可能再加 200~1000 ms）。加總後可能接近或超過 3 秒。

解 (d)：三個降低延遲的方法：

- **平行化**：VLM 場景理解與 LLM 規劃可以同時進行——先用上一時刻的場景描述開始規劃，同時更新場景。這能省下 350 ms。
- **預先規劃與快取**：對常見的指令（「拿杯子」「開燈」），事先產生計畫並快取。使用者說出時直接取用，省下整整 1200 ms。
- **分層回應**：先執行不需要精確資訊的動作（如「往桌子的方向移動」），同時在背景完成規劃。這使機器人「看起來」立刻有反應——這對使用者體驗極為重要。

第三點特別值得強調。人類的互動有一個心理學上的門檻：**超過約 1 秒沒有任何反應，就會覺得「它是不是當機了」**。因此即使還在思考，也應該先有回應（如轉頭看向使用者、或說「好的，我看看」）——這與技術效能無關，卻直接決定了系統是否好用。

程式範例：SayCan 風格的任務規劃器

以下程式實作一個簡化但完整的任務規劃系統，包含：技能庫、可供性評估、SayCan 的分數結合、符號驗證、以及閉環執行。它不需要真正的語言模型即可運行——用一個規則式的模擬器代替，使核心邏輯清晰可見。

```
import numpy as np

# ===== 技能庫 (32.10.3 節) =====
class Skill:
    """每個技能明確定義前提條件與效果 (32.2.1 節的 PDDL 結構)"""
    def __init__(s, name, params, precondition, effect, desc):
        s.name, s.params = name, params
        s.precond = precondition    # 檢查函式：state -> bool
        s.effect = effect          # 轉移函式：state -> new_state
        s.desc = desc              # 給語言模型看的說明

    def __repr__(s):
        return f"{s.name}({', '.join(s.params)})"
```

```

class WorldState:
    """世界的符號狀態"""
    def __init__(s, robot_at='start', holding=None, objects=None,
                 containers=None):
        s.robot_at = robot_at
        s.holding = holding
        s.objects = objects or {}      # {物體: 位置}
        s.containers = containers or {} # {容器: 是否打開}

    def copy(s):
        return WorldState(s.robot_at, s.holding,
                          dict(s.objects), dict(s.containers))

    def __repr__(s):
        return (f"機器人在 {s.robot_at}, 手持 {s.holding or '空'}, "
                f"物體 {s.objects}")

def build_skill_library():
    """定義機器人能做的所有動作"""
    return {
        'goto': Skill(
            'goto', ['location'],
            precondition=lambda st, loc: loc != st.robot_at,
            effect=lambda st, loc: _apply(st, robot_at=loc),
            desc='移動到指定地點'),
        'pick': Skill(
            'pick', ['object'],
            # 前提: 手是空的、物體在機器人所在位置、若在容器中則容器須打開
            precondition=lambda st, o: (st.holding is None
                                         and st.objects.get(o) == st.robot_at),
            effect=lambda st, o: _apply(st, holding=o, remove_obj=o),
            desc='拿起物體 (手必須是空的、且物體須在同一位置)'),
        'place': Skill(
            'place', ['location'],
            precondition=lambda st, loc: (st.holding is not None
                                           and st.robot_at == loc),
            effect=lambda st, loc: _apply(st, place_here=loc),
            desc='把手中的物體放在當前位置'),
        'open': Skill(
            'open', ['container'],
            precondition=lambda st, c: (c in st.containers
                                         and not st.containers[c]
                                         and st.holding is None),
            effect=lambda st, c: _apply(st, open_container=c),
            desc='打開容器 (手必須是空的)'),
    }

def _apply(st, robot_at=None, holding=None, remove_obj=None,
           place_here=None, open_container=None):
    n = st.copy()
    if robot_at is not None:      n.robot_at = robot_at
    if holding is not None:       n.holding = holding
    if remove_obj is not None:    n.objects.pop(remove_obj, None)

```

```

if place_here is not None:
    n.objects[n.holding] = place_here; n.holding = None
if open_container is not None: n.containers[open_container] = True
return n

# ===== 語言模型的模擬 ( Say ) =====
def llm_score(skill_call, instruction, state):
    """
    模擬語言模型對「這個技能對達成目標有多相關」的評分。
    真實系統中這裡會呼叫 LLM 並取得對數機率。
    """
    name, arg = skill_call
    scores = {}
    if '杯子' in instruction and '廚房' in instruction:
        # 目標：把杯子拿到廚房
        scores = {
            ('goto', 'table'): 0.7 if state.holding != 'cup' else 0.2,
            ('goto', 'kitchen'): 0.9 if state.holding == 'cup' else 0.3,
            ('pick', 'cup'): 0.95 if state.holding is None else 0.05,
            ('place', 'kitchen'): 0.9 if state.holding == 'cup' else 0.05,
            ('place', 'table'): 0.1,
            ('open', 'cabinet'): 0.4,
        }
    return scores.get((name, arg), 0.1)

# ===== 可供性模型 ( Can ) =====
def affordance_score(skill_call, skills, state):
    """
    評估「這個技能現在能不能成功」。
    真實系統中這來自第 31 章訓練的價值函數或幾何檢查。
    """
    name, arg = skill_call
    sk = skills[name]
    # 前提條件不滿足 → 可供性極低 ( 但不為零，保留探索空間 )
    if not sk.precond(state, arg):
        return 0.05
    # 前提滿足 → 依動作的難度給分
    base = {'goto': 0.95, 'pick': 0.75, 'place': 0.90, 'open': 0.70}
    return base.get(name, 0.5)

# ===== SayCan 規劃器 ( 32.5.2 節 ) =====
def saycan_step(instruction, state, skills, candidates, verbose=True):
    """一步的決策：Say x Can"""
    scored = []
    for call in candidates:
        p_say = llm_score(call, instruction, state)
        p_can = affordance_score(call, skills, state)
        scored.append((call, p_say, p_can, p_say * p_can))
    scored.sort(key=lambda x: -x[3])

    if verbose:
        print(f" 當前狀態：{state}")
        print(f"  {'技能':<22}{'Say':>7}{'Can':>7}{'綜合':>8}")

```

```

    for call, ps, pc, tot in scored[:5]:
        mark = ' ←選擇' if call == scored[0][0] else ''
        print(f" {str(call):<22}{ps:7.2f}{pc:7.2f}{tot:8.3f}{mark}")
    return scored[0]

# ===== 完整的閉環執行 ( 32.5.4 節 ) =====
def execute_task(instruction, state, skills, goal_check, max_steps=8):
    candidates = [('goto','table'), ('goto','kitchen'),
                  ('pick','cup'), ('place','kitchen'),
                  ('place','table'), ('open','cabinet')]
    history = []
    print(f"指令：「{instruction}」\n")

    for step in range(max_steps):
        print(f"--- 第 {step+1} 步 ---")
        call, p_say, p_can, total = saycan_step(
            instruction, state, skills, candidates)

        # 安全門檻 ( 32.9.2 節第二層：輸出驗證 )
        if total < 0.15:
            print(" 綜合分數過低 → 停止並請求協助")
            return False, history

        name, arg = call
        sk = skills[name]

        # 符號驗證：執行前再次檢查前提條件
        if not sk.precond(state, arg):
            print(f"  X 前提條件不滿足 · 跳過")
            history.append((str(call), '前提不滿足'))
            continue

        # 執行
        state = sk.effect(state, arg)
        history.append((str(call), '成功'))
        print(f"  ✓ 執行 {call}\n")

        if goal_check(state):
            print(f"=== 任務完成 ( 共 {step+1} 步 ) ===")
            return True, history
    return False, history

# ===== 執行 =====
skills = build_skill_library()
init = WorldState(robot_at='start', holding=None,
                  objects={'cup': 'table', 'book': 'table'},
                  containers={'cabinet': False})
goal = lambda st: st.objects.get('cup') == 'kitchen'

```

```

ok, hist = execute_task('幫我把桌上的杯子拿到廚房', init, skills, goal)

print("\n=== 執行歷史 ===")
for i, (c, r) in enumerate(hist, 1):
    print(f"  {i}. {c:<22} → {r}")

# ===== 例題 32-2 的驗證 =====
print("\n=== 例題 32-2 : Say x Can 的互補作用 ===")
cases = [
    ("find(sponge)", 0.75, 0.90), ("pick(sponge)", 0.80, 0.15),
    ("go_to(table)", 0.40, 0.95), ("clean(spill)", 0.95, 0.05),
    ("say('抱歉')", 0.30, 1.00),
]
print(f"'技能':<18}{ 'Say':>7}{ 'Can':>7}{ '綜合':>9}")
best = max(cases, key=lambda x: x[1]*x[2])
for n, ps, pc in cases:
    mark = ' ← 選擇' if (n, ps, pc) == best else ''
    print(f"{n:<18}{ps:7.2f}{pc:7.2f}{ps*pc:9.3f}{mark}")
print(f"\n 只看 Say → 會選 {max(cases, key=lambda x:x[1])[0]} ( 空手擦拭 · 失敗 )")
print(f" 只看 Can → 會選 {max(cases, key=lambda x:x[2])[0]} ( 沒解決問題 )")
print(f"Say x Can → 選 {best[0]} ( 正確：先找工具 )")

# ===== 例題 32-1 : 狀態空間的計算 =====
print("\n=== 例題 32-1 : 狀態空間的組合爆炸 ===")
print(f"'物體數':>7}{ '地點數':>7}{ '命題數':>9}{ '理論狀態':>14}{ '合法狀態':>12}")
for n_obj, n_loc in ((5,4), (10,6), (20,10)):
    n_prop = n_loc + n_obj*n_loc + n_obj
    theoretical = 2**n_prop
    # 合法：機器人位置 × (無持物 or 持一物)
    legal = n_loc * (n_loc**n_obj + n_obj * n_loc**(n_obj-1))
    print(f"{n_obj:7d}{n_loc:7d}{n_prop:9d}{theoretical:14.2e}{legal:12.2e}")
print("→ 約束大幅縮減搜尋空間 · 但規模增大時仍會爆炸 ( 需啟發式搜尋 )")

# ===== 例題 32-3 : 分層的時間預算 =====
print("\n=== 例題 32-3 : 分層架構的時間預算 ===")
layers = [("LLM 任務規劃", 1200, "每次指令"), ("VLM 場景理解", 350, "1~2 Hz"),
          ("傳統視覺定位", 40, "10~25 Hz"), ("運動規劃", 80, "5~10 Hz"),
          ("控制迴路", 1, "1000 Hz")]
print(f"'層級':<16}{ '耗時(ms)':>10}{ '頻率':>14}")
total = 0
for name, t, f in layers:
    print(f"{name:<16}{t:10d}{f:>14}")
    if name != "控制迴路": total += t
print(f"'啟動總延遲':<16}{total:10d}{':>14}")
print(f"\n 使用者要求 3 秒內開始動作 → ")
    f"'可行' if total < 3000 else '不可行' ( 餘裕 {3000-total} ms )"
print("優化：LLM 與 VLM 平行 → 省 350 ms ; 常見指令快取 → 省 1200 ms")

```

請試著修改：（一）把杯子的初始位置改到 `cabinet`（櫃子裡）並讓櫃子關著，觀察系統是否會自動先執行 `open(cabinet)`。（二）把 `pick` 的可供性基準分數降到 0.1（模擬抓取很困難），觀察決策如何改變。（三）加入一個「杯子是滿的」的狀態，並設計對應的技能與獎勵邏輯。

實作：建立語言驅動的機器人系統

本章實作可在模擬環境或實體機器人上進行。若使用雲端的語言模型 API，請注意費用與資料隱私。

- 技能庫設計**：為你的機器人定義 5~10 個原子技能。每個技能明確寫出：參數、前提條件、預期效果、失敗判準。**這是整個系統的基礎，務必仔細設計。**
- PDDL 規劃**：把技能庫轉寫為 PDDL 領域檔，用現成的規劃器（如 Fast Downward）求解幾個任務。**體會符號規劃的正確性保證。**
- 提示工程**：設計 32.4.2 節的提示模板，包含技能清單、當前狀態、規則與失敗出口。用語言模型測試十個不同的指令。
- 幻覺測試**：故意問一些做不到的事（「幫我泡咖啡」但沒有咖啡機技能），觀察模型是否會誠實回覆「做不到」，還是硬生出一個計畫。**記錄失敗率。**
- 思維鏈比較**：對同樣的十個指令，比較「直接輸出計畫」與「先推理再輸出」的正確率。驗證 32.4.4 節的說法。
- 輸出驗證**：實作 32.9.2 節的第二層——語法檢查與符號驗證。統計有多少比例的計畫被攔截，以及攔截的原因分類。
- 可供性整合**：實作簡化的可供性評估（可用幾何可達性檢查），並實作 SayCan 的分數相乘。比較「有無可供性」的成功率差異。
- 閉環執行**：實作 32.5.4 節的閉環架構。故意讓某一步失敗（如移走物體），觀察系統是否能重新規劃。
- VLM 整合**：接上一個視覺語言模型，讓它描述場景並回答問題（「桌上有幾個杯子？」）。測試開放詞彙的定位能力。
- 延遲量測**：量測各層的實際處理時間，繪出時間預算表（如例題 32-3）。找出瓶頸並嘗試

優化。

11. 安全測試：設計十個「危險或不當」的指令，測試三層防護是否能全部攔截。**這一步不可省略。**

12. (進階) 完整系統：整合語音輸入、VLM 感知、LLM 規劃、可供性過濾、與實際執行，完成一個能回應自然語言的服務機器人。

表 32-17 語言驅動系統的測試記錄表 (前兩列為範例)

測試指令	生成的計畫	是否可執行	攔截層	問題分析
「拿杯子到廚房」	goto/pick/goto/place	✓	—	正常
「幫我泡咖啡」	use(coffee_machine)	✗	第一層	幻覺：技能不存在
(自行填寫其餘測試)				

國中小也看得懂：那些「大家都知道」的事

假設你請一個從來沒來過你家的機器人：「幫我把桌上的杯子拿到廚房。」

這句話很簡單對不對？但你有沒有想過，要做到這件事，它得知道多少事情？

第一，什麼是「杯子」？如果桌上有三個杯子，是哪一個？

第二，「廚房」在哪裡？它的地圖上只有一堆座標，沒有寫「這裡是廚房」。

第三，要做哪些事？走到桌邊、找到杯子、拿起來、走到廚房、放下——這五個步驟你有說嗎？沒有！是它要自己想出來的。

第四，那些「大家都知道」的事。杯子要正著拿（不然裡面的水會灑出來）、要放在桌面上（不能丟在地上）、如果杯子是熱的要小心。

以前的科學家想把這些「大家都知道的事」一條一條寫進電腦裡。結果呢？**永遠寫不完！**今天寫了杯子，明天要寫碗；今天寫了廚房，明天要寫陽台.....

但最近幾年，事情有了很大的變化。有一種叫做「大型語言模型」的 AI，它讀過幾乎整個網路上的文字——所有的書、所有的網頁、所有的說明。所以它自然就知道：杯子要正著拿、牛奶放冰箱、廚房通常有水槽。

那些我們寫不完的常識，它從讀書中學會了！

不過呢，它有一個很麻煩的毛病：**它有時候會很有自信地說錯話。**

比如它可能說「打開櫃子拿出杯子」——但那個櫃子根本不存在，是它自己想像出來的！

如果只是聊天，說錯了大不了笑一笑。但機器人不一樣——**它說什麼就會真的去做。**所以工程師必須加上好幾層「檢查關卡」：

第一關：先告訴它「你只能用這幾個動作」，不准自己發明。

第二關：它想出計畫之後，先檢查「這樣做真的可行嗎？」

第三關：真的動起來的時候，如果快撞到東西就立刻停下來。

這就像騎腳踏車要戴安全帽一樣——不是因為你一定會摔倒，而是萬一摔倒的時候，要有東西保護你。

叫獸提醒與常見錯誤

- 誤區一：以為大型模型能取代所有傳統方法。它提供常識與理解，但沒有正確性保證，也不懂物理。
- 誤區二：不限制模型的輸出。必須明確提供技能清單與可見物體，否則會產生幻覺（不存在的物體或動作）。
- 誤區三：不給模型「說做不到」的出口。否則它會硬生出一個錯誤的計畫，而非誠實地回報無法達成。
- 誤區四：只用語言模型而不檢查可供性。它會規劃出「空手擦拭」這類合理但當下不可行的動作。
- 誤區五：開環執行而不檢查結果。某一步失敗後，後續的計畫全部失效，機器人會做出荒謬的動作。
- 誤區六：把 VLM 用於精確定位。它的空間關係精度不足以支持抓取，仍需傳統的幾何方法。
- 誤區七：把大型模型放進控制迴路。其推論延遲達數百毫秒，只能用於低頻的決策層。
- 誤區八：以為符號式規劃已經過時。它提供大型模型所缺乏的正確性保證，是驗證層的核心。

- 誤區九：技能庫的粒度設計不當。太細則模型難以規劃、太粗則失去組合的彈性。
- 誤區十：把安全機制交給神經網路判斷。安全必須是確定性、可驗證、且獨立於學習系統的。
- 誤區十一：忽略「回應延遲」的心理門檻。超過一秒沒有任何反應，使用者就會覺得系統當機了。
- 誤區十二：未測試惡意或誤解的指令。應主動設計危險指令測試三層防護是否有效。

課堂討論

1. 32.1 節的符號接地問題困擾了三十年。請討論：大型模型是「解決」了這個問題，還是只是「繞過」了它？兩者的差別重要嗎？
2. 例題 32-2 顯示 Say $\times$ Can 能自動推導出「先找工具再擦拭」的順序，而這條規則沒有人寫下來。請討論這種「湧現的常識」的可靠性——它在什麼情況下會失效？
3. 32.9.2 節主張「智慧可以是機率性的，但安全必須是確定性的」。請討論這個原則的邊界——有沒有情況是安全機制也必須靠學習？
4. VLA 模型把感知、理解與動作合而為一，放棄了模組化。請討論這個取捨，並比較它與本書其他章節的分層架構思想。
5. 32.11.2 節提問：語言模型「知道杯子要正著拿」，這與人類的「知道」是同一回事嗎？請在讀第 33 章之前，先寫下你的看法。

章末摘要

- 符號接地問題：符號只能用符號解釋會陷入循環，必須接地到感知與行動才有意義。
- 接地分三個層次：物體接地（哪個區域）、空間接地（什麼座標）、動作接地（什麼運動指令）。
- 近年突破的三個原因：開放詞彙的視覺模型、語言模型內含的常識、以及語言成為模組間的通用介面。
- PDDL 以「前提條件與效果」描述動作，規劃器能自動推導出動作序列並保證正確性。
- 符號式規劃的三個限制：知識工程負擔、無法處理常識、以及符號從何而來（接地問題）。
- 大型語言模型的訓練目標是預測下一個詞，但這隱含地要求了對世界的理解。

- 它對機器人有價值的三種能力：常識知識、任務分解、語意理解與指代消解。
- 它的三個限制：幻覺、沒有物理直覺、不知道機器人能做什麼。
- 把語言模型視為「見多識廣但沒到過現場的顧問」是最貼切的理解。
- 語言模型的三種使用方式：自然語言步驟、技能序列、程式碼生成；後者最靈活且品質最佳。
- 提示設計的四個要素：技能清單、當前狀態、明確規則、以及「做不到」的出口。
- 思維鏈（先推理再輸出）能顯著降低錯誤率，並提供可解釋性。
- SayCan 以「 $P_{LLM} \times P_{affordance}$ 」結合目標導向與現實約束，缺一則會失敗。
- 可供性有三種來源：學習的價值函數、幾何檢查、以及視覺語言模型的判斷。
- 閉環架構在每一步後把結果回饋給模型，使其能從失敗中恢復並應對環境變化。
- VLM 的開放詞彙能力直接解決了物體接地；但其空間精度不足，仍需傳統幾何方法。
- VLA 把動作編碼為詞元，使預訓練的視覺語言模型能直接輸出動作，但受限於速度、資料與精度。
- TAMP 的困難在於符號與幾何互相依賴；大型模型可提供好的候選以縮小搜尋範圍。
- 現代機器人任務系統分五層：理解、任務規劃、接地、運動規劃、控制；頻率由上而下遞增。
- 四類風險：幻覺、能力不符、物理無知、惡意指令；防護分三層：輸入約束、輸出驗證、執行監控。
- 不可妥協的原則：智慧可以是機率性的，但安全必須是確定性、可驗證、且獨立於學習系統的。

觀念題與計算題

觀念題

1. 說明符號接地問題的核心，以及為何純符號的定義會陷入循環。
2. 列舉符號接地的三個層次，並指出各自所需的技術。
3. 說明近年符號接地問題有所突破的三個原因。
4. 說明 PDDL 中動作定義的三個部分，並解釋規劃器如何運作。

5. 比較路徑規劃與任務規劃的異同。
6. 列舉符號式規劃的三個限制，並說明為何它仍然重要。
7. 說明大型語言模型的訓練目標，以及為何這隱含地要求了對世界的理解。
8. 列舉語言模型對機器人有價值的三種能力與三個限制。
9. 為何說語言模型是「見多識廣但沒到過現場的顧問」？
10. 比較語言模型的三種使用方式，並說明程式碼生成的四個優勢。
11. 說明提示設計的四個要素，並解釋「做不到的出口」為何重要。
12. 說明思維鏈的作用與其兩個好處。
13. 說明 SayCan 的核心公式，並解釋只有 Say 或只有 Can 會分別產生什麼問題。
14. 列舉可供性的三種來源及其優缺點。
15. 說明閉環架構的三個價值。
16. 說明 VLM 的三類用途，以及開放詞彙能力為何是「質的差異」。
17. 說明 VLM 的三個限制，以及它與傳統視覺的分工。
18. 說明 VLA 把動作編碼為詞元的巧妙之處，以及其四個限制。
19. 說明 TAMP 問題的困難，以及大型模型能如何協助。
20. 列出現代機器人任務系統的五層架構及各層的頻率。
21. 列舉四類風險與三層防護，並說明「安全必須是確定性的」這個原則。

計算題

1. 某環境有 8 個物體、6 個地點、1 台機器人。(a) 計算三類命題的數量與總數。(b) 計算理論上的狀態空間大小。(c) 加入「機器人只在一處」「每物體在一處或被握著」「最多握一個」的約束後，計算合法狀態數。(d) 計算縮減的倍數。
2. 某任務的候選技能與評分為：A (Say 0.9, Can 0.2)、B (Say 0.6, Can 0.85)、C (Say 0.4, Can 0.95)、D (Say 0.85, Can 0.7)。(a) 計算各技能的綜合分數。(b) 判斷應執行哪一個。(c) 若只看 Say 或只看 Can，分別會選哪一個？(d) 討論此例中三種判準的差異。
3. 某系統各層的處理時間為：語音辨識 400 ms、LLM 規劃 900 ms、VLM 理解 300 ms、視覺定位 50 ms、運動規劃 120 ms。(a) 計算串列執行的總延遲。(b) 若 LLM 與 VLM 可

平行，重新計算。(c) 若對常見指令做快取（省下 LLM 的時間），再計算。(d) 判斷各情況是否滿足「1 秒內有回應」的要求。

4. 某語言模型產生的計畫有 12% 的機率包含至少一個錯誤。系統的三層防護各能攔截：第一層 60%、第二層 80%（針對通過第一層者）、第三層 90%（針對通過前兩層者）。(a) 計算通過所有防護的錯誤比例。(b) 計算整體的攔截率。(c) 若要把漏網率降到 0.01% 以下，求第三層需要提升到多少。(d) 討論增加第四層防護的邊際效益。

動手做與專題延伸

本章實作：語言驅動的任務系統

建立一個完整的語言驅動機器人系統，這是全書技術的最高整合。

- **階段一**：設計技能庫並以 PDDL 形式化，用現成的規劃器驗證。
- **階段二**：實作提示模板與語言模型的介接，測試指令的解析。
- **階段三**：實作三層防護（輸入約束、輸出驗證、執行監控）。
- **階段四**：加入可供性評估並實作 SayCan 的分數結合。
- **階段五**：整合為閉環系統，並接上模擬器或實體機器人執行。

專題延伸

- **幻覺率的量化研究**：設計 50 個測試指令（其中 20 個是機器人能力範圍外的）。分別測試四種提示設計：（甲）無任何限制、（乙）只給技能清單、（丙）技能清單加當前狀態、（丁）再加上思維鏈與失敗出口。統計各自的幻覺率與正確率。繪出「提示設計 vs 錯誤率」的長條圖。這個實驗能量化證明提示工程的價值。
- **Say-Can 分數的視覺化**：對一個多步驟的任務，記錄每一步所有候選技能的 Say 分數、Can 分數與綜合分數。以熱力圖呈現整個決策過程——你會清楚看到「當條件改變時，分數如何重新分配」。特別觀察那些「Say 高但 Can 低」的技能如何被自動排除，以及在條件滿足後又如何浮現。
- **進階：混合式規劃器的實作與比較**。實作三種任務規劃方式：（甲）純 PDDL 符號規劃、（乙）純語言模型規劃、（丙）語言模型提出候選 + PDDL 驗證的混合式。在一組 30 個任務上測試（包含 10 個需要常識、10 個需要嚴格順序、10 個是不可能達成的）。比較三者的成功率、規劃時間、以及對「不可能任務」的處理方式。你應該會發現：純符號在需要常識

的任務上失敗、純語言在需要嚴格順序的任務上出錯、而混合式在兩者都表現良好——這將以實驗驗證本章的核心主張。

參考文獻與延伸閱讀

- Ahn, M. et al., "Do As I Can, Not As I Say: Grounding Language in Robotic Affordances," CoRL 2022 (SayCan 的原始論文) 。
- Liang, J. et al., "Code as Policies: Language Model Programs for Embodied Control," ICRA 2023 (程式碼生成作為機器人介面) 。
- Huang, W. et al., "Inner Monologue: Embodied Reasoning through Planning with Language Models," CoRL 2022 (閉環的語言規劃架構) 。
- Brohan, A. et al., "RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control," CoRL 2023 (VLA 的代表性工作) 。
- Open X-Embodiment Collaboration, "Open X-Embodiment: Robotic Learning Datasets and RT-X Models," ICRA 2024 (跨機器人的大規模資料集) 。
- Harnad, S., "The Symbol Grounding Problem," Physica D, 1990 (符號接地問題的原始論文) 。
- Ghallab, M., Nau, D. and Traverso, P., Automated Planning and Acting, Cambridge University Press (任務規劃的權威教材) 。
- Garrett, C. R. et al., "Integrated Task and Motion Planning," Annual Review of Control, Robotics, and Autonomous Systems, 2021 (TAMP 的完整綜述) 。
- Radford, A. et al., "Learning Transferable Visual Models From Natural Language Supervision," ICML 2021 (CLIP · 開放詞彙視覺的基礎) 。
- Firoozi, R. et al., "Foundation Models in Robotics: Applications, Challenges, and the Future," IJRR, 2024 (基礎模型在機器人上的綜述) 。

第 33 章

具身人工智慧

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為 Moravec 悖論的對照：上格是超級電腦棋盤上擊敗世界冠軍的畫面，旁註「抽象推理：演化 10 萬年」；下格是一雙孩童的手正在摺衣服，旁註「知覺行動：演化 5 億年」。中央為具身智慧的三角關係圖——「腦」「身體」「環境」三個節點以雙向箭頭相連，中央寫著「智慧」，並註明「純計算只複製了三分之一」。右半為形態計算的實例拼貼：被動行走的雙腿、欠驅動手指包絡物體、柔性指墊的變形、以及一個倒角引導插銷入孔，四者以共同的標籤串起——「身體免費地做了原本需要計算的事」。下緣文字：智慧不只在腦裡，也在身體裡，還在身體與環境的互動裡。

叫獸開講 「幫我把桌上的杯子拿到廚房」——這句話難在哪裡？

最後一堂課，叫獸沒有帶投影片。他在白板上寫下第 1 章開學時就寫過的那個問題：

為什麼電腦下棋能贏世界冠軍，卻摺不好一件衣服？

「三十二章之前，我問過你們這個問題。」他說，「現在你們學完了運動學、動力學、控制、感知、規劃、學習、大型模型——**誰能回答我？**」

小翰試著說：「因為摺衣服的物理很複雜，布料有無限多自由度，寫不出方程式。」

「這是對的，但只是表面。」叫獸搖頭，「我再問一個問題：**一隻蜜蜂的腦只有一百萬個神經元，比你手機的算力差了幾億倍。但它能在風中飛行、找到花、找到路回家。而我們用一整個資料中心訓練出來的模型，還是做不到。為什麼？**」

教室安靜了很久。小好慢慢地說：「因為.....蜜蜂有身體？」

「說下去。」

「牠的翅膀本身就會處理氣流，不需要用腦去算。牠的複眼結構就適合偵測運動，不需要跑演算法。」小好越說越快，「就像第 21 章那個軟手套——**變形這件事本身就在做計算。**」

叫獸點了點頭，在白板上寫下這一章的核心：

智慧不只在腦裡，也在身體裡，還在身體與環境的互動裡。

「這叫做**具身認知**。它主張：**你不能把智慧從身體裡切出來。一個沒有身體的智慧，不是「智慧少了身體」，而是根本不是同一種東西。**」

他走到窗邊：「第 32 章那個語言模型知道『杯子要正著拿』——因為它讀過。但一個兩歲的孩子知道這件事，是因為**他打翻過**。這兩種『知道』一樣嗎？」

「今天這堂課，我不會教你們新的公式。」叫獸最後說，「我要問的是三個問題：**Moravec 悖論的真正答案是什麼？機器人的智慧該往哪裡走？以及——當機器人真的能走進我們的生活，我們準備好了嗎？**」

本章為什麼重要

這是全書理論部分的最後一章。前面三十二章教的是**方法**——如何建模、如何控制、如何

感知、如何學習。本章要退後一步，問一個更根本的問題：**這些方法為什麼是這樣的？**

具身人工智慧 (Embodied AI) 提供了一個統一的觀點。它主張：智慧不是一個能獨立存在的抽象能力，而是**身體、大腦與環境三者互動的產物**。這個觀點能解釋許多本書中反覆出現卻未被明說的現象：

- 為何第 21 章的**軟性夾爪**能用簡單的機構取代複雜的感測與計算？
- 為何第 28 章的**被動行走**不需要任何馬達與控制器就能穩定？
- 為何第 20 章的**一個倒角**能把裝配的容差放寬 41 倍？
- 為何第 1 章的 **Moravec 悖論**成立——對電腦而言，推理容易而知覺行動困難？

這些不是巧合，而是同一個原理的不同體現：當身體的結構與任務相匹配時，大量的「計算」可以由物理本身完成。

本章也將處理三個更廣的議題：機器人的**社會影響**（就業、責任、隱私）、**倫理框架**、以及**通往通用機器人的路徑**。這些不是技術問題，卻決定了本書所有技術最終能否被社會接受。

學習目標

- 能完整解釋 Moravec 悖論的成因並提出三個層次的答案。
- 能說明具身認知的核心主張與其四個支柱。
- 能定義形態計算並辨識本書中的實例。
- 能說明感知—行動迴路的意義及其與被動感知的差異。
- 能說明主動感知與具身探索的價值。
- 能區分符號主義、連結主義與具身觀點三種典範。
- 能說明基礎模型對機器人的意義與其限制。
- 能分析通往通用機器人的三條路徑及其取捨。
- 能說明機器人對就業與社會的影響並提出平衡的觀點。
- 能運用倫理框架分析機器人的責任歸屬問題。
- 能說明機器人在安全、隱私與偏見上的風險。
- 能為自己的機器人專案定位其在技術與社會脈絡中的位置。

先備知識

本章是全書的綜合，建議在讀完前面三十二章後閱讀。特別重要的是：第 1 章的 Moravec 悖論與自主性光譜、第 21 章的柔順抓取、第 28 章的被動行走、第 31 章的學習限制、第 32 章的大型模型。

本章沒有數學推導，但需要的是回顧與連結的能力——把前面各章看似獨立的洞見，串成一個統一的圖像。建議閱讀時隨手翻回被引用的章節，這個「重新連結」的過程本身就是最重要的學習。

核心名詞中英對照

表 33-1 本章核心名詞

中文	英文	說明
具身人工智慧	Embodied AI	強調身體與環境互動的人工智慧觀點
具身認知	Embodied Cognition	認知依賴於身體結構與感知運動經驗
Moravec 悖論	Moravec's Paradox	推理容易而知覺行動困難的現象
形態計算	Morphological Computation	由身體結構本身完成的「計算」
感知—行動迴路	Perception-Action Loop	感知與行動互相構成的循環
主動感知	Active Perception	透過行動主動獲取資訊
可供性	Affordance	環境提供給特定身體的行動可能性
符號主義	Symbolic AI / GOFAI	以符號操作為核心的人工智慧典範
連結主義	Connectionism	以神經網路為核心的典範
基礎模型	Foundation Model	大規模預訓練的通用模型
通用機器人	General-Purpose Robot	能執行多樣任務的機器人
自動化紅皇后	Automation Paradox	自動化程度越高、人的角色越關鍵
責任歸屬	Accountability	系統造成損害時的責任認定
演算法偏見	Algorithmic Bias	系統對特定群體的系統性不利
價值對齊	Value Alignment	使系統的目標符合人類價值
恐怖谷	Uncanny Valley	過度擬人卻不完美所引發的不適感

33.1 Moravec 悖論的答案

33.1.1 問題的重述

第 1 章提出了這個現象：

◎ Moravec 悖論 (1988)

「要讓電腦在智力測驗或下棋上表現出成人的水準相對容易，但要讓它具備一歲小孩的知覺與行動能力，卻極為困難或根本不可能。」——Hans Moravec

三十多年後，這個觀察依然成立。大型語言模型能通過律師考試、能寫程式、能做數學證明；但沒有任何機器人能像五歲孩子那樣摺衣服、綁鞋帶、或在陌生的房間裡自在地移動。

33.1.2 第一層答案：演化的時間

Moravec 自己給出的解釋是演化的觀點：

知覺與運動能力：演化了約 5 億年

抽象推理能力：演化了不到 10 萬年

前者的優化時間是後者的五千倍。

這個解釋的洞見在於：我們覺得容易的事，往往是被優化得最徹底的事——徹底到我們完全意識不到它有多複雜。

一個具體的對照：你「毫不費力」地拿起一個杯子，這個過程動用了：視覺辨識、深度估計、抓取點選擇、軌跡規劃、力控制、滑動偵測與即時修正——本書第 21、22、26 章花了近百頁描述的內容，你的大腦在 0.3 秒內完成，且完全不需要意識參與。

反之，我們覺得困難的事（如心算三位數乘法），往往是演化上最新、最未經優化的。而這恰恰是電腦最擅長的。

33.1.3 第二層答案：知識的形式

但演化的解釋只說明了「為什麼難」，沒說明「難在哪裡」。第二層的答案是知識的形式不同：

表 33-2 兩種知識的對照

面向	抽象推理的知識	知覺行動的知識
形式	顯性、可陳述	隱性、難以言說
範例	「西洋棋的規則是……」	「腳踏車要這樣平衡」
能否寫下	可以（規則、公式）	極難（你說得出怎麼騎車嗎？）

面向	抽象推理的知識	知覺行動的知識
獲得方式	學習、閱讀	練習、身體經驗
表示	符號、邏輯	感知運動的模式
對電腦	容易編碼	難以編碼

這正是第 32.2.3 節「知識工程的負擔」的根源。符號式方法要求把知識寫成明確的規則——而知覺行動的知識本質上就寫不出來。這不是我們還沒找到方法，而是這類知識的形式就不是符號的。

這也解釋了為何第 31 章的學習方法在此有效：既然寫不出來，就不要寫——讓系統從經驗中直接學習那個「說不出來的模式」。

33.1.4 第三層答案：身體的參與

但還有更深的一層，而這正是本章的核心主張：

◎ 第三層答案

知覺與行動之所以困難，是因為它們不只發生在「腦」裡——它們發生在「腦、身體、環境」的互動中。我們試圖用純計算的方式重現它們，等於是只複製了三分之一的系統。一隻蜜蜂的腦只有一百萬個神經元（不到人腦的十萬分之一），卻能完成我們的機器人做不到的飛行任務——因為牠的翅膀、複眼、體重分布，本身就在「計算」。

這個觀點帶來一個關鍵的推論：

若身體與任務不匹配 → 大腦必須做更多計算來補償

若身體與任務匹配 → 大量的計算可以被「省掉」

這正是 33.3 節形態計算的核心，也是本書多處實例的統一解釋。

33.2 具身認知

33.2.1 核心主張

具身認知 (Embodied Cognition) 是認知科學中的一個立場，其核心主張是：

認知不是「腦中的計算」，而是「身體在環境中的活動」

這是對傳統認知科學的根本挑戰。

傳統的觀點把認知視為資訊處理：感官輸入 → 大腦計算 → 動作輸出。在這個觀點下，身體

只是「輸入裝置」與「輸出裝置」，本身不參與思考。

具身認知則主張：**身體的結構直接塑造了認知的形式**。同樣的「智慧」在不同的身體中，會是不同的東西。

33.2.2 四個支柱

具身認知的主張可以拆解為四個支柱，本書中都有對應：

表 33-3 具身認知的四個支柱

支柱	主張	本書中的體現
身體塑造認知	認知的形式取決於身體的結構	第 21 章：軟手的抓取策略與硬手完全不同
環境是資源	環境不只是問題，也是計算的一部分	第 28 章：被動行走利用重力與地形
行動構成感知	感知不是被動接收，而是主動探索	第 22 章：主動視覺；33.4 節
即時互動優於內部表示	與其建立完整的世界模型，不如快速反應	第 26 章：DWA 的局部反應優於全域重規劃

33.2.3 一個具體的對照

以「抓取一個未知的物體」為例，比較兩種取徑：

取徑一（傳統計算觀點）：

1. 用視覺重建物體的完整三維模型。
2. 估計其質量、質心、摩擦係數。
3. 計算力封閉（第 21.3 節）並最佳化抓取品質。
4. 規劃無碰撞的軌跡（第 26 章）。
5. 以精確的力控制執行（第 20 章）。

取徑二（具身觀點）：

1. 用一隻柔軟、欠驅動的手（第 21.8 節）。
2. 粗略地移動到物體附近。
3. 閉合手指——機構會自動貼合物體的形狀。
4. 若滑動，材料的變形會自動增加摩擦（第 21.7.1 節）。

取徑二用了幾乎為零的計算，成功率卻可能更高。這不是因為它「更聰明」，而是因為它把計算的負擔轉移到了身體上。

◎ 這不是說計算不重要

必須澄清一個常見的誤解。具身認知不是主張「不需要計算」，而是主張「計算、身體與環境應該協同設計」。第 20 章的力控制、第 26 章的規劃、第 31 章的學習——這些都極為重要。具身觀點所補充的是：在投入計算資源之前，先問「這個問題能不能用身體解決」。這正是本書從第 20 章開始反覆出現的那句話——「在把問題丟給控制器之前，先問能不能用機構把問題變簡單」。

33.3 形態計算

33.3.1 身體如何「計算」

形態計算 (Morphological Computation) 是具身認知在工程上最具體的體現。它的定義是：

◎ 形態計算

由身體的物理結構 (形態) 本身所完成的功能，若用控制器實現則需要感測與計算。換句話說：身體「免費地」做了原本需要計算的事。

判斷的標準是：若把這個結構換成剛性的、簡單的版本，是否需要加上感測器與控制演算法才能達成同樣的功能？若是，那麼原本的結構就在做形態計算。

33.3.2 本書中的實例

回顧全書，形態計算的實例俯拾皆是——只是當時沒有用這個名字：

表 33-4 本書中的形態計算實例

實例	章節	身體做了什麼	否則需要什麼
被動動態行走	28.1.3	機構的動力學自然產生穩定步態	完整的步態規劃與平衡控制
欠驅動手指	21.8.2	連桿自動分配力量、包絡物體	多個力感測器 + 各指獨立控制
柔性指墊	21.7.1	變形貼合、滑動時自動增加摩擦	滑動偵測 + 即時力控制
RCC 順應裝置	20.8.2	順應中心落在尖端，自動對位	高精度視覺 + 力控制迴路
孔口倒角	20.9.3	把精確對準轉為落入範圍	定位精度提升 41 倍
串聯彈性驅動器	5.9、28.6.4	儲存並回收能量、吸收衝擊	高頻力控制 + 更大的馬達
顆粒堵塞夾爪	21.8.3	同時實現柔軟適應與剛性固定	可變剛度的複雜機構與控制

實例	章節	身體做了什麼	否則需要什麼
麥克納姆輪	23.5	機構解除非完整約束	複雜的路徑規劃（第 26.5 節）

這張表是全書一條隱藏主線的顯影。從第 5 章到第 28 章，我們反覆遇到「用機構解決問題」的例子——現在它們有了一個統一的名字。

33.3.3 三個層次的形態計算

形態計算可以分為三個層次，複雜度遞增：

層次一：被動的結構特性。最單純的形式——形狀、剛度、阻尼的設計。例如倒角、柔性指墊。這一層幾乎是免費的（不增加成本與複雜度），卻常帶來巨大的效益。

層次二：機構的耦合。透過連桿、彈簧、筋腱把多個自由度耦合起來，使它們自動協調。例如欠驅動手指、被動行走的腿。這一層需要巧妙的設計，但一旦設計完成，運作極為可靠。

層次三：與環境的動態耦合。利用環境的物理特性（重力、氣流、地形）作為系統的一部分。例如被動行走利用重力位能、鳥類利用氣流滑翔。這一層最為精妙，但也最難設計與分析。

33.3.4 形態計算的限制

但形態計算不是萬靈丹，它有三個明確的限制：

- **不可重新編程。**一個為抓球設計的柔性手，無法透過「改參數」變成適合抓平板的手。機構一旦製造出來就固定了——而軟體可以更新。
- **難以分析與驗證。**柔性結構的行為難以用第 13 章的剛體動力學描述，也難以用第 19 章的方法證明穩定性。
- **設計困難。**要設計出「恰好能自動完成某功能」的機構，需要深厚的直覺與大量的試誤——這比寫一個控制器困難得多。

因此正確的態度是「協同設計」：用形態計算處理那些穩定不變、且需要快速反應的部分（如接觸的順應性）；用控制與學習處理那些需要彈性與適應的部分（如任務的切換）。

33.4 感知—行動迴路

33.4.1 感知不是被動的

傳統的觀點把感知視為單向的資訊流：世界 → 感測器 → 處理 → 理解。但具身觀點主張：感知本質上是一個迴路。

行動 → 改變感知 → 影響下一個行動 → 再改變感知 → ...

感知與行動互相構成，無法分割。

三個具體的體現：

- 你的眼睛不停地在動。人眼每秒有 3~4 次跳視 (saccade) ——你「看見」的完整畫面，其實是主動掃描後拼湊出來的。被動盯著看一個點，你反而什麼都看不清楚（除了中央凹的一小塊）。
- 你摸東西時手會動。要判斷一個表面的粗糙度，你會來回摩擦而非靜止按壓。靜止的觸覺幾乎無法分辨材質。
- 你會走過去看。若視線被遮擋，你不會用演算法推測遮擋後面是什麼——你會移動身體換一個角度。

33.4.2 主動感知

主動感知 (Active Perception) 把這個洞見變成工程方法：與其被動地處理接收到的資料，不如主動採取行動以獲取更好的資料。

表 33-5 被動感知與主動感知的對照

問題	被動感知的作法	主動感知的作法	本書章節
物體被遮擋	用推論或先驗補全	移動到另一個視角	第 22 章
不知道物體多重	由外觀估計質量	輕輕提起試試	第 21 章
深度不確定	用單目深度估計	移動相機產生視差	第 22.4.2 節
定位不確定	增加濾波器的複雜度	主動走向地標	第 24、25 章
不知道抓不抓得住	計算力封閉品質	先夾一下看會不會滑	第 21 章

主動感知常常比被動的演算法改良更有效。以「判斷物體重量」為例：由影像估計質量的誤差可能達 50%，而輕輕提起試一下的誤差只有幾個百分點。

這也是第 24.1.3 節「綁架機器人問題」的最佳解——與其設計更複雜的濾波器，不如主動移動到一個特徵明顯的位置重新定位。

33.4.3 探索與具身經驗

感知—行動迴路還有一個更深的含義：機器人可以透過行動來「學習世界是什麼」。

一個嬰兒如何知道「球會滾、積木不會」？不是有人告訴他，而是他推過、丟過、看過結果。這個過程稱為具身探索 (embodied exploration)。

在機器人上，這對應到：

- **互動式感知**：推一下物體看它怎麼動，從而判斷它的質量、摩擦與是否被固定。這比純視覺可靠得多。
- **自我監督學習**：機器人自己嘗試並記錄結果 (第 31.3.2 節)，不需要人工標註。
- **好奇心驅動的探索**：主動尋找「預測不準」的情況——因為那裡才有新資訊。這是強化學習中處理稀疏獎勵的重要方法。

◎ 為何這對第 32 章的大型模型是個提醒

第 32 章的語言模型「知道」杯子會破——因為它讀過。但它從未經歷過。這個差異在本書的脈絡下有具體的後果：它沒有物理直覺 (32.3.3 節的限制二)。它知道「玻璃會破」這個命題，卻不知道「這個角度施力 5 牛頓會不會破」——因為後者不是一個能從文字中學到的知識，而是必須從身體經驗中獲得的。這正是為何目前最有效的架構仍是混合式：語言模型提供從文字學到的常識，而具身的經驗 (無論是模擬或真機) 提供物理的直覺。兩者缺一不可。

33.5 三種典範的辯證

33.5.1 人工智慧的三條路線

要理解具身觀點的定位，必須把它放在人工智慧發展史的脈絡中。三種典範依序登場，而每一種都是對前一種局限的回應：

表 33-6 人工智慧的三種典範

典範	盛期	核心主張	代表成果	遇到的困境
符號主義	1960~1980s	智慧 = 符號的操作與推理	專家系統、下棋程式	符號接地問題、常識無法窮舉
連結主義	1990s~至今	智慧 = 從資料中學習的統計模式	深度學習、大型模型	沒有保證、需要海量資料、缺乏物理直覺
具身觀點	1990s 興起	智慧 = 身體與環境的互動	行為式機器人、軟體機器人	難以擴展到高階認知、設計困難

33.5.2 Brooks 的挑戰

具身觀點在機器人學中的關鍵人物是 **Rodney Brooks**。他於 1990 年發表了一篇著名的論文，標題極為挑釁：《大象不下棋》（Elephants Don't Play Chess）。

他的論點是：

◎ Brooks 的核心主張 (1986~1991)

「智慧不需要表示。」（Intelligence without representation）傳統的機器人架構是「感知 → 建立世界模型 → 規劃 → 行動」，Brooks 主張這個中間的「世界模型」是不必要的，甚至是有害的——因為世界本身就是最好的模型（The world is its own best model）。他提出的包容式架構（Subsumption Architecture）把行為分成多個並行的層次，每一層直接把感知連到行動，不經過中央的世界模型。他用這個架構做出的機器人（如 Genghis）能在真實環境中穩健地行走，而當時基於世界模型的機器人在實驗室外幾乎寸步難行。

這個主張在當時極具衝擊，因為它挑戰了整個符號主義的基礎。而它的實證支持也很有力——簡單的反應式機器人在真實世界中的表現，往往勝過複雜的規劃式機器人。

33.5.3 三者的辯證與整合

但三十年後，我們可以看得更清楚：三種典範各自處理不同層次的問題，而完整的系統需要三者。

本書的架構本身就是這個整合的體現（回顧第 32.8.4 節的五層）：

表 33-7 三種典範在本書架構中的位置

層級	主導的典範	本書章節	為何是這個典範
任務理解與規劃	符號 + 連結	第 32 章	需要常識（連結）與正確性保證（符號）
感知與辨識	連結主義	第 22、31 章	難以寫出規則，但有大量資料
運動規劃	符號 / 幾何	第 26 章	有明確的數學結構與保證
運動控制	符號 / 模型	第 16~20 章	物理定律明確、需要即時保證
接觸與互動	具身	第 20、21、28 章	用機構的柔順取代感測與計算
身體設計	具身	第 4、5、21 章	形態計算：讓身體分擔計算

最後兩列是本書一條隱藏的主線。從第 4 章討論機構、第 5 章的驅動器選型、到第 21 章的

軟性夾爪與第 28 章的被動行走——「身體的設計」始終與「演算法的設計」同等重要。

◎ 為何典範之爭是假問題

科技史上常見「A 取代 B」的敘事——符號主義失敗了、深度學習贏了、接下來輪到具身智慧。但這個敘事幾乎總是錯的。真實的情況是：每個典範都解決了前者無法處理的問題，同時暴露了自己的局限。符號主義給了我們形式化與保證（第 26、32 章的規劃器至今仍在用）；連結主義給了我們泛化與感知（第 22、31 章）；具身觀點給了我們對「身體參與智慧」的理解（第 20、21、28 章）。一個真實的機器人系統，三者缺一不可——而知道哪一層該用哪一個，正是本書最想傳達的工程判斷力。

33.6 基礎模型與機器人

33.6.1 「機器人的 GPT 時刻」會來嗎

自然語言處理領域在 2020 年前後經歷了一次典範轉移：與其為每個任務訓練一個模型，不如訓練一個通用的大模型再微調。這種模型稱為**基礎模型**（Foundation Model）。

機器人領域自然會問：同樣的事會發生在機器人上嗎？

支持的論點：

- 第 32.7 節的 VLA 已展現了跨任務的泛化——RT-2 能理解「把水果放到能裝水的容器裡」，即使訓練資料中沒有這個組合。
- Open X-Embodiment 等跨機構的資料集正在建立，規模持續擴大。
- 視覺與語言的基礎模型可以直接遷移——機器人不需要從零學會「什麼是杯子」。

懷疑的論點：

- 資料的規模差了幾個數量級。網路上有數兆個詞元，而機器人資料集只有數百萬筆軌跡——而且無法從網路上取得（第 31.2.2 節）。
- 具身的多樣性。每台機器人的形態、感測器、動力學都不同。語言的「詞」是通用的，但「動作」不是——一個為七軸手臂學到的動作，無法直接用在六軸手臂上。
- 物理的不可壓縮性。語言可以「說得含糊」，但動作必須精確到公釐與毫秒。錯誤的代價不對稱（第 31.2.2 節）。

33.6.2 一個折衷的判斷

目前較合理的判斷是：基礎模型會顯著改變機器人的「上層」，但「下層」仍需傳統方法。

上層（理解、規劃、常識）→ 基礎模型帶來根本的改變

下層（控制、接觸、安全）→ 物理定律與即時性的要求不變

這正是第 32.8.4 節五層架構的預測。

具體的理由：上層處理的是「語意與判斷」——這正是基礎模型的強項，且對延遲不敏感（可以想一秒）。下層處理的是「物理與時序」——需要 1 kHz 的確定性運算與可證明的穩定性，這是神經網路的弱項。

33.6.3 資料的瓶頸

若基礎模型的路線要成功，資料是最關鍵的瓶頸。目前有四條路：

表 33-8 機器人資料的四條路徑

路徑	作法	優勢	限制
模擬	大規模平行模擬（第 30 章）	量大、便宜、安全	Sim-to-Real 落差
遙操作示範	人類遙控機器人收集資料	真實、高品質	極為昂貴（每筆需人力）
人類影片	從網路影片學習人的動作	量極大	沒有動作標註、形態不同
機器人自主探索	機器人自己嘗試（第 33.4.3 節）	真實、可持續	慢、且需要安全機制

目前的共識是四者並用：用模擬取得規模、用遙操作取得品質、用影片取得多樣性、用自主探索持續改善。

33.7 通往通用機器人的三條路徑

33.7.1 三種賭注

「通用機器人」——能像人一樣執行多樣任務的機器人——是這個領域的長期目標。目前有三條主要的路徑，代表三種不同的賭注：

表 33-9 通往通用機器人的三條路徑

路徑	核心主張	代表方向	主要風險
規模路線	「資料與算力夠大就會湧現」	VLA、基礎模型、大規模預訓練	資料瓶頸、物理精度不足
結構路線	「架構與模型才是關鍵」	分層架構、模型式控制加學習	設計複雜、泛化受限

路徑	核心主張	代表方向	主要風險
身體路線	「更好的身體勝過更好的演算法」	軟體機器人、形態計算、仿生設計	難以擴展到高階認知

33.7.2 三條路徑的檢視

規模路線是目前最受矚目的。它的論據是自然語言處理的成功先例——當模型與資料規模跨越某個門檻，能力會「湧現」。

但機器人有一個語言沒有的限制：**物理不能靠規模繞過**。無論模型多大，若機器人的手指精度只有 5 公釐，就無法完成需要 1 公釐容差的裝配（第 20.9 節）。這是硬體的限制，不是資料的限制。

結構路線主張精心設計的分層架構——正是本書第 32.8.4 節所呈現的。它的優勢是**每一層都可驗證**，且能結合各典範的長處。缺點是**設計的負擔重**，且模組之間的介面需要人為定義。

身體路線則主張：與其讓演算法變聰明，不如讓身體變得更適合任務。第 33.3 節的所有實例都支持這個方向——**一隻好的手，能讓平庸的演算法表現優異**。

◎ 一個歷史的教訓

這三條路徑的爭論，在機器人學史上已經出現過多次，而每一次的答案都是「都需要」。1980 年代有人相信只要專家系統夠完整就能通用；1990 年代 Brooks 主張丟掉世界模型；2010 年代深度學習興起時，也有人預測「端到端會取代一切」。每一次，單一路線都在某個範圍內取得成功，然後撞上自己的邊界。本書的立場是：**三條路徑處理的是不同的瓶頸**——規模解決「常識與泛化」、結構解決「保證與整合」、身體解決「接觸與效率」。一個真正通用的機器人，很可能三者都需要。

33.7.3 還缺什麼

誠實地說，目前的機器人距離「通用」仍有明顯的距離。四個最關鍵的缺口：

- **靈巧操作**：第 21.10.2 節的手內操作至今仍極為笨拙。人類能在手中轉筆、能單手打開瓶蓋——這些看似簡單的動作，機器人做不到。
- **長時序任務**：目前的系統能執行數十秒的任務，但「整理一個房間」需要數十分鐘與數百個步驟，且中間會不斷遇到意外。
- **真正的物理直覺**：第 33.4.3 節的知識框已指出——知道「玻璃會破」與知道「這個力道會不

會破」是兩回事。

- **可靠度**：實驗室的 90% 成功率，在真實部署中是不可接受的。**要達到工業級的 99.9%**，往往比從 0% 到 90% 更難。

最後一項特別值得強調。第 30、31 章反覆提到的「沒有保證」問題，在此成為部署的最大障礙。一台每一百次會失敗一次的家用機器人，若失敗意味著打翻熱湯——它就無法上市。

33.8 機器人與社會

33.8.1 就業：一個需要細緻討論的議題

「機器人會不會搶走工作」是最常被問到的問題，也是最容易被簡化的問題。

歷史的證據是混合的。過去兩百年的自動化並未造成長期的大規模失業——但它確實造成了劇烈的結構性轉移，且轉移的成本由特定的群體承擔。

表 33-10 自動化對工作的三種影響

影響	說明	機器人領域的例子
取代 (Substitution)	機器完全接手該工作	產線的重複性焊接、搬運
互補 (Complementation)	機器承擔部分，人的生產力提高	外科手術機器人、協作機器人
創造 (Creation)	產生原本不存在的工作	機器人維護、系統整合、資料標註

關鍵的判斷不是「會不會取代」，而是「哪些任務會被取代」。近年的研究指出，自動化影響的是任務而非職業——多數職業由多項任務構成，其中部分可被自動化、部分不能。

對機器人領域而言，一個具體的觀察是：

最容易被自動化的：重複、可預測、在結構化環境中的體力工作

最難被自動化的：需要靈巧操作、應變、或人際互動的工作

這正是 Moravec 悖論的社會版本——那些「低技能」的工作（如清潔、照護）反而最難自動化，因為它們需要的正是知覺與行動的能力。

33.8.2 自動化的悖論

有一個常被忽略的現象，稱為**自動化悖論** (Automation Paradox)：

◎ 自動化悖論

系統的自動化程度越高，剩下那少數需要人介入的時刻就越關鍵、也越困難。因為：一、人的技能

會退化（長期不操作就生疏）；二、需要介入的必定是異常情況（正常的都被自動化了）；三、人失去了情境覺察（不知道系統剛才在做什麼）。航空業對此有深刻的教訓——多起事故的原因是自動駕駛在異常時交還控制權，而飛行員無法在數秒內重建情境並正確反應。

這對機器人的設計有直接的啟示：

- 不要設計「平常完全自動、出事才叫人」的系統——那個「叫人」的瞬間往往來不及。
- 應保持人的參與度（第 32.9.3 節的人在迴路）——即使只是定期的確認。
- 介面必須快速地傳達情境——不只是「出錯了」，而是「剛才發生了什麼、現在是什麼狀態」。

33.8.3 責任歸屬

當機器人造成損害時，誰該負責？這個問題在法律上仍未有定論，但可以辨識出幾個關鍵的角色：

表 33-11 責任歸屬的可能對象

對象	何時應負責	困難之處
製造商	設計缺陷、未盡安全義務	如何界定「合理的安全」
軟體開發者	演算法錯誤、未做適當測試	學習式系統的行為無法完全預測
部署者 / 雇主	使用不當、未做風險評估	是否具備判斷的專業
操作者	誤用、忽視警告	自動化悖論下的技能退化
機器人本身	（目前法律上不成立）	無法律人格、無資產

最困難的是第二列。第 31.10.1 節指出，學習式系統「在訓練分布之外的行為不可預測」——那麼開發者對「無法預測的行為」該負多少責任？

目前的實務傾向是：要求可追溯性與盡職義務。開發者不需要保證「永不出錯」，但必須能證明：做了合理的測試、有適當的安全機制、且事故發生後能追溯原因。這正是第 29.9.3 節「每次測試都錄 rosbag」的深層理由——它不只是除錯工具，也是責任的證據。

33.9 倫理與風險

33.9.1 四類倫理風險

除了就業與責任，機器人還帶來四類需要正視的風險：

表 33-12 機器人的四類倫理風險

風險	具體表現	本書中的相關章節
安全	碰撞傷害、失控、系統故障	第 7、20、27、28 章
隱私	持續的影像與位置紀錄	第 22、25 章
偏見	對特定群體的辨識或服務品質較差	第 22、31 章
自主與尊嚴	照護機器人取代人際接觸、決策權轉移	第 1 章的自主性光譜

33.9.2 隱私：一個被低估的問題

一台家用機器人為了導航與操作，必須**持續地掃描家中的環境**（第 25 章的 SLAM）。這意味著：

- 它建立了你家的**完整三維地圖**——包含每個房間的格局與家具擺設。
- 它的相機**看見了一切**——包含你在做什麼、家裡有誰、桌上放著什麼。
- 這些資料若上傳雲端（第 32.10.1 節的雲端模型），就**離開了你的控制**。

這比手機更為敏感，因為機器人會**主動移動**——它會走到你沒預期的地方，且是**持續而非間歇地**記錄。

工程上的因應（呼應第 32.10.1 節）：

- **優先在本地處理**：把影像處理留在機器人上，只上傳必要的抽象結果（如「已完成任務」而非原始影像）。
- **明確的指示燈**：讓使用者知道相機何時在運作。
- **資料最小化**：只保留任務所需的資料，用完即刪。
- **可控的禁區**：允許使用者指定「不進入、不記錄」的區域。

33.9.3 偏見：不只是資料問題

第 31 章的學習方法會**繼承訓練資料中的偏見**。在機器人上，這有具體的後果：

- **辨識的不平等**：若訓練資料中某些膚色的樣本較少，人員偵測的準確率會較低——而這直接關係到安全（第 20.10 節的碰撞避免）。
- **互動的不平等**：語音辨識對某些口音的準確率較低，使部分使用者難以操作。
- **設計的不平等**：機器人的高度、力道、介面若以特定的身材或能力為基準設計，會排除其他

使用者。

最後一項尤其值得注意——它不是「演算法偏見」而是**設計偏見**，且往往在需求階段就已注定。若測試者都是實驗室的年輕研究生，就不會發現長者難以使用的問題。

33.9.4 一個工程師的檢核清單

面對這些問題，一個實務的作法是在專案中定期檢視以下問題：

1. 失效時會怎樣？最壞的情況是什麼？有沒有機制阻止它？（第 7.9、31.10.2、32.9 節）
2. 誰可能受影響？不只是使用者，還有旁觀者、被服務者、以及被取代的工作者。
3. 測試涵蓋了誰？測試的樣本是否涵蓋了實際使用者的多樣性？
4. 資料去了哪裡？收集了什麼、儲存多久、誰能存取？
5. 人能介入嗎？使用者能否理解系統在做什麼、並在必要時停止它？
6. 出事後能追溯嗎？是否記錄了足夠的資料以釐清原因？

這六個問題不需要哲學訓練就能回答，但認真回答它們，能避免絕大多數的倫理問題。

33.10 全書的主線回顧

33.10.1 六條貫穿全書的線索

在結束之前，讓我們回顧貫穿三十三章的六條主線。這些線索比任何單一的公式都更值得記住：

線索一：模型的價值不在完美，而在減負。第 13 章的動力學模型有 10~20% 的誤差，第 17 章仍能靠它把回授的負擔降低五倍；第 30 章更指出「模擬不必更真，策略必須更穩」。不完美的模型仍然極有價值。

線索二：分層與時間尺度分離。第 3 章的時間尺度金字塔、第 17 章的內外迴路、第 27 章的四層飛控、第 28 章的三層腿式控制、第 32 章的五層任務系統——同一個原則反覆出現：讓每一層只處理自己時間尺度上的問題，並把下層視為理想的執行器。

線索三：與其消除不確定性，不如設計對它不敏感的系統。第 20 章的柔順控制、第 21 章的軟性夾爪、第 24 章的機率估測、第 30 章的領域隨機化——這是本書最重要的工程哲學。

線索四：機構與控制必須協同設計。第 20 章的倒角把容差放寬 41 倍、第 21 章的欠驅動手

取代了感測與計算、第 28 章的被動行走不需要控制器——在把問題丟給控制器之前，先問能不能用機構把問題變簡單。這正是本章 33.3 節形態計算的統一解釋。

線索五：模型提供保證，學習提供泛化。第 21 章的抓取、第 25 章的 SLAM、第 28 章的運動控制、第 31 章的學習架構、第 32 章的任務規劃——每一次的結論都是「兩者分工」而非「誰取代誰」。

線索六：理論的各部分互相支撐。第 13 章那個「看似無用」的斜對稱性，在第 19 章成為證明的樞紐；第 11 章的雅可比對偶，先後成為第 17 章操作空間控制、第 20 章力映射、第 21 章抓取矩陣的基礎；第 18 章的 Bellman 方程，在第 31 章以強化學習的形式重現。這不是巧合，而是理論的內在結構。

33.10.2 三十三章的地圖

最後，用一張表回顧全書的架構：

表 33-13 全書架構總覽

篇	章	核心問題	關鍵洞見
導讀篇	1~2	機器人是什麼？該怎麼學？	Moravec 悖論、自主性光譜
第一篇	3~7	機器人由什麼構成？	時間尺度金字塔、機構與驅動的取捨
第二篇	8~12	它能到哪裡？	座標變換、逆解的多重性、奇異點
第三篇	13~21	該用多大的力氣？	模型加回授、最佳性與穩定性、柔順接觸
第四篇	22~28	它在哪裡？該往哪走？	機率估測、SLAM 的聯合求解、分層導航
第五篇	29~33	它知道自己在做什麼嗎？	模擬與學習、大型模型、具身智慧

注意每一篇的核心問題如何層層遞進——由「是什麼」到「能到哪」、由「怎麼動」到「在哪裡」、最終到「知道自己在做什麼」。

33.11 結語：給讀者的話

33.11.1 你學到了什麼

三十三章之後，你已經具備了完整的機器人學基礎：能建立運動學與動力學模型、能設計並證明控制器、能處理感知的不確定性、能規劃路徑與任務、能運用學習方法、也能整合為一個完整的系統。

但更重要的可能不是這些知識本身，而是三種判斷力：

- 知道何時該用哪個工具——模型或學習、精確或柔順、規劃或反應。
- 知道每個方法的邊界——它在什麼假設下成立、在什麼情況下會失效。
- 知道問題的本質——很多時候，正確的答案不是更好的演算法，而是重新定義問題。

33.11.2 這個領域還很年輕

讀完一本教科書容易產生一個錯覺：這個領域已經成熟了。但機器人學遠非如此。

回顧本書，有太多地方寫著「目前仍是研究前沿」：手內操作、長時序任務、變形物體、可靠度、以及本章討論的通用機器人。這些不是留給後人的習題，而是留給你的。

一個提醒：本書中那些看起來最成熟的部分（如第 9 章的運動學），在四十年前也是前沿；而那些現在最熱門的部分（如第 32 章的大型模型），十年後可能已被更好的方法取代。唯一不變的是那六條主線——它們是三十年後仍然適用的思維方式。

33.11.3 最後一段

第 1 章開頭，我們問：為什麼電腦下棋能贏世界冠軍，卻摺不好一件衣服？

現在你知道答案了：因為摺衣服需要的不只是計算，還需要一雙能感受布料的手、一個能與環境互動的身體、以及無數次「試了才知道」的經驗。

而這正是機器人學迷人的地方——它是唯一一個必須同時面對數學、物理、與真實世界的學科。你可以在紙上證明一個控制器穩定，但機器人會用它的行為告訴你，你的模型漏掉了什麼。

這種「現實會回應你」的特質，是機器人學最好的老師。

願你在往後的路，保持那份「想看看它到底會怎麼動」的好奇心。

那才是機器人學真正的起點。

綜合思考題

本章沒有計算例題，取而代之的是四個綜合性的思考題。它們沒有標準答案，但認真思考的過程，會把前面三十三章的知識連結起來。

思考題 33-1 形態計算的辨識與設計

題目：以下三個情境，各自可以用「加強演算法」或「改良機構」來解決。請為每一個提出兩種方案並比較。

情境一：一台移動機器人在磁磚地面上打滑，里程計誤差極大（第 23.8 節）。

參考分析：

- **演算法路線：**加裝 IMU 並用第 24 章的融合估測、或用第 25 章的 SLAM 提供絕對參照。優點是不需改硬體；缺點是增加了計算與感測成本，且打滑時的里程計資料本身就是錯的（垃圾進、垃圾出）。
- **機構路線：**換用摩擦係數更高的輪胎、增加配重以提高正向力、或改用第 23.6 節的履帶。優點是從源頭減少打滑；缺點是增加重量與能耗。
- **判斷：**兩者應並用。機構改善減少了誤差的來源，融合估測處理剩餘的誤差。單靠演算法無法補償嚴重的打滑（第 23.8.2 節的隨機誤差無法校正）。

情境二：機械手臂在插銷入孔時，因定位誤差而卡住（第 20.9 節）。

情境三：四足機器人在碎石地面上容易失去平衡（第 28 章）。

請自行分析情境二與三，並回答：在什麼情況下，機構路線的投資報酬率會高於演算法路線？

提示：考慮改變的頻率（機構固定、演算法可更新）、以及第 33.3.4 節的三個限制。

思考題 33-2 Moravec 悖論的當代檢驗

題目：2020 年之後，大型語言模型在許多「智力測驗」上超越了人類平均水準。請分析：

1. 這是否強化了 Moravec 悖論？（提示：抽象推理又進步了，而知覺行動仍然困難）
2. 第 32.7 節的 VLA 模型能由影像直接輸出動作。這是否代表悖論正在被打破？
3. 若要設計一個實驗來檢驗 Moravec 悖論是否仍然成立，你會怎麼設計？需要控制哪些變因？

一個參考的思考方向：注意 VLA 目前的成功案例多集中在「桌面上的簡單取放」——這與五歲孩子能做的事（摺衣服、綁鞋帶、在陌生房間移動）仍有明顯差距。差距在哪裡？是資料量、是身體、還是別的？

思考題 33-3 為一個真實場景做完整的系統設計

題目：為「醫院中運送檢體與藥品的服務機器人」做完整的系統設計。請涵蓋：

1. **技術架構**：依第 32.8.4 節的五層列出各層的技術選擇與理由。
2. **硬體選型**：移動平台（第 23 章）、感測器（第 6、22 章）、是否需要手臂（第 4、5 章）。
3. **安全設計**：三層防護（第 32.9 節）、失效行為（第 7.9 節）、與人共處的考量（第 20.10 節）。
4. **倫理檢核**：用 33.9.4 節的六個問題檢視你的設計。
5. **社會影響**：這台機器人會取代什麼工作？會創造什麼工作？醫護人員會如何看待它？

這個题目的價值在於「沒有純技術的答案」。一個技術上完美但護理人員不願使用的系統，是失敗的設計。

思考題 33-4 你的立場是什麼

題目：回答 33.7 節的三條路徑之爭——你認為通往通用機器人的關鍵是什麼？

請以本書的具體內容支持你的論點，並誠實地說明你的立場可能錯在哪裡。

一個提示：最好的答案往往不是「選一個」，而是能清楚地說明「在什麼條件下，哪一條路徑會勝出」。例如：若資料收集的成本大幅下降，規模路線的勝算會提高；若材料科學有突破（如更好的人工肌肉），身體路線的價值會上升。

實作：回顧、整合與展望

本章的「實作」不是寫程式，而是**回顧與整合**。以下九項活動，建議在課程結束時進行。

1. **繪製你的知識地圖**：不看書，憑記憶畫出全書三十三章的架構圖，標出各章之間的連結。畫完後再與表 33-13 對照，看看哪些連結你記得、哪些遺漏了。遺漏的部分正是需要複習的。
2. **找出五個形態計算的實例**：除了表 33-4 列出的，在你的生活或專案中再找出五個「用結構取代計算」的例子。可以是機器人的、生物的、或日常用品的。
3. **重讀第 1 章**：讀完全書後再讀一次第 1 章。你會發現當時看不懂或覺得空泛的段落，現在都有了具體的意義。記下你的感受變化。
4. **分析一個真實的機器人**：選一台你感興趣的機器人（掃地機器人、協作手臂、四足機器人、或自駕車），試著推測它用了本書的哪些技術。特別注意它「沒有做」的事——那往往反映了工程上的取捨。

5. **設計一個形態計算的解法**：選一個你曾經用演算法解決的問題，試著設計一個「用機構解決」的替代方案。比較兩者的成本、可靠度與彈性。
6. **做一次倫理檢核**：用 33.9.4 節的六個問題檢視你做過的任何一個專案（不限機器人）。誠實地回答，並記下你發現的問題。
7. **寫下三個你想解決的問題**：讀完全書後，有哪三個機器人問題是你最想解決的？為每一個寫下：為何重要、目前的困難、以及你會從哪裡開始。
8. **與人討論**：找一位非工程背景的朋友或家人，用五分鐘解釋「為什麼機器人摺衣服很難」。若你能講清楚，代表你真的懂了。
9. **（給教師）課程回顧討論**：以本章的四個思考題為基礎，安排一次沒有標準答案的討論課。重點不是得出結論，而是讓學生練習「用學過的知識支持自己的判斷」。

國中小也看得懂：聰明不只在腦袋裡

在這本書的第一頁，我們問過一個問題：為什麼電腦可以打敗世界棋王，卻不會摺衣服？

現在，讓我們來回答它。

先做一個實驗：閉上眼睛，用手在口袋或書包裡摸索，找出你的鑰匙。

你做到了對不對？很輕鬆，幾秒鐘就找到了。

但你有沒有想過，你剛才做了多少事情？你要分辨鑰匙和硬幣的形狀差別、要感覺金屬和塑膠的不同、要在摸到的時候調整手指的力道、還要判斷「這個形狀是不是我要的」——而這一切你完全不用思考，身體自己就會了。

這就是答案的第一部分：我們覺得簡單的事，其實超級複雜——只是我們的身體太熟練了，熟練到我們根本感覺不到。

答案的第二部分更有趣：因為你有一雙手。

想想看，你的手指是軟的、會變形、指腹有紋路增加摩擦、手掌可以彎起來包住東西。這些「身體的特性」本身就在幫你做事——你不需要用腦袋計算「該用幾牛頓的力」，因為軟軟的手指自己就會調整。

科學家發現了一件很重要的事：一隻蜜蜂的腦只有一百萬個腦細胞，比你的手機笨好幾億倍。

但牠可以在風中飛、可以找到花、還可以飛回家。為什麼？

因為牠的翅膀本身就會處理氣流，不需要用腦去算。

所以這本書最後想告訴你的是：

聰明，不只在腦袋裡，也在身體裡，還在身體跟世界互動的過程裡。

這就是為什麼，如果你想做出厲害的機器人，不能只會寫程式——你還要懂機構、懂材料、懂它會怎麼碰到這個世界。

最後，送你一句話：

這個世界上還有好多好多事情，是機器人不會做的。摺衣服、綁鞋帶、在陌生的房間裡走來走去不撞到東西——這些五歲小孩都會的事，全世界最厲害的科學家還在努力。

也許，解決其中一個人的人，就是正在讀這本書的你。

叫獸提醒與常見錯誤

- 誤區一：以為 Moravec 悖論只是「還沒做到」。它反映的是知識形式的根本差異——知覺行動的知識本質上難以符號化。
- 誤區二：把具身認知理解為「不需要計算」。它主張的是「計算、身體與環境應協同設計」，而非否定計算。
- 誤區三：忽略形態計算的可能性。在投入演算法之前，應先問「這個問題能不能用機構解決」。
- 誤區四：以為形態計算萬能。它不可重新編程、難以驗證、且設計困難，只適合穩定不變的需求。
- 誤區五：把感知視為被動的。主動感知（移動視角、輕推試探）常比改良演算法更有效。
- 誤區六：相信「新典範取代舊典範」的敘事。符號、連結、具身三者各有其層次，完整系統缺一不可。
- 誤區七：以為基礎模型會解決一切。它會改變上層的理解與規劃，但下層的控制與安全仍需傳統方法。
- 誤區八：低估資料瓶頸。機器人資料無法從網路取得，且具身的多樣性使遷移困難。

- 誤區九：把可靠度視為次要問題。從 90% 提升到 99.9% 往往比從 0% 到 90% 更難，卻是部署的門檻。
- 誤區十：設計「平常全自動、出事才叫人」的系統。自動化悖論指出，那個交還控制權的瞬間往往來不及。
- 誤區十一：忽略隱私問題。機器人會主動移動且持續記錄，其敏感度高於手機。
- 誤區十二：以為倫理是「別人的事」。33.9.4 節的六個問題不需要哲學訓練，但認真回答能避免多數問題。

課堂討論

1. 33.1.4 節主張知覺行動的困難源自「只複製了三分之一的系統」。請討論：若要「複製另外三分之二」，工程上具體該做什麼？
2. 表 33-4 列出了本書中八個形態計算的實例。請討論它們的共同結構，並嘗試提出一個判準——什麼樣的問題適合用形態計算解決？
3. 33.5.3 節主張三種典範各有其層次。請討論：有沒有可能出現第四種典範？它會處理什麼目前無法處理的問題？
4. 33.8.2 節的自動化悖論指出「自動化程度越高，人的介入越關鍵也越困難」。請以本書中的具體系統（如第 27 章的無人機、第 32 章的語言驅動系統）為例，設計一個能緩解這個悖論的人機介面。
5. 33.11.3 節說「現實會回應你，這是機器人學最好的老師」。請分享一個你在實作中「模型與現實不符」的經驗，以及你從中學到了什麼。

章末摘要

- Moravec 悖論有三層答案：演化的時間（知覺行動優化了 5 億年）、知識的形式（隱性且難以符號化）、以及身體的參與。
- 第三層是最深的：知覺與行動發生在「腦、身體、環境」的互動中，純計算只複製了三分之一的系統。
- 具身認知的四個支柱：身體塑造認知、環境是資源、行動構成感知、即時互動優於內部表示。

- 具身認知不否定計算，而是主張計算、身體與環境應協同設計。
- 形態計算是指由身體結構本身完成的功能——若換成剛性簡單的版本則需要感測與計算。
- 本書中的形態計算實例包含：被動行走、欠驅動手指、柔性指墊、RCC、倒角、串聯彈性驅動器、顆粒堵塞夾爪。
- 形態計算分三個層次：被動結構特性、機構的耦合、與環境的動態耦合。
- 形態計算的三個限制：不可重新編程、難以驗證、設計困難；因此應與控制協同設計。
- 感知不是被動接收而是一個迴路——行動改變感知、感知影響行動。
- 主動感知（移動視角、輕推試探）常比改良被動演算法更有效。
- 人工智慧的三種典範：符號主義（形式化與保證）、連結主義（泛化與感知）、具身觀點（身體參與智慧）。
- 三種典範各處理不同層次，完整的機器人系統三者缺一不可。
- 基礎模型會顯著改變機器人的上層（理解與規劃），但下層（控制與安全）仍需傳統方法。
- 機器人資料的四條路徑：模擬、遙操作示範、人類影片、自主探索；目前需四者並用。
- 通往通用機器人的三條路徑：規模、結構、身體；三者處理不同的瓶頸，很可能都需要。
- 目前的四個關鍵缺口：靈巧操作、長時序任務、物理直覺、以及可靠度。
- 自動化影響的是任務而非職業；最難自動化的反而是需要靈巧與應變的工作（Moravec 悖論的社會版本）。
- 自動化悖論：自動化程度越高，剩下需要人介入的時刻越關鍵也越困難。
- 責任歸屬的實務傾向是要求可追溯性與盡職義務，而非保證永不出錯。
- 四類倫理風險：安全、隱私、偏見、自主與尊嚴；六個檢核問題能避免多數問題。
- 全書的六條主線：模型的價值在減負、分層與時間尺度分離、設計對不確定性不敏感的系統、機構與控制協同設計、模型提供保證而學習提供泛化、理論各部分互相支撐。

觀念題與綜合題

觀念題

1. 陳述 Moravec 悖論，並說明其三層答案。
2. 說明演化的時間差如何解釋「我們覺得容易的事對電腦卻困難」。

3. 比較抽象推理與知覺行動兩種知識的形式，並說明這對編碼方式的影響。
4. 說明具身認知的核心主張及其對傳統認知科學的挑戰。
5. 列舉具身認知的四個支柱，並各舉一個本書中的例子。
6. 定義形態計算，並說明判斷的標準。
7. 由表 33-4 中選三個實例，說明「身體做了什麼」與「否則需要什麼」。
8. 說明形態計算的三個層次與三個限制。
9. 解釋為何說「感知是一個迴路而非單向的資訊流」，並舉三個人類的例子。
10. 比較被動感知與主動感知，並說明主動感知為何常更有效。
11. 說明具身探索的意義，以及它對第 32 章大型模型的提醒。
12. 比較人工智慧的三種典範，並說明各自的核心主張與困境。
13. 說明 Brooks 的「智慧不需要表示」主張及其實證支持。
14. 為何說「典範之爭是假問題」？請以本書的架構說明三者的分工。
15. 說明基礎模型對機器人的可能影響，並列舉支持與懷疑的論點各三個。
16. 列舉機器人資料的四條路徑及其優缺點。
17. 比較通往通用機器人的三條路徑，並說明各自的主要風險。
18. 列舉目前機器人的四個關鍵缺口，並說明為何可靠度特別困難。
19. 說明自動化對工作的三種影響，以及「最難自動化的是什麼工作」。
20. 說明自動化悖論及其對機器人介面設計的三個啟示。
21. 列舉責任歸屬的可能對象，並說明學習式系統帶來的特殊困難。
22. 列舉四類倫理風險，並說明隱私問題為何在機器人上比手機更敏感。
23. 列出 33.9.4 節的六個檢核問題。
24. 陳述全書的六條主線，並各舉一個具體的章節例證。

綜合題

1. **跨章整合**：以第 21 章的軟性夾爪為例，說明它如何同時體現：形態計算（33.3 節）、對不確定性不敏感的設計（第 20、30 章）、以及機構與控制的協同（第 5 章）。並討論若改用剛性夾爪，需要額外增加哪些感測與計算。

2. **方法論的比較**：對「讓機器人學會開門」這個任務，分別設計三種方案：（甲）純模型式（第 20、26 章）、（乙）純學習式（第 31 章）、（丙）混合式。比較三者所需的開發成本、可靠度、與泛化能力，並說明你會選哪一個及理由。
3. **架構設計**：為一台「協助長者的居家機器人」設計完整的五層架構（第 32.8.4 節）。除技術選擇外，請特別說明：如何處理自動化悖論（33.8.2 節）、如何保護隱私（33.9.2 節）、以及如何避免設計偏見（33.9.3 節）。
4. **前瞻分析**：選擇本書提到的一個「目前仍是研究前沿」的問題（手內操作、長時序任務、變形物體操作、或可靠度），分析：（a）目前的主要障礙為何；（b）三條路徑（規模、結構、身體）分別會如何嘗試解決；（c）你認為哪一條最有希望，以及需要什麼條件才能成功。

動手做與專題延伸

本章實作：整合與展望

本章的實作是**回顧與連結**，詳見前面的「實作」一節。此處提供三個延伸的方向。

- **方向一：形態計算的實驗**。設計一組對照實驗，量化「機構改良」與「演算法改良」的效益。例如：同一個抓取任務，分別測試（甲）剛性夾爪加精密視覺、（乙）柔性夾爪加粗略視覺。比較成功率、成本與開發時間。
- **方向二：主動感知的驗證**。設計一個實驗比較被動與主動感知。例如：判斷未知物體的重量——（甲）純視覺估計、（乙）輕輕提起試探。量化兩者的誤差差異。
- **方向三：完整系統的整合**。把前面各章的模組整合為一個完整的系統，並用 33.9.4 節的六個問題做倫理檢核。**這是本書所有內容的最終整合。**

專題延伸

- **形態計算的量化研究**：選擇一個具體的任務（建議：抓取形狀多變的物體）。設計並製作三種夾爪：剛性平行夾爪、加了柔性指墊的夾爪、以及欠驅動的自適應夾爪（第 21.8.2 節）。對同一組物體（至少 15 種形狀）各測試 20 次，記錄成功率、所需的定位精度、以及所需的感測與計算資源。繪出「機構複雜度 vs 演算法需求」的關係圖。這個專題能把本章最核心的主張變成一組具體的數據。
- **Moravec 悖論的當代測試**：設計一組任務，涵蓋由「純抽象推理」到「純知覺行動」的光譜

(例如：解數學題 → 下棋 → 辨識物體 → 抓取物體 → 摺衣服)。分別評估：目前的 AI 系統能達到什麼水準、五歲孩子能達到什麼水準。**繪出兩條曲線並找出交叉點**。討論這個交叉點在過去十年如何移動、以及未來可能如何移動。

- **進階：一個完整的畢業專題**。以本書的全部內容為基礎，設計並實作一個完整的機器人系統。建議的規格：（一）能接受自然語言指令（第 32 章）；（二）能在未知環境中導航（第 25、26 章）；（三）能抓取與操作物體（第 20、21 章）；（四）具備三層安全防護（第 32.9 節）；（五）通過 33.9.4 節的倫理檢核。**這個專題會用到本書幾乎每一章的內容——而完成它，就是你真正掌握了機器人學的證明。**

參考文獻與延伸閱讀

- Moravec, H., *Mind Children: The Future of Robot and Human Intelligence*, Harvard University Press, 1988 (Moravec 悖論的出處)。
- Brooks, R. A., "Intelligence Without Representation," *Artificial Intelligence*, 1991 (具身觀點在機器人學中的宣言)。
- Brooks, R. A., "Elephants Don't Play Chess," *Robotics and Autonomous Systems*, 1990 (包容式架構與對符號主義的批判)。
- Pfeifer, R. and Bongard, J., *How the Body Shapes the Way We Think: A New View of Intelligence*, MIT Press, 2006 (具身智慧的權威著作，形態計算的系統性論述)。
- Clark, A., *Being There: Putting Brain, Body, and World Together Again*, MIT Press, 1997 (具身認知的哲學基礎)。
- Gibson, J. J., *The Ecological Approach to Visual Perception*, 1979 (可供性概念的原始出處)。
- Bajcsy, R., Aloimonos, Y. and Tsotsos, J. K., "Revisiting Active Perception," *Autonomous Robots*, 2018 (主動感知的回顧與展望)。
- Firoozi, R. et al., "Foundation Models in Robotics: Applications, Challenges, and the Future," *IJRR*, 2024 (基礎模型在機器人上的綜述)。
- Autor, D. H., "Why Are There Still So Many Jobs? The History and Future of Workplace Automation," *Journal of Economic Perspectives*, 2015 (自動化對就業影響的經濟學分析)。

- IEEE Global Initiative on Ethics of Autonomous and Intelligent Systems, Ethically Aligned Design (自主系統倫理設計的國際指引) 。
- Lin, P., Abney, K. and Bekey, G. A. (Eds.), Robot Ethics: The Ethical and Social Implications of Robotics, MIT Press (機器人倫理的論文集) 。
- Bicchi, A. and Tonietti, G., "Fast and Soft Arm Tactics: Dealing with the Safety-Performance Tradeoff," IEEE Robotics & Automation Magazine, 2004 (安全與性能取舍的經典分析，呼應本章的形態計算) 。